

**LAPORAN AKHIR PRAKTIKUM  
PEMROGRAMAN MOBILE**



**Oleh:**

**Nazmi Hakim**

**NIM. 2310817210012**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
JUNI 2025**

## **LEMBAR PENGESAHAN**

### **LAPORAN AKHIR PRAKTIKUM PEMROGRAMAN MOBILE**

Laporan Akhir Praktikum Pemrograman Mobile ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Nazmi Hakim  
NIM : 2310817210012

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar  
NIM. 2210817210012

Ir. Eka Setya Wijaya, S.T., M.Kom.  
NIP. 198205082008011010

## DAFTAR ISI

|                                             |    |
|---------------------------------------------|----|
| LEMBAR PENGESAHAN .....                     | 2  |
| DAFTAR ISI.....                             | 3  |
| DAFTAR GAMBAR .....                         | 5  |
| DAFTAR TABEL .....                          | 7  |
| MODUL 1 : Android Basics With Compose ..... | 9  |
| SOAL 1 .....                                | 9  |
| A. Source Code.....                         | 11 |
| B. Output Program.....                      | 16 |
| C. Pembahasan .....                         | 19 |
| D. Tautan Git.....                          | 24 |
| MODUL 2 : Android Layout With Compose ..... | 25 |
| SOAL 1 .....                                | 25 |
| A. Source Code.....                         | 26 |
| B. Output Program.....                      | 34 |
| C. Pembahasan .....                         | 40 |
| D. Tautan Git .....                         | 43 |
| SOAL 2 .....                                | 44 |
| A. Pembahasan .....                         | 44 |
| MODUL 3 : Build A Scrollable List.....      | 45 |
| SOAL 1 .....                                | 45 |
| A. Source Code.....                         | 47 |
| B. Output Program.....                      | 61 |
| C. Pembahasan .....                         | 65 |
| D. Tautan Git .....                         | 76 |
| SOAL 2 .....                                | 77 |
| A. Pembahasan .....                         | 77 |
| MODUL 4 : Viewmodel and Debugging.....      | 78 |
| SOAL 1 .....                                | 78 |

|                 |                     |     |
|-----------------|---------------------|-----|
| A.              | Source Code.....    | 79  |
| B.              | Output Program..... | 97  |
| C.              | Pembahasan .....    | 105 |
| D.              | Tautan Git .....    | 109 |
| SOAL 2          | .....               | 110 |
| A.              | Pembahasan .....    | 110 |
| MODUL 5 : Array | .....               | 111 |
| SOAL 1          | .....               | 111 |
| A.              | Source Code.....    | 111 |
| B.              | Output Program..... | 143 |
| C.              | Pembahasan .....    | 147 |
| D.              | Tautan Git .....    | 163 |

## DAFTAR GAMBAR

|                                                                  |    |
|------------------------------------------------------------------|----|
| Gambar 1. Tampilan Awal Aplikasi .....                           | 9  |
| Gambar 2. Tampilan Dadu Setelah Di-Roll.....                     | 10 |
| Gambar 3. Tampilan Roll Dadu Double .....                        | 10 |
| Gambar 4. Screenshot Hasil Jawaban Soal 1 XML .....              | 16 |
| Gambar 5. Screenshot Hasil Jawaban Soal 1 XML .....              | 17 |
| Gambar 6. Screenshot Hasil Jawaban Soal 1 XML .....              | 17 |
| Gambar 7. Screenshot Hasil Jawaban Soal 1 Jetpack Compose .....  | 18 |
| Gambar 8. Screenshot Hasil Jawaban Soal 1 Jetpack Compose .....  | 18 |
| Gambar 9. Screenshot Hasil Jawaban Soal 1 Jetpack Compose .....  | 19 |
| Gambar 10. Tampilan Awal Aplikasi . .....                        | 25 |
| Gambar 11. Tampilan Pilihan Persentase Tip.....                  | 26 |
| Gambar 12. Screenshot Hasil Jawaban Soal 1 XML .....             | 35 |
| Gambar 13. Screenshot Hasil Jawaban Soal 1 XML .....             | 36 |
| Gambar 14. Screenshot Hasil Jawaban Soal 1 XML .....             | 37 |
| Gambar 15. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 38 |
| Gambar 16. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 39 |
| Gambar 17. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 40 |
| Gambar 18. Contoh UI List.....                                   | 46 |
| Gambar 19. Contoh UI Detail .....                                | 47 |
| Gambar 20. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 61 |
| Gambar 21. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 62 |
| Gambar 22. Screenshot Hasil Jawaban Soal 1 XML .....             | 63 |
| Gambar 23. Screenshot Hasil Jawaban Soal 1 XML .....             | 64 |
| Gambar 24. Contoh Penggunaan Debugger.....                       | 78 |
| Gambar 25. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 97 |
| Gambar 26. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 97 |
| Gambar 27. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 98 |
| Gambar 28. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 98 |
| Gambar 29. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 99 |

|                                                                  |     |
|------------------------------------------------------------------|-----|
| Gambar 30. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 99  |
| Gambar 31. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 100 |
| Gambar 32. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 100 |
| Gambar 33. Screenshot Hasil Jawaban Soal 1 XML .....             | 101 |
| Gambar 34. Screenshot Hasil Jawaban Soal 1 XML .....             | 101 |
| Gambar 35. Screenshot Hasil Jawaban Soal 1 XML .....             | 102 |
| Gambar 36. Screenshot Hasil Jawaban Soal 1 XML .....             | 102 |
| Gambar 37. Screenshot Hasil Jawaban Soal 1 XML .....             | 103 |
| Gambar 38. Screenshot Hasil Jawaban Soal 1 XML .....             | 103 |
| Gambar 39. Screenshot Hasil Jawaban Soal 1 XML .....             | 104 |
| Gambar 40. Screenshot Hasil Jawaban Soal 1 XML .....             | 104 |
| Gambar 41. Screenshot Hasil Jawaban Soal 1 XML .....             | 105 |
| Gambar 42. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 143 |
| Gambar 43. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 144 |
| Gambar 44. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 145 |
| Gambar 45. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 146 |
| Gambar 46. Screenshot Hasil Jawaban Soal 1 Jetpack Compose ..... | 147 |

## DAFTAR TABEL

|                                                            |    |
|------------------------------------------------------------|----|
| Tabel 1. Source Code Jawaban Soal 1 XML .....              | 11 |
| Tabel 2. Source Code Jawaban Soal 1 XML .....              | 12 |
| Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose .....  | 13 |
| Tabel 4. Source Code Jawaban Soal 1 XML .....              | 26 |
| Tabel 5. Source Code Jawaban Soal 1 XML .....              | 28 |
| Tabel 6. Source Code Jawaban Soal 1 XML .....              | 30 |
| Tabel 7. Source Code Jawaban Soal 1 Jetpack Compose .....  | 30 |
| Tabel 8. Source Code Jawaban Soal 1 Jetpack Compose .....  | 47 |
| Tabel 9. Source Code Jawaban Soal 1 Jetpack Compose .....  | 48 |
| Tabel 10. Source Code Jawaban Soal 1 Jetpack Compose ..... | 49 |
| Tabel 11. Source Code Jawaban Soal 1 Jetpack Compose ..... | 51 |
| Tabel 12. Source Code Jawaban Soal 1 XML .....             | 52 |
| Tabel 13. Source Code Jawaban Soal 1 XML .....             | 52 |
| Tabel 14. Source Code Jawaban Soal 1 XML .....             | 53 |
| Tabel 15. Source Code Jawaban Soal 1 XML .....             | 55 |
| Tabel 16. Source Code Jawaban Soal 1 XML .....             | 56 |
| Tabel 17. Source Code Jawaban Soal 1 XML .....             | 57 |
| Tabel 18. Source Code Jawaban Soal 1 XML .....             | 58 |
| Tabel 19. Source Code Jawaban Soal 1 XML .....             | 59 |
| Tabel 20. Source Code Jawaban Soal 1 XML .....             | 60 |
| Tabel 21. Source Code Jawaban Soal 1 XML .....             | 60 |
| Tabel 22. Source Code Jawaban Soal 1 Jetpack Compose ..... | 79 |
| Tabel 23. Source Code Jawaban Soal 1 Jetpack Compose ..... | 80 |
| Tabel 24. Source Code Jawaban Soal 1 Jetpack Compose ..... | 81 |
| Tabel 25. Source Code Jawaban Soal 1 Jetpack Compose ..... | 83 |
| Tabel 26. Source Code Jawaban Soal 1 Jetpack Compose ..... | 84 |
| Tabel 27. Source Code Jawaban Soal 1 Jetpack Compose ..... | 85 |
| Tabel 28. Source Code Jawaban Soal 1 XML .....             | 86 |

|                                                            |     |
|------------------------------------------------------------|-----|
| Tabel 29. Source Code Jawaban Soal 1 XML .....             | 86  |
| Tabel 30. Source Code Jawaban Soal 1 XML .....             | 87  |
| Tabel 31. Source Code Jawaban Soal 1 XML .....             | 88  |
| Tabel 32. Source Code Jawaban Soal 1 XML .....             | 90  |
| Tabel 33. Source Code Jawaban Soal 1 XML .....             | 91  |
| Tabel 34. Source Code Jawaban Soal 1 XML .....             | 92  |
| Tabel 35. Source Code Jawaban Soal 1 XML .....             | 93  |
| Tabel 36. Source Code Jawaban Soal 1 XML .....             | 94  |
| Tabel 37. Source Code Jawaban Soal 1 XML .....             | 94  |
| Tabel 38. Source Code Jawaban Soal 1 XML .....             | 94  |
| Tabel 39. Source Code Jawaban Soal 1 XML .....             | 96  |
| Tabel 40. Source Code Jawaban Soal 1 Jetpack Compose ..... | 111 |
| Tabel 41. Source Code Jawaban Soal 1 Jetpack Compose ..... | 114 |
| Tabel 42. Source Code Jawaban Soal 1 Jetpack Compose ..... | 115 |
| Tabel 43. Source Code Jawaban Soal 1 Jetpack Compose ..... | 115 |
| Tabel 44. Source Code Jawaban Soal 1 Jetpack Compose ..... | 117 |
| Tabel 45. Source Code Jawaban Soal 1 Jetpack Compose ..... | 118 |
| Tabel 46. Source Code Jawaban Soal 1 Jetpack Compose ..... | 120 |
| Tabel 47. Source Code Jawaban Soal 1 Jetpack Compose ..... | 121 |
| Tabel 48. Source Code Jawaban Soal 1 Jetpack Compose ..... | 121 |
| Tabel 49. Source Code Jawaban Soal 1 Jetpack Compose ..... | 124 |
| Tabel 50. Source Code Jawaban Soal 1 Jetpack Compose ..... | 128 |
| Tabel 51. Source Code Jawaban Soal 1 Jetpack Compose ..... | 133 |
| Tabel 52. Source Code Jawaban Soal 1 Jetpack Compose ..... | 137 |
| Tabel 53. Source Code Jawaban Soal 1 Jetpack Compose ..... | 138 |
| Tabel 54. Source Code Jawaban Soal 1 Jetpack Compose ..... | 141 |

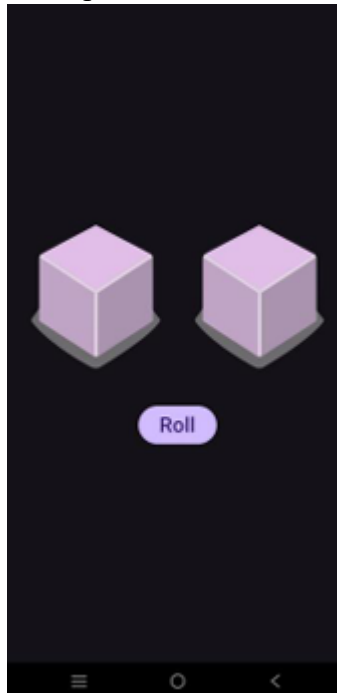


## MODUL 1 : Android Basics With Compose

### SOAL 1

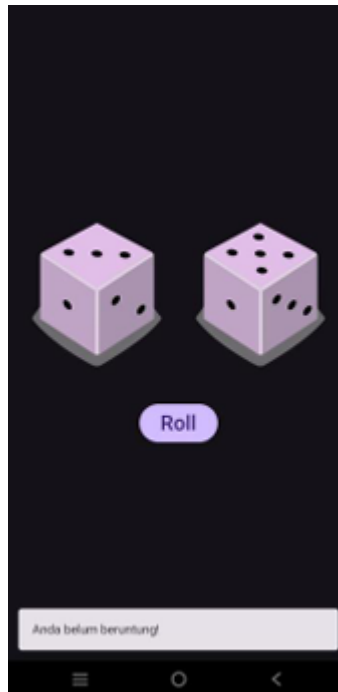
Soal Praktikum: Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



*Gambar 1. Tampilan Awal Aplikasi*

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



*Gambar 2. Tampilan Dadu Setelah Di-Roll*

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



*Gambar 3. Tampilan Roll Dadu Double*

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam folder Modul 1 dalam bentuk Project. Jangan lupa untuk melakukan Clean Project sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:  
[https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd\\_9SgFh8kw8X9ySm/view?usp=sharing](https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing)

## A. Source Code

**XML :**

**activity\_main.xml :**

*Tabel 1. Source Code Jawaban Soal 1 XML*

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <LinearLayout                                              |
| 3  | xmlns:android="http://schemas.android.com/apk/res/android" |
| 4  | d"                                                         |
| 5  | android:layout_width="match_parent"                        |
| 6  | android:layout_height="match_parent"                       |
| 7  | android:orientation="vertical"                             |
| 8  | android:gravity="center"                                   |
| 9  | android:padding="16dp">                                    |
| 10 |                                                            |
| 11 | <LinearLayout                                              |
| 12 | android:layout_width="match_parent"                        |
| 13 | android:layout_height="wrap_content"                       |
| 14 | android:orientation="horizontal"                           |
| 15 | android:gravity="center"                                   |
| 16 | android:layout_marginBottom="16dp">                        |
| 17 |                                                            |
| 18 | <ImageView                                                 |
| 19 | android:id="@+id/imageView1"                               |
| 20 | android:layout_width="0dp"                                 |
| 21 | android:layout_height="200dp"                              |
| 22 | android:layout_weight="1"                                  |
| 23 | android:contentDescription="Dice 1"                        |
| 24 | android:src="@drawable/dice_0"                             |
| 25 | android:scaleType="centerCrop"/>                           |
| 26 |                                                            |
| 27 | <ImageView                                                 |
| 28 | android:id="@+id/imageView2"                               |
| 29 | android:layout_width="0dp"                                 |
| 30 | android:layout_height="200dp"                              |

|    |                                                     |
|----|-----------------------------------------------------|
| 31 | android:layout_weight="1"                           |
| 32 | android:contentDescription="Dice 2"                 |
| 33 | android:src="@drawable/dice_0"                      |
| 34 | android:scaleType="centerCrop"/>                    |
| 35 | </LinearLayout>                                     |
| 36 |                                                     |
| 37 | <Button                                             |
| 38 | android:id="@+id/rollButton"                        |
| 39 | android:layout_width="wrap_content"                 |
| 40 | android:layout_height="wrap_content"                |
| 41 | android:layout_marginTop="16dp"                     |
| 42 | android:backgroundTint="@android:color/holo_purple" |
|    | android:text="Roll" />                              |
|    | </LinearLayout>                                     |

### MainActivity.kt :

*Tabel 2. Source Code Jawaban Soal 1 XML*

|    |                                                      |
|----|------------------------------------------------------|
| 1  | package com.example.dicerollerxml                    |
| 2  |                                                      |
| 3  | import android.os.Bundle                             |
| 4  | import android.widget.Button                         |
| 5  | import android.widget.ImageView                      |
| 6  | import androidx.appcompat.app.AppCompatActivity      |
| 7  | import com.google.android.material.snackbar.Snackbar |
| 8  |                                                      |
| 9  | class MainActivity : AppCompatActivity() {           |
| 10 |                                                      |
| 11 | lateinit var imageView1: ImageView                   |
| 12 | lateinit var imageView2: ImageView                   |
| 13 | lateinit var rollButton: Button                      |
| 14 |                                                      |
| 15 | override fun onCreate(savedInstanceState: Bundle?) { |
| 16 | super.onCreate(savedInstanceState)                   |
| 17 | setContentView(R.layout.activity_main)               |
| 18 |                                                      |
| 19 | imageView1 = findViewById(R.id.imageView1)           |
| 20 | imageView2 = findViewById(R.id.imageView2)           |
| 21 | rollButton = findViewById(R.id.rollButton)           |
| 22 |                                                      |
| 23 | rollButton.setOnClickListener {                      |
| 24 | val hasil = (1..6).random()                          |
| 25 | val hasil2 = (1..6).random()                         |
| 26 |                                                      |
| 27 | setImage(imageView1, hasil)                          |
| 28 | setImage(imageView2, hasil2)                         |
| 29 | }                                                    |

|    |                                                        |
|----|--------------------------------------------------------|
| 30 | val message = if (hasil == hasil2) "Selamat,           |
| 31 | anda dapat dadu double!" else "Anda belum beruntung!"  |
| 32 |                                                        |
| 33 | Snackbar.make(findViewById(android.R.id.content),      |
| 34 | message, Snackbar.LENGTH_SHORT).show()                 |
| 35 | }                                                      |
| 36 | }                                                      |
| 37 |                                                        |
| 38 | private fun setImage(imageView: ImageView, hasil: Int) |
| 39 | {                                                      |
| 40 | val imageResId = when (hasil) {                        |
| 41 | 1 -> R.drawable.dice_1                                 |
| 42 | 2 -> R.drawable.dice_2                                 |
| 43 | 3 -> R.drawable.dice_3                                 |
| 44 | 4 -> R.drawable.dice_4                                 |
| 45 | 5 -> R.drawable.dice_5                                 |
| 46 | else -> R.drawable.dice_6                              |
|    | }                                                      |
|    | imageView.setImageResource(imageResId)                 |
|    | }                                                      |
|    | }                                                      |

## Jetpack Compose :

### MainActivity.kt :

*Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.example.dicerollercompose                    |
| 2  |                                                          |
| 3  | import android.os.Bundle                                 |
| 4  | import androidx.activity.ComponentActivity               |
| 5  | import androidx.activity.compose.setContent              |
| 6  | import androidx.compose.foundation.Image                 |
| 7  | import androidx.compose.foundation.layout.*              |
| 8  | import androidx.compose.material3.*                      |
| 9  | import androidx.compose.runtime.*                        |
| 10 | import androidx.compose.ui.Alignment                     |
| 11 | import androidx.compose.ui.Modifier                      |
| 12 | import androidx.compose.ui.res.painterResource           |
| 13 | import androidx.compose.ui.tooling.preview.Preview       |
| 14 | import androidx.compose.ui.unit.dp                       |
| 15 | import                                                   |
| 16 | com.example.dicerollercompose.ui.theme.DiceRollerCompose |
| 17 | Theme                                                    |
| 18 | import kotlinx.coroutines.launch                         |
| 19 |                                                          |
| 20 | class MainActivity : ComponentActivity() {               |
| 21 | override fun onCreate(savedInstanceState: Bundle?) {     |

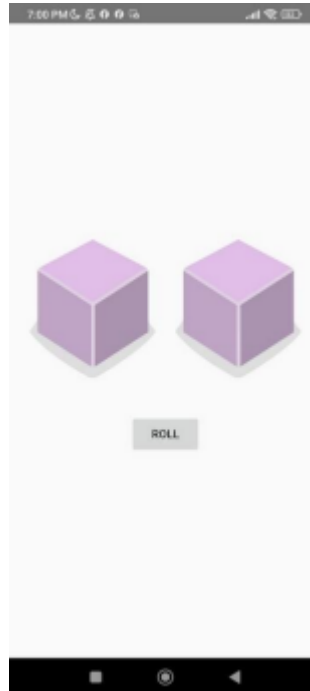
```

22         super.onCreate(savedInstanceState)
23         setContent {
24             DiceRollerComposeTheme {
25                 DiceRollerApp()
26             }
27         }
28     }
29 }
30
31 @Preview(showBackground = true)
32 @Composable
33 fun DiceRollerApp() {
34     DiceWithButtonAndImage()
35 }
36
37 @Composable
38 fun DiceWithButtonAndImage() {
39     var hasil by remember { mutableStateOf(0) }
40     var hasil2 by remember { mutableStateOf(0) }
41     val snackbarHostState = remember { SnackbarHostState() }
42 }
43     val coroutineScope = rememberCoroutineScope()
44
45     fun getDiceImage(hasil: Int) = when (hasil) {
46         0 -> R.drawable.dice_0
47         1 -> R.drawable.dice_1
48         2 -> R.drawable.dice_2
49         3 -> R.drawable.dice_3
50         4 -> R.drawable.dice_4
51         5 -> R.drawable.dice_5
52         else -> R.drawable.dice_6
53     }
54
55     val gambarDadu = getDiceImage(hasil)
56     val gambarDadu2 = getDiceImage(hasil2)
57
58     Box(
59         modifier = Modifier.fillMaxSize(),
60         contentAlignment = Alignment.Center
61     ) {
62         Column(
63             horizontalAlignment = Alignment.CenterHorizontally
64         ) {
65             Row(
66                 horizontalArrangement = Arrangement.Center,
67                 verticalAlignment = Alignment.CenterVertically
68             ) {
69                 // ...
70             }

```

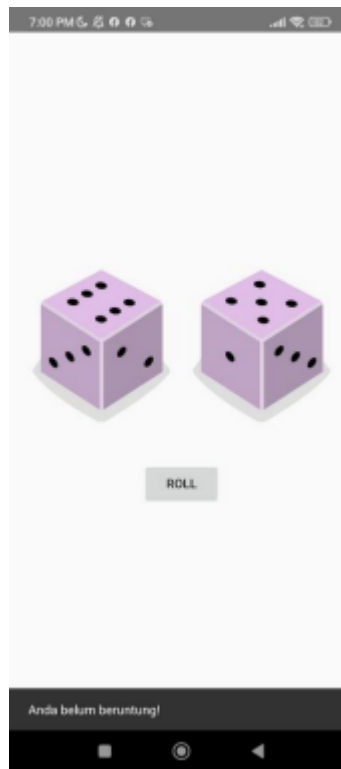
|    |                                                          |
|----|----------------------------------------------------------|
| 71 | ) {                                                      |
| 72 | Image(painter =                                          |
| 73 | painterResource(gambarDadu), contentDescription = null,  |
| 74 | modifier = Modifier.size(200.dp))                        |
| 75 | Image(painter =                                          |
| 76 | painterResource(gambarDadu2), contentDescription = null, |
| 77 | modifier = Modifier.size(200.dp))                        |
| 78 | }                                                        |
| 79 | Spacer(modifier = Modifier.height(16.dp))                |
| 80 | Button(onClick = {                                       |
| 81 | hasil = (1..6).random()                                  |
| 82 | hasil2 = (1..6).random()                                 |
| 83 | val message = if (hasil == hasil2)                       |
| 84 | "Selamat, anda dapat dadu double!" else "Anda belum      |
| 85 | beruntung!"                                              |
| 86 |                                                          |
| 87 | coroutineScope.launch {                                  |
| 88 |                                                          |
| 89 | snackbarHostState.currentSnackbarData?.dismiss()         |
| 90 |                                                          |
| 91 | snackbarHostState.showSnackbar(message, duration =       |
| 92 | SnackbarDuration.Short)                                  |
|    | }                                                        |
|    | }) {                                                     |
|    | Text(text = "Roll")                                      |
|    | }                                                        |
|    | }                                                        |
|    | SnackbarHost(                                            |
|    | hostState = snackbarHostState,                           |
|    | modifier = Modifier                                      |
|    | .fillMaxWidth()                                          |
|    | .align(Alignment.BottomCenter)                           |
|    | .padding(bottom = 16.dp)                                 |
|    | )                                                        |
|    | }                                                        |
|    | }                                                        |

## B. Output Program

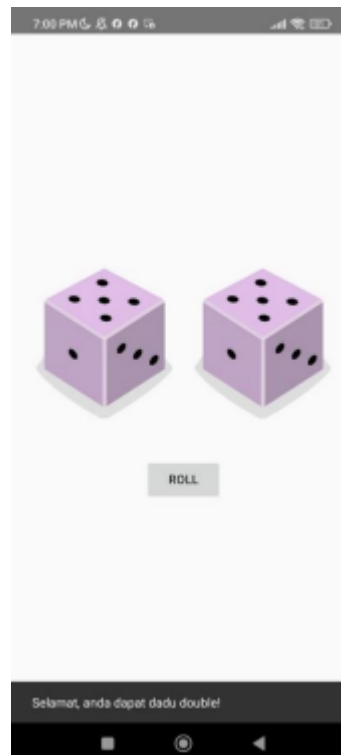


*Gambar 4. Screenshot Hasil Jawaban Soal 1 XML*

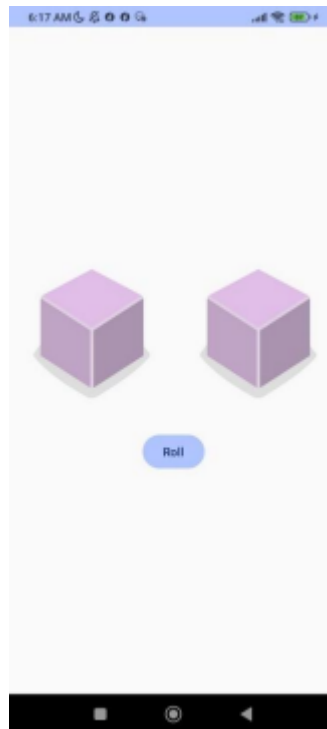




*Gambar 5. Screenshot Hasil Jawaban Soal 1 XML*



*Gambar 6. Screenshot Hasil Jawaban Soal 1 XML*



*Gambar 7. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*



*Gambar 8. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*



Gambar 9. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

### C. Pembahasan

#### XML :

##### **activity\_main.xml :**

Line 1, deklarasi standar XML yang menandai awal dokumen.

Line 2, elemen root layout: `LinearLayout`, menyusun elemen secara vertikal karena `android:orientation="vertical"` (line 5). `xmlns:android` adalah deklarasi namespace Android

agar atribut seperti `android:layout_width` dikenali.

Line 3 dan 4, Layout akan mengisi seluruh lebar dan tinggi layar menggunakan `”match_parent”`

Line 5, semua elemen anak (children) akan ditata dari atas ke bawah.

Line 6, seluruh isi `LinearLayout` akan di-tengah-kan secara horizontal dan vertikal.

Line 7, memberi jarak 16dp dari sisi layout terhadap konten (padding di semua sisi).

Line 9 sampai 14, Ini adalah layout horizontal di dalam layout utama, untuk menampung 2

gambar dadu berdampingan. `layout_marginBottom="16dp"` memberikan jarak antara baris

gambar dan tombol di bawahnya.

Line 16 sampai 23, `@+id/imageView1` adalah ID unik yang bisa diakses dari Kotlin.

`layout_width="0dp" + layout_weight="1"` → Membagi lebar sama rata dengan `ImageView`

lain. `src="@drawable/dice_0"` → Gambar awal adalah dadu kosong (nilai 0).

`scaleType="centerCrop"` → Gambar dipotong supaya penuh tanpa merusak rasio.

Line 25 sampai 32, gambar dadu kedua, sama seperti line 16 sampai 23.

Line 35 sampai 41, menampilkan tombol yang ukurannya mengikuti teks atau konten didalamnya, dengan warna latar belakang ungu dan teks "Roll"

### **MainActivity.kt :**

Line 1, adalah nama package sesuai dengan struktur project

Line 3 sampai 7, Mengimpor komponen View (`ImageView`, `Button`) dan `Snackbar` dari `Material Design`.

Line 9, adalah aktivitas utama aplikasi, mewarisi `AppCompatActivity`.

Line 11 sampai 13, Variabel View yang akan dihubungkan ke XML nanti dengan `findViewById`.

Line 15 sampai 16, Fungsi yang dijalankan saat aplikasi pertama kali dibuka.

Line 17, Menghubungkan tampilan layout XML ke kode Kotlin (menampilkan `activity_main.xml`).

Line 19 sampai 21, Menghubungkan variabel di Kotlin dengan komponen yang ada di XML

menggunakan ID masing-masing.

Line 23 sampai 32, menambahkan action ke tombol atau button, yaitu jika di-klik akan menghasilkan angka acak dari 1 sampai 6 dan menyimpannya untuk variabel `hasil` dan `hasil2`

secara terpisah. Lalu memanggil fungsi `setImage` untuk mengganti gambar berdasarkan angka, lalu jika nilai dari `hasil` dan `hasil2` sama maka akan mengisi variabel `message` dengan

teks "Selamat, anda dapat dadu double!", Jika tidak maka akan diisi dengan "Anda belum

beruntung!", lalu menampilkan notifikasi `snackbar` dengan pesan yang ada di variabel `message` dengan waktu yang singkat

17

Line 35 sampai 44, adalah fungsi untuk mengatur gambar dadu sesuai dengan variabel `hasil`

di fungsi `setImage`. Jika `hasil` adalah 1 maka akan menampilkan gambar dadu 1, jika 2 maka

akan memperlihatkan gambar dadu 2, dan seterusnya.

## Jetpack Compose :

### MainActivity.kt:

Line 1, package com.example.dicerollercompose menentukan namespace aplikasi yang

berguna untuk membedakan aplikasi android diceroller dari aplikasi lainnya.

Line 3 sampai 16, berisi import import yang berfungsi agar bisa memakai kembali fungsi,

komponen, kelas, dan fitur dari library lain ke dalam kode seperti :

1. android.os.Bundle :

Digunakan untuk menyimpan dan mengelola data yang dikirimkan antar aktivitas dalam aplikasi Android. Dalam kode ini, digunakan di dalam onCreate untuk menangani instance state aplikasi.

2. androidx.activity.ComponentActivity :

Merupakan kelas dasar untuk aktivitas yang menggunakan Jetpack Compose.

MainActivity mewarisi ComponentActivity agar bisa menjalankan UI berbasis Compose.

3. androidx.activity.compose.setContent :

Digunakan untuk menetapkan UI aplikasi menggunakan Jetpack Compose. Dalam kode ini, setContent memanggil DiceRollerComposeTheme { DiceRollerApp() } untuk menampilkan UI utama.

4. androidx.compose.foundation.Image :

Komponen Jetpack Compose yang digunakan untuk menampilkan gambar. Dalam kode ini, digunakan untuk menampilkan dadu dengan gambar yang sesuai.

5. androidx.compose.foundation.layout.\* :

Mengimpor berbagai komponen tata letak seperti Box, Column, Row, Spacer, dan Modifier. Digunakan untuk menyusun elemen-elemen UI seperti posisi dadu, tombol, dan snackbar.

6. androidx.compose.material3.\* :

Mengimpor komponen dari Material Design 3, termasuk Button, Text, SnackbarHost, dan SnackbarHostState. Digunakan untuk membuat tombol "Roll" dan menampilkan snackbar sebagai notifikasi.

7. \*androidx.compose.runtime.\* :

Mengimpor fungsi-fungsi yang mendukung state management seperti remember dan mutableStateOf. Digunakan untuk menyimpan dan memperbarui nilai dadu saat tombol ditekan.

8. androidx.compose.ui.Alignment :

Digunakan untuk mengatur perataan elemen dalam tata letak, misalnya Alignment.Center dalam Box untuk memusatkan dadu.

9. androidx.compose.ui.Modifier:

Digunakan untuk mengubah atau menyesuaikan tampilan dan perilaku elemen UI. Dalam kode ini, digunakan untuk mengatur ukuran gambar, margin, padding, dan posisi elemen.

10. `androidx.compose.ui.res.painterResource :`

18

Digunakan untuk memuat gambar dari drawable berdasarkan ID sumber daya. Dalam kode ini, digunakan untuk menampilkan gambar dadu berdasarkan nilai yang dihasilkan.

11. `androidx.compose.ui.tooling.preview.Preview :`

Digunakan untuk menampilkan pratinjau UI dalam Android Studio. Dalam kode ini, digunakan di `DiceRollerApp()` agar tampilan aplikasi bisa dilihat sebelum dijalankan.

12. `androidx.compose.ui.unit.dp :`

Digunakan untuk mengatur ukuran elemen dalam satuan density-independent pixels (dp). Contohnya, `Modifier.size(200.dp)` digunakan untuk menentukan ukuran gambar dadu.

13. `com.example.dicerollercompose.ui.theme.DiceRollerComposeTheme :`

Mengimpor tema khusus yang telah dibuat dalam proyek untuk konsistensi tampilan aplikasi.

14. `kotlinx.coroutines.launch`

Digunakan untuk menjalankan operasi asinkron dalam coroutine. Dalam kode ini, digunakan untuk menampilkan snackbar secara non-blocking ketika tombol ditekan.

Line 18, Mendeklarasikan kelas `MainActivity`, mewarisi `ComponentActivity`.

Line 19, Override `onCreate`, dipanggil saat Activity dibuat.

Line 20, Memanggil `super.onCreate()` dari parent class.

Line 21 sampai 27, Mengatur tampilan UI menggunakan Jetpack Compose dengan tema dan

fungsi utama `DiceRollerApp()`.

Line 29, `@Preview(showBackground = true)` Menampilkan tampilan UI dalam mode preview di Android Studio.

Line 30, Mendeklarasikan `DiceRollerApp` sebagai fungsi compose, menandakan bahwa

fungsi ini adalah UI yang dapat dikomposisi ulang (Jetpack Compose).

Line 31, `DiceRollerApp()`, Fungsi utama yang memanggil `DiceWithButtonAndImage()`

untuk menampilkan dadu dan tombol roll.

Line 32 sampai 33, Memanggil UI utama dari aplikasi, `DiceWithButtonAndImage()`.

Line 35, Mendeklarasikan `DiceWithButtonAndImage` sebagai fungsi compose, menandakan

bahwa fungsi ini adalah UI yang dapat dikomposisi ulang (Jetpack Compose).

Line 36, Fungsi utama yang menyusun antarmuka dan logika aplikasi.

Line 37 sampai 40, `hasil` & `hasil2` adalah variabel yang menyimpan angka dadu yang akan ditampilkan, `mutableStateOf(0)` artinya nilai awalnya adalah 0. remember memastikan nilai state tetap bertahan selama UI tidak direkomposisi ulang. `snackbarHostState` menyimpan state untuk snackbar yang menampilkan pesan notifikasi. `coroutineScope` digunakan untuk menjalankan proses asynchronous dalam coroutine.

Line 42 `getDiceImage(hasil: Int)` adalah fungsi untuk mengembalikan gambar dadu berdasarkan nilai `hasil`.

Line 43 sampai 49, Struktur kontrol seperti switch-case (when). Jika `hasil` adalah 0, akan menampilkan `dice_0`, jika 1 akan `dice_1`, dan seterusnya.

Line 52 dan 53, `gambarDadu` & `gambarDadu2` menyimpan gambar dadu yang sesuai dengan nilai `hasil` dan `hasil2`.

Line 55 sampai 58, membuat layout Box dengan ukuran layar penuh dan isi di tengah layar.

Line 59 sampai 61, menyusun elemen UI secara vertikal dan sejajar tengah secara horizontal.

Line 62 sampai 65, menyusun agar dua dadu ditempatkan secara horizontal di tengah layar.

Line 66 dan 67, menampilkan dua gambar dadu dengan ukuran 200dp.

19

Line 69, menambahkan jarak vertikal 16dp sebelum tombol.

Line 70 sampai 73, menampilkan sebuah tombol atau button yang apabila di tekan atau klik maka akan mengacak variabel `hasil` dan `hasil2` dari angka 1 sampai angka 6 menggunakan fungsi `random`. Apabila nilai Dari variabel `hasil` dan `hasil2` berbeda maka akan menampilkan pesan "Anda belum beruntung!", sebaliknya "Selamat, anda dapat dadu double!"

Line 75, `coroutineScope.launch` menjalankan operasi snackbar dalam coroutine.

Line 76, `snackbarHostState.currentSnackbarData?.dismiss()` berguna jika ada snackbar yang sedang ditampilkan, tutup terlebih dahulu.

Line 77, `snackbarHostState.showSnackbar(message, duration = SnackbarDuration.Short)` → Menampilkan snackbar dengan pesan yang sesuai (dari fungsi `message`), dan dengan durasi

yang singkat menggunakan `duration = SnackbarDuration.Short`  
Line 79 sampai 81, menampilkan teks "Roll"  
Line 84, `SnackbarHost` menampilkan snackbar di layar.  
Line 85, `hostState = snackbarHostState` digunakan untuk menghubungkan `SnackbarHost` dengan state pengatur snackbar agar dapat menampilkan pesan notifikasi.  
Line 86, `modifier = Modifier` berfungsi untuk mengatur ukuran, posisi, dan tata letak visual dari elemen UI  
Line 87, `Modifier.fillMaxWidth()` memastikan snackbar mengisi lebar layar  
Line 88, `align(Alignment.BottomCenter)` memposisikan snackbar di bawah layar.  
Line 89, `padding(bottom = 16.dp)` → memberi jarak 16 dp dari bawah.

#### **D. Tautan Git**

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul1>

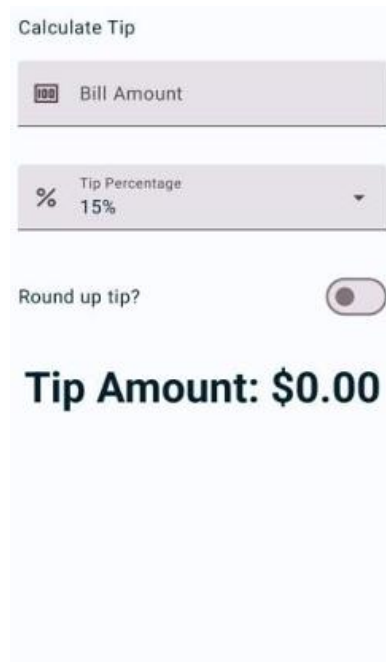


## MODUL 2 : Android Layout With Compose

### SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 10. Tampilan Awal Aplikasi .



Gambar 11. Tampilan Pilihan Persentase Tip

## A. Source Code

### XML :

#### activity\_main.xml :

Tabel 4. Source Code Jawaban Soal 1 XML

|    |                                                            |               |                    |
|----|------------------------------------------------------------|---------------|--------------------|
| 1  | <?xml                                                      | version="1.0" | encoding="utf-8"?> |
| 2  | <LinearLayout                                              |               |                    |
| 3  | xmlns:android="http://schemas.android.com/apk/res/android" |               |                    |
| 4  | android:layout_width="match_parent"                        |               |                    |
| 5  | android:layout_height="match_parent"                       |               |                    |
| 6  | android:orientation="vertical"                             |               |                    |
| 7  | android:padding="24dp">                                    |               |                    |
| 8  |                                                            |               |                    |
| 9  | <TextView                                                  |               |                    |
| 10 | android:id="@+id/titleText"                                |               |                    |
| 11 | android:layout_width="wrap_content"                        |               |                    |
| 12 | android:layout_height="wrap_content"                       |               |                    |
| 13 | android:layout_gravity="start"                             |               |                    |
| 14 | android:paddingBottom="16dp"                               |               |                    |
| 15 | android:text="@string/app_name"                            |               |                    |
| 16 |                                                            |               |                    |

```

17 android:textAppearance="?android:attr/textAppearanceLarge"
18 />
19
20 <EditText
21     android:id="@+id/costOfService"
22     android:layout_width="match_parent"
23     android:layout_height="wrap_content"
24     android:hint="@string/hint_service_amount"
25     android:inputType="numberDecimal"
26     android:imeOptions="actionDone"
27     android:padding="12dp"
28
29 <TextView
30     android:id="@+id/selectTipLabel"
31     android:layout_width="wrap_content"
32     android:layout_height="wrap_content"
33     android:text="@string/tip_percentage"
34     android:layout_marginTop="16dp"
35     android:layout_marginBottom="8dp"
36
37 <Spinner
38     android:id="@+id/tipOptions"
39     android:layout_width="match_parent"
40     android:layout_height="wrap_content"
41     android:minHeight="48dp"
42     android:spinnerMode="dropdown"
43
44 <androidx.constraintlayout.widget.ConstraintLayout
45     android:layout_width="match_parent"
46     android:layout_height="wrap_content"
47     android:layout_marginTop="16dp"
48     xmlns:app="http://schemas.android.com/apk/res-
49 auto">
50
51     <TextView
52         android:id="@+id/labelText"
53         android:layout_width="wrap_content"
54         android:layout_height="wrap_content"
55         android:text="@string/round_up_tip"
56         app:layout_constraintStart_toStartOf="parent"
57         app:layout_constraintTop_toTopOf="parent"
58
59 app:layout_constraintBottom_toBottomOf="parent"
60
61     <androidx.appcompat.widget.SwitchCompat
62         android:id="@+id/roundUpSwitch"
63         android:layout_width="wrap_content"
64         android:layout_height="wrap_content"
65

```

|    |                                                      |
|----|------------------------------------------------------|
| 66 | android:contentDescription="@string/round_up_tip"    |
| 67 | app:layout_constraintEnd_toEndOf="parent"            |
| 68 | app:layout_constraintTop_toTopOf="parent"            |
| 69 |                                                      |
| 70 | app:layout_constraintBottom_toBottomOf="parent" />   |
| 71 |                                                      |
| 72 | </androidx.constraintlayout.widget.ConstraintLayout> |
| 73 |                                                      |
| 74 | <TextView                                            |
| 75 | android:id="@+id/tipResult"                          |
| 76 | android:layout_width="match_parent"                  |
|    | android:layout_height="wrap_content"                 |
|    | android:layout_marginTop="32dp"                      |
|    | android:text="@string/tip_result_placeholder"        |
|    | android:textColor="#000000"                          |
|    | android:textSize="37sp"                              |
|    | android:textStyle="bold" />                          |
|    | </LinearLayout>                                      |

## MainActivity.kt :

Tabel 5. Source Code Jawaban Soal 1 XML

|    |                                                      |
|----|------------------------------------------------------|
| 1  | package com.example.tipcalculatorxml                 |
| 2  |                                                      |
| 3  | import android.os.Bundle                             |
| 4  | import android.view.View                             |
| 5  | import android.widget.*                              |
| 6  | import androidx.appcompat.app.AppCompatActivity      |
| 7  | import androidx.appcompat.widget.SwitchCompat        |
| 8  | import java.text.NumberFormat                        |
| 9  | import kotlin.math.ceil                              |
| 10 |                                                      |
| 11 | class MainActivity : AppCompatActivity() {           |
| 12 |                                                      |
| 13 | lateinit var costEditText: EditText                  |
| 14 | lateinit var tipSpinner: Spinner                     |
| 15 | lateinit var roundUpSwitch: SwitchCompat             |
| 16 | lateinit var tipResultTextView: TextView             |
| 17 |                                                      |
| 18 | override fun onCreate(savedInstanceState: Bundle?) { |
| 19 | super.onCreate(savedInstanceState)                   |

```

20         setContentView(R.layout.activity_main)
21
22         costEditText = findViewById(R.id.costOfService)
23         tipSpinner = findViewById(R.id.tipOptions)
24         roundUpSwitch = findViewById(R.id.roundUpSwitch)
25         tipResultTextView = findViewById(R.id.tipResult)
26
27         val tipOptions = listOf("15%", "18%", "20%")
28         ArrayAdapter(
29             this,
30             android.R.layout.simple_spinner_item,
31             tipOptions
32         ).also {
33
34             it.setDropDownViewResource(android.R.layout.simple_spinner
35             _dropdown_item)
36             tipSpinner.adapter = it
37         }
38
39         costEditText.setOnEditorActionListener { _, _, _ -
40     >
41             calculateTip()
42             false
43         }
44
45         tipSpinner.onItemSelectedListener = object :
46     AdapterView.OnItemSelectedListener {
47             override fun onItemSelected(
48                 parent: AdapterView<*>, view: View?,
49                 position: Int, id: Long
50             ) {
51                 calculateTip()
52             }
53
54             override fun onNothingSelected(parent:
55     AdapterView<*>) {}
56         }
57
58         roundUpSwitch.setOnCheckedChangeListener { _, _ ->
59             calculateTip()
60         }
61     }
62
63     private fun calculateTip() {
64         val cost =
65     costEditText.text.toString().toDoubleOrNull()
66
67         if (cost == null) {
68

```

|    |                                                            |
|----|------------------------------------------------------------|
| 69 | tipResultTextView.setText(R.string.tip_result_placeholder) |
| 70 | return                                                     |
| 71 | }                                                          |
| 72 |                                                            |
| 73 | val tipPercentage = when                                   |
| 74 | (tipSpinner.selectedItem.toString()) {                     |
| 75 | "20%" -> 0.20                                              |
| 76 | "18%" -> 0.18                                              |
| 77 | else -> 0.15                                               |
| 78 | }                                                          |
| 79 |                                                            |
|    | var tip = cost * tipPercentage                             |
|    | if (roundUpSwitch.isChecked) {                             |
|    | tip = ceil(tip)                                            |
|    | }                                                          |
|    |                                                            |
|    | val formattedTip =                                         |
|    | NumberFormat.getCurrencyInstance().format(tip)             |
|    | tipResultTextView.text =                                   |
|    | getString(R.string.tip_result, formattedTip)               |
|    | }                                                          |
|    | }                                                          |

### strings.xml :

Tabel 6. Source Code Jawaban Soal 1 XML

|   |                                                       |
|---|-------------------------------------------------------|
| 1 | <resources>                                           |
| 2 | <string name="app_name">Calculate Tip</string>        |
| 3 | <string name="hint_service_amount">Bill               |
| 4 | Amount</string>                                       |
| 5 | <string name="tip_percentage">Tip percentage</string> |
| 6 | <string name="round_up_tip">Round up tip?</string>    |
| 7 | <string name="tip_result">Tip Amount: %1\$s</string>  |
| 8 | <string name="tip_result_placeholder">Tip Amount:     |
|   | </string>                                             |
|   | </resources>                                          |

### Jetpack Compose :

#### MainActivity.kt :

Tabel 7. Source Code Jawaban Soal 1 Jetpack Compose

|   |                                          |
|---|------------------------------------------|
| 1 | package com.example.tipcalculatorcompose |
| 2 |                                          |

```

3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.layout.*
7 import androidx.compose.foundation.text.KeyboardOptions
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.text.input.KeyboardType
13 import androidx.compose.ui.tooling.preview.Preview
14 import androidx.compose.ui.unit.dp
15 import java.text.NumberFormat
16 import kotlin.math.ceil
17
18 class MainActivity : ComponentActivity() {
19     override fun onCreate(savedInstanceState: Bundle?) {
20         super.onCreate(savedInstanceState)
21         setContent { TipCalculatorApp() }
22     }
23 }
24
25 @Composable
26 fun TipCalculatorApp() {
27     Surface(modifier = Modifier.fillMaxSize()) {
28         TipCalculatorLayout()
29     }
30 }
31
32 @Composable
33 fun TipCalculatorLayout() {
34     var serviceAmount by remember { mutableStateOf("") }
35     var selectedTip by remember { mutableStateOf("20%") }
36     var roundUp by remember { mutableStateOf(false) }
37
38     val tipOptions = listOf("20%", "18%", "15%")
39     val tipPercent =
40     selectedTip.dropLast(1).toDoubleOrNull() ?: 0.0
41     val amount = serviceAmount.toDoubleOrNull() ?: 0.0
42     val formattedTip =
43     NumberFormat.getCurrencyInstance().format(
44         calculateTip(amount, tipPercent, roundUp)
45     )
46
47     Column(
48         modifier = Modifier
49             .padding(16.dp)
50             .fillMaxWidth()
51     ) {

```

```

52         with(MaterialTheme.typography) {
53             Text("Tip Calculator", style = headlineSmall)
54             Spacer(modifier = Modifier.height(16.dp))
55
56             OutlinedTextField(
57                 value = serviceAmount,
58                 onValueChange = { serviceAmount = it },
59                 label = { Text("Service Amount") },
60                 keyboardOptions =
61 KeyboardOptions(keyboardType = KeyboardType.Number),
62                 modifier = Modifier.fillMaxWidth()
63             )
64
65             Spacer(modifier = Modifier.height(16.dp))
66
67             TipDropdown(
68                 options = tipOptions,
69                 selected = selectedTip,
70                 onSelect = { selectedTip = it }
71             )
72
73             Spacer(modifier = Modifier.height(16.dp))
74
75             Row(
76                 modifier = Modifier.fillMaxWidth(),
77                 verticalAlignment =
78 Alignment.CenterVertically
79             ) {
80                 Text("Round up tip?")
81                 Spacer(modifier = Modifier.weight(1f))
82                 Switch(checked = roundUp, onCheckedChange
83 = { roundUp = it })
84             }
85
86             Spacer(modifier = Modifier.height(24.dp))
87             Text("Tip Amount: $formattedTip", style =
88 bodyLarge)
89         }
90     }
91 }
92
93 @OptIn(ExperimentalMaterial3Api::class)
94 @Composable
95 fun TipDropdown(
96     options: List<String>,
97     selected: String,
98     onSelect: (String) -> Unit
99 ) {
100     var expanded by remember { mutableStateOf(false) }

```



```

101
102     ExposedDropDownMenuBox(
103         expanded = expanded,
104         onExpandedChange = { expanded = !expanded },
105         modifier = Modifier.fillMaxWidth()
106     ) {
107         OutlinedTextField(
108             value = selected,
109             onValueChange = {},
110             readOnly = true,
111             label = { Text("Tip Percentage") },
112             trailingIcon = {
113 ExposedDropDownMenuDefaults.TrailingIcon(expanded) },
114             modifier = Modifier
115                 .menuAnchor()
116                 .fillMaxWidth()
117         )
118         ExposedDropDownMenu(
119             expanded = expanded,
120             onDismissRequest = { expanded = false },
121             modifier = Modifier.fillMaxWidth()
122         ) {
123             options.forEach {
124                 DropdownMenuItem(
125                     text = { Text(it) },
126                     onClick = {
127                         onSelect(it)
128                         expanded = false
129                     }
130                 )
131             }
132         }
133     }
134 }
135
136 fun calculateTip(amount: Double, tipPercent: Double,
137     roundUp: Boolean): Double {
138     var tip = amount * (tipPercent / 100)
139     if (roundUp) tip = ceil(tip)
140     return tip
141 }
142
143 @Preview(showBackground = true)
144 @Composable
145 fun TipCalculatorPreview() {
146     TipCalculatorApp()
147 }

```

## **B. Output Program**

**XML :**

### Calculate Tip

Bill Amount

Tip percentage

15%

Round up tip?

Tip Amount:

*Gambar 12. Screenshot Hasil Jawaban Soal 1 XML*

Calculate Tip

60

Tip percentage

15%

18%

20%

▼

☐

Tip Amount:

Gambar 13. Screenshot Hasil Jawaban Soal 1 XML

## Calculate Tip

60

Tip percentage

18%

Round up tip?



**Tip Amount:  
Rp11,00**

*Gambar 14. Screenshot Hasil Jawaban Soal 1 XML*

**Compose :**

### Tip Calculator

Tip Percentage

20% ▼

Round up tip? ☒

Tip Amount: Rp0,00

*Gambar 15. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*

## Tip Calculator

Service Amount

60

Tip Percentage

20%

20%

18%

15%

Gambar 16. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Tip Calculator

Service Amount

60

Tip Percentage

18%

Round up tip?

Tip Amount: Rp11,00

*Gambar 17. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*

### **C. Pembahasan**

**XML :**

**activity\_main.xml :**

Line 1, adalah deklarasi XML standar, menyatakan bahwa file ini menggunakan XML versi 1.0 dan encoding UTF-8.



Line - sampai -, Linear layout mengatur semua komponen UI di dalamnya dengan tipe vertical, yang berarti semua elemen di dalamnya akan disusun dari atas ke bawah dengan padding 24dp(jarak antara tepi layar dan konten) agar tidak konten tidak terlalu menempel ke layar. Tinggi dan lebar layar menyesuaikan dengan parent (mengisi seluruh layar)

Line – sampai -, untuk menampilkan teks dari "@string/app\_name" yang berisikan "Calculate Tip" dengan lebar dan tinggi menyesuaikan isi teks (wrap\_content), posisi teksnya di kiri dengan menggunakan layout\_gravity="start", dengan jarak bawah 16dp, dengan tampilan teks besar (textAppearanceLarge)

Line – sampai -, adalah kolom untuk meng-input teks, input berupa angka desimal (numberDecimal), dan keyboard akan menampilkan tombol "Done"

Line 26 sampai 32, Menampilkan teks "Tip Percentage" dengan lebar dan tinggi menyesuaikan dengan isi teks

Line 34 sampai 39, menampilkan spinner dengan mode dropdown

Line 41 sampai 45, membuat layout yang fleksibel dengan constraint antar komponen dari line 41 sampai 65

Line 47 sampai 54, menampilkan teks "Round up tip?" dengan constraint awalan dari parent (kiri) dan constraint atas dan bawah dari parent

Line 56 sampai 63, menampilkan sebuah switch dengan constraint akhiran dari parent (kanan) dan constraint atas dan bawah dari parent

Line 67 sampai 76, menampilkan teks "Tip Amount" dengan ukuran 37sp, warna hitam, dan style bold

### **MainActivity.kt :**

Line 1, menyatakan bahwa file ini berada di dalam package com.example.tipcalculatorxml

Line 3 sampai 9, Mengimpor semua class dan fungsi yang dibutuhkan:

Bundle: untuk menyimpan dan memulihkan state aktivitas.

View, EditText, Spinner, TextView, AdapterView: komponen UI.

AppCompatActivity: base class untuk activity dengan dukungan ActionBar.

SwitchCompat: komponen switch modern dari AndroidX.

NumberFormat: untuk format mata uang.

ceil: membulatkan angka ke atas.

Line 11, Mendefinisikan MainActivity sebagai subclass dari AppCompatActivity.

Line 13 sampai 16, mendefinisikan variabel untuk menyimpan referensi ke elemen UI di layout (EditText, Spinner, SwitchCompat, dan TextView).

Line 18 sampai 20, adalah fungsi yang dipanggil saat activity pertama kali dibuat.

setContentView() menghubungkan file XML activity\_main.xml dengan kelas Kotlin ini.

Line 22 sampai 25, Menghubungkan masing-masing variabel dengan elemen UI yang berada di layout activity\_main berdasarkan id.

Line 27, mendeklarasikan variabel "TipOptions" untuk menyimpan daftar pilihan tip yang bersifat immutable (tidak bisa dirubah) karena menggunakan fungsi listOf

Line 28 sampai 35, Membuat ArrayAdapter untuk menghubungkan tipOptions ke Spinner, menentukan tampilan dropdown, dan mengatur adapter ke tipSpinner.

Line 37 sampai 40, Ketika pengguna menekan enter/selesai setelah mengisi EditText, fungsi calculateTip() akan dipanggil.

Line 42 sampai 50, Ketika pengguna memilih item dari spinner, akan memanggil calculateTip().

Line 52 sampai 54, Ketika switch diubah (on/off), akan memicu kalkulasi ulang.

Line 57 sampai 58, Mengambil teks dari EditText, mengubah ke Double. Kalau kosong atau bukan angka, hasilnya null.

Line 60 sampai 63, Jika input tidak valid, tampilkan placeholder dan keluar dari fungsi.

Line 65 sampai 69, Menentukan persentase tip sesuai pilihan dari spinner.

Line 71 sampai 74, Mengalikan cost dengan tipPercentage. Jika switch aktif, hasilnya dibulatkan ke atas.

Line 76 dan 77, Format hasil ke dalam mata uang lokal (misal: Rp jika device disetel ke Indonesia dan \$ jika device disetel ke US) lalu ditampilkan ke TextView.

### **Jetpack Compose :**

#### **MainActivity.kt:**

Line 1, Menyatakan bahwa file ini berada dalam package com.example.tipcalculatorcompose.

Line 3 sampai 16, Mengimpor semua komponen yang dibutuhkan, termasuk:

ComponentActivity, setContent() untuk activity Compose.

Modifier, Column, Text, OutlinedTextField, Switch, Dropdown, dll dari Compose. remember, mutableStateOf untuk state management.

NumberFormat dan ceil untuk menghitung dan memformat tip.

Line 18 sampai 23, MainActivity adalah entry point dari aplikasi. setContent { TipCalculatorApp() } akan menampilkan UI dari fungsi TipCalculatorApp().

Line 25 sampai 30, Fungsi Compose utama yang membungkus UI dalam sebuah Surface, fillMaxSize() membuatnya memenuhi seluruh layar, lalu memanggil fungsi TipCalculatorLayout() untuk isi layoutnya.

Line 34, Menyimpan input pengguna untuk jumlah uang (EditText).

Line 35, Menyimpan pilihan tip (%) dengan default 20%.

Line 36, Menyimpan status switch untuk membulatkan tip atau tidak dengan default off (false).

Line 38, Daftar opsi dropdown tip.

Line 39, Mengambil angka dari selectedTip (misal "20%") menjadi 20.0.

Line 40, Mengubah input pengguna ke Double, default ke 0.0 jika kosong/invalid.

Line 41 dan 43, Memanggil calculateTip dan memformat hasilnya menjadi bentuk uang.

Line 45 sampai 49, Mengatur layout vertikal dengan padding 16dp dan lebar penuh.

Line 51, Menampilkan teks judul aplikasi yaitu "Tip Calculator".

Line 52, spacer untuk memberikan jarak sebesar 16dp

Line 54 sampai 60, Input untuk jumlah layanan (serviceAmount), hanya menerima angka (keyboardType = Number).

Line 64 sampai 68, Dropdown untuk memilih persentase tip.  
Line 72 sampai 79, Baris dengan teks dan switch untuk membulatkan tip.  
Line 82, Menampilkan hasil tip dalam bentuk mata uang.  
Line 87 sampai 93, Fungsi dropdown custom dengan ExposedDropdownMenuBox dari Material 3.  
Line 94, Menyimpan status apakah dropdown sedang terbuka atau tidak.  
Line 96 sampai 100, Komponen dari Material 3 untuk dropdown menu terintegrasi dengan OutlinedTextField.  
Line 101 sampai 110, Text field yang menampilkan pilihan saat ini (tidak bisa diketik/manual).  
Line 111 sampai 125, daftar opsi yang muncul ketika dropdown dibuka.  
Ketika item diklik: Memanggil onSelect() untuk mengganti nilai dan menutup dropdown (expanded = false).  
Line 129 sampai 133, Menghitung tip dari amount \* persen. Jika roundUp == true, hasilnya dibulatkan ke atas dengan ceil().  
Line 135 sampai 139, Untuk menampilkan preview UI di Android Studio tanpa menjalankan app-nya.

#### **D. Tautan Git**

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul2>

## SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

### A. Pembahasan

Perbedaan implementasi XML dan Jetpack Compose terletak pada pendekatan dan cara mendesain tampilan. XML menggunakan deklarasi UI secara terpisah dalam file layout .xml, sedangkan Jetpack Compose menggunakan kode Kotlin langsung untuk mendefinisikan UI secara deklaratif.

Di XML, UI didefinisikan secara statis dalam file XML, lalu dikaitkan dengan logika di file Kotlin/Java menggunakan findViewById() atau view binding. Kelebihannya adalah banyak developer sudah familiar, banyak referensi dan dokumentasi yang tersedia, serta mudah diintegrasikan dengan berbagai library UI lama. Kekurangannya, proses pengembangan terasa lebih terpisah antara UI dan logika, membutuhkan lebih banyak boilerplate code, dan cenderung kurang fleksibel saat UI kompleks atau dinamis.

Di Jetpack Compose, UI dibangun secara deklaratif di dalam kode Kotlin, tanpa perlu file XML terpisah. Kelebihannya, kode menjadi lebih ringkas, lebih mudah dibaca, dan lebih fleksibel. Compose mendukung state management secara langsung sehingga lebih mudah untuk membuat UI dinamis dan interaktif. Kekurangannya, masih terbilang baru sehingga dokumentasi atau dukungan komunitas belum sebanyak XML, dan belum semua fitur Android mendukung Compose secara penuh (terutama untuk project lama).

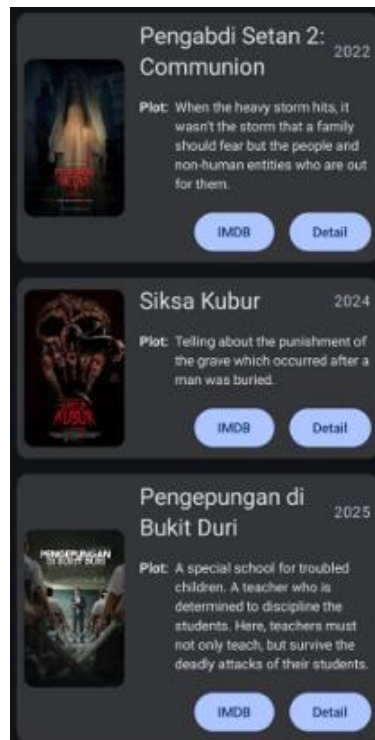
## **MODUL 3 : Build A Scrollable List**

### **SOAL 1**

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
  - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
  - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Dusahakan agar desain UI item list menyerupai UI berikut:



*Gambar 18. Contoh UI List*

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 19. Contoh UI Detail

## A. Source Code

### Jetpack Compose :

#### MainActivity.kt :

Tabel 8. Source Code Jawaban Soal 1 Jetpack Compose

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.example.listcompose                          |
| 2  |                                                          |
| 3  | import android.os.Bundle                                 |
| 4  | import androidx.activity.ComponentActivity               |
| 5  | import androidx.activity.compose.setContent              |
| 6  | import androidx.navigation.NavType                       |
| 7  | import androidx.navigation.compose.NavHost               |
| 8  | import androidx.navigation.compose.composable            |
| 9  | import androidx.navigation.compose.rememberNavController |
| 10 | import androidx.navigation.navArgument                   |
| 11 | import com.example.listcompose.ui.theme.ListComposeTheme |
| 12 |                                                          |
| 13 | class MainActivity : ComponentActivity() {               |
| 14 | override fun onCreate(savedInstanceState: Bundle?) {     |

```

15         super.onCreate(savedInstanceState)
16         setContent {
17             ListComposeTheme {
18                 val navController =
19 rememberNavController()
20                 NavHost(navController, startDestination
21 = "list") {
22                     composable("list") {
23                         ItemListScreen(navController)
24                     }
25                     composable(
26                         "detail/{itemId}",
27                         arguments =
28 listOf(navArgument("itemId") { type = NavType.IntType })
29                     ) { backStackEntry ->
30                         val itemId =
31 backStackEntry.arguments?.getInt("itemId") ?: 0
32                         DetailScreen(item =
33 ItemData.items[itemId])
34                     }
35                 }
16             }
17         }
18     }
19 }

```

## Item.kt :

Tabel 9. Source Code Jawaban Soal 1 Jetpack Compose

```

1 package com.example.listcompose
2
3 data class Item(
4     val id: Int,
5     val title: String,
6     val author: String,
7     val info1: String,
8     val info2: String,
9     val imageResId: Int,
10    val url: String
11 )
12
13 object ItemData {
14     val items = listOf(
15         Item(0, "Lookism", "PTJ", "Action, Fighting,
16 Comedy, Romance", "Status: OnGoing", R.drawable.lookism,
17 "https://lookism.fandom.com/wiki/Lookism"),
18         Item(1, "Bones", "This, Picture: Mu Mu-min",

```



```

1 "Action, Fighting", "Status: OnGoing", R.drawable.bones,
4 "https://en.namu.wiki/w/%EB%B8%EC%A6%88(%EC%9B%B9%ED%8
1 8%B0)"),
5     Item(2, "WEE!!!", "Amoeba UwU", "Slice of Life,
1 Romance", "OnGoing", R.drawable.wee,
6 "https://webtoon.fandom.com/id/wiki/WEE!!!"),
1     Item(3, "Hero Has Returned", "FUNGBACK", "Action,
7 Fighting, Fantasy", "OnGoing",
1 R.drawable.herohasreturned, "https://hero-has-
8 returned.fandom.com/wiki/Hero_Has_Returned_Wiki"),
1     Item(4, "Study Group", "Blue String", "Action,
9 Fighting, Comedy", "OnGoing", R.drawable.studygroup,
2 "https://study-group.fandom.com/wiki/Study_Group_Wiki")
0     )
2 }
1

```

### ItemListScreen.kt :

*Tabel 10. Source Code Jawaban Soal 1 Jetpack Compose*

```

1 package com.example.listcompose
2
3 import android.content.Intent
4 import android.net.Uri
5 import androidx.compose.foundation.Image
6 import androidx.compose.foundation.layout.Arrangement
7 import androidx.compose.foundation.layout.Column
8 import androidx.compose.foundation.layout.PaddingValues
9 import androidx.compose.foundation.layout.Row
10 import androidx.compose.foundation.layout.Spacer
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.lazy.LazyColumn
15 import
16 androidx.compose.foundation.shape.RoundedCornerShape
17 import androidx.compose.material3.Button
18 import androidx.compose.material3.Card
19 import androidx.compose.material3.CardDefaults
20 import androidx.compose.material3.MaterialTheme
21 import androidx.compose.material3.Text
22 import androidx.compose.runtime.Composable
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.draw.clip
25 import androidx.compose.ui.graphics.Color
26 import androidx.compose.ui.layout.ContentScale
27 import androidx.compose.ui.platform.LocalContext
28 import androidx.compose.ui.res.painterResource

```

```

29 import androidx.compose.ui.unit.dp
30 import androidx.navigation.NavController
31
32 @Composable
33 fun ItemListScreen(navController: NavController) {
34     LazyColumn(contentPadding = PaddingValues(8.dp)) {
35         items(ItemData.items.size) { index ->
36             val item = ItemData.items[index]
37             Card(
38                 shape = RoundedCornerShape(16.dp),
39                 modifier = Modifier
40                     .padding(8.dp)
41                     .fillMaxWidth(),
42                 colors =
43                 CardDefaults.cardColors(containerColor =
44                 Color.DarkGray),
45                 elevation =
46                 CardDefaults.cardElevation(defaultElevation = 4.dp)
47             ) {
48                 Column(modifier =
49                 Modifier.padding(16.dp)) {
50                     Image(
51                         painter = painterResource(id =
52                         item.imageResId),
53                         contentDescription = item.title,
54                         modifier = Modifier
55                             .fillMaxWidth()
56                             .height(180.dp)
57                     )
58                     .clip(RoundedCornerShape(16.dp)),
59                     contentScale = ContentScale.Crop
60                 )
61                 Spacer(modifier =
62                 Modifier.height(8.dp))
63                 Text(text = item.title, style =
64                 MaterialTheme.typography.titleLarge, color =
65                 Color.White)
66                 Text(text = item.author, style =
67                 MaterialTheme.typography.bodyMedium, color =
68                 Color.White)
69                 Text(text = item.info1, style =
70                 MaterialTheme.typography.bodySmall, color = Color.White)
71                 Text(text = item.info2, style =
72                 MaterialTheme.typography.bodySmall, color = Color.White)
73                 Spacer(modifier =
74                 Modifier.height(8.dp))
75                 Row(horizontalArrangement =
76                 Arrangement.spacedBy(8.dp)) {
77                     val context =

```

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 78 | <pre> LocalContext.current                     Button(onClick = {                         val intent = Intent(Intent.ACTION_VIEW, Uri.parse(item.url))  context.startActivity(intent)                     }) {                         Text("Lihat Web")                     }                     Button(onClick = {  navController.navigate("detail/\${item.id}")                     }) {                         Text("Detail")                     }                 }             }         }     } } </pre> |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### DetailScreen.kt :

*Tabel 11. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                      |
|----|------------------------------------------------------|
| 1  | package com.example.listcompose                      |
| 2  |                                                      |
| 3  | import androidx.compose.foundation.Image             |
| 4  | import androidx.compose.foundation.layout.*          |
| 5  | import                                               |
| 6  | androidx.compose.foundation.shape.RoundedCornerShape |
| 7  | import androidx.compose.material3.*                  |
| 8  | import androidx.compose.runtime.Composable           |
| 9  | import androidx.compose.ui.Modifier                  |
| 10 | import androidx.compose.ui.draw.clip                 |
| 11 | import androidx.compose.ui.layout.ContentScale       |
| 12 | import androidx.compose.ui.res.painterResource       |
| 13 | import androidx.compose.ui.unit.dp                   |
| 14 |                                                      |
| 15 | @Composable                                          |
| 16 | fun DetailScreen(item: Item) {                       |
| 17 | Column(modifier = Modifier.padding(16.dp)) {         |
| 18 | Image(                                               |
| 19 | painter = painterResource(id =                       |
| 20 | item.imageResId),                                    |
| 21 | contentDescription = item.title,                     |
| 22 | modifier = Modifier                                  |

|    |                                                        |
|----|--------------------------------------------------------|
| 23 | <code>.fillMaxWidth()</code>                           |
| 24 | <code>.height(240.dp)</code>                           |
| 25 | <code>.clip(RoundedCornerShape(16.dp)),</code>         |
| 26 | <code>contentScale = ContentScale.Crop</code>          |
| 27 | <code>)</code>                                         |
| 28 | <code>Spacer(modifier = Modifier.height(16.dp))</code> |
| 29 | <code>Text(text = item.title, style =</code>           |
| 30 | <code>MaterialTheme.typography.headlineMedium)</code>  |
| 31 | <code>Text(text = item.author, style =</code>          |
| 32 | <code>MaterialTheme.typography.bodyLarge)</code>       |
|    | <code>Text(text = item.info1, style =</code>           |
|    | <code>MaterialTheme.typography.bodyMedium)</code>      |
|    | <code>Text(text = item.info2, style =</code>           |
|    | <code>MaterialTheme.typography.bodyMedium)</code>      |
|    | <code>}</code>                                         |
|    | <code>}</code>                                         |

**XML :**

**Item.kt :**

*Tabel 12. Source Code Jawaban Soal 1 XML*

|   |                               |                                  |
|---|-------------------------------|----------------------------------|
| 1 | <code>package</code>          | <code>com.example.listxml</code> |
| 2 |                               |                                  |
| 3 | <code>data</code>             | <code>class Item(</code>         |
| 4 | <code>val</code>              | <code>imageResId: Int,</code>    |
| 5 | <code>val</code>              | <code>title: String,</code>      |
| 6 | <code>val</code>              | <code>author: String,</code>     |
| 7 | <code>val url: String)</code> |                                  |

**ItemAdapter.kt :**

*Tabel 13. Source Code Jawaban Soal 1 XML*

|   |                                                                     |
|---|---------------------------------------------------------------------|
| 1 | <code>package com.example.listxml</code>                            |
| 2 |                                                                     |
| 3 | <code>import android.view.LayoutInflater</code>                     |
| 4 | <code>import android.view.ViewGroup</code>                          |
| 5 | <code>import androidx.recyclerview.widget.RecyclerView</code>       |
| 6 | <code>import com.example.listxml.databinding.ItemListBinding</code> |
| 7 |                                                                     |
| 8 | <code>class ItemAdapter(</code>                                     |

|    |                                                          |
|----|----------------------------------------------------------|
| 9  | private val items: List<Item>,                           |
| 10 | private val onOpenClick: (String) -> Unit,               |
| 11 | private val onDetailClick: (Item) -> Unit                |
| 12 | ) : RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() { |
| 13 |                                                          |
| 14 | inner class ItemViewHolder(val binding:                  |
| 15 | ItemListBinding) : RecyclerView.ViewHolder(binding.root) |
| 16 |                                                          |
| 17 | override fun onCreateViewHolder(parent: ViewGroup,       |
| 18 | viewType: Int): ItemViewHolder {                         |
| 19 | val binding =                                            |
| 20 | ItemListBinding.inflate(LayoutInflater.from(parent.conte |
| 21 | xt), parent, false)                                      |
| 22 | return ItemViewHolder(binding)                           |
| 23 | }                                                        |
| 24 |                                                          |
| 25 | override fun onBindViewHolder(holder:                    |
| 26 | ItemViewHolder, position: Int) {                         |
| 27 | val item = items[position]                               |
| 28 | with(holder.binding) {                                   |
| 29 | itemImage.setImageResource(item.imageResId)              |
| 30 | titleText.text = item.title                              |
| 31 | subtitleText.text = item.author                          |
| 32 |                                                          |
| 33 | openButton.setOnClickListener {                          |
| 34 | onOpenClick(item.url)                                    |
| 35 | }                                                        |
| 36 | detailButton.setOnClickListener {                        |
| 37 | onDetailClick(item)                                      |
| 38 | }                                                        |
| 39 | }                                                        |
|    |                                                          |
|    | override fun getItemCount(): Int = items.size            |
|    | }                                                        |

**item\_list.xml :**

*Tabel 14. Source Code Jawaban Soal 1 XML*

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                   |
| 2  | <androidx.cardview.widget.CardView                       |
| 3  |                                                          |
| 4  | xmlns:android="http://schemas.android.com/apk/res/androi |
| 5  | d"                                                       |
| 6  | xmlns:card_view="http://schemas.android.com/apk/res-     |
| 7  | auto"                                                    |
| 8  | xmlns:app="http://schemas.android.com/apk/res-auto"      |
| 9  | android:layout_width="match_parent"                      |
| 10 | android:layout_height="wrap_content"                     |

```

11     android:layout_marginBottom="12dp"
12     card_view:cardCornerRadius="16dp"
13     card_view:cardElevation="4dp">
14
15     <LinearLayout
16         android:layout_width="match_parent"
17         android:layout_height="wrap_content"
18         android:orientation="vertical"
19         android:padding="16dp">
20
21
22     <com.google.android.material.imageview.ShapeableImageVie
23 w
24         android:id="@+id/itemImage"
25         android:layout_width="match_parent"
26         android:layout_height="180dp"
27         android:scaleType="centerCrop"
28
29     app:shapeAppearanceOverlay="@style/RoundedImageView"/>
30
31     <TextView
32         android:id="@+id/titleText"
33         android:layout_width="wrap_content"
34         android:layout_height="wrap_content"
35         android:text="Judul"
36         android:textStyle="bold"
37         android:textSize="18sp"
38         android:paddingTop="8dp"/>
39
40     <TextView
41         android:id="@+id/subtitleText"
42         android:layout_width="wrap_content"
43         android:layout_height="wrap_content"
44         android:text="Subjudul"
45
46     <LinearLayout
47         android:layout_width="match_parent"
48         android:layout_height="wrap_content"
49         android:orientation="horizontal"
50         android:gravity="end">
51
52     <Button
53         android:id="@+id/openButton"
54         android:layout_width="wrap_content"
55         android:layout_height="wrap_content"
56         android:text="Open"
57
58     <Button
59         android:id="@+id/detailButton"

```

|    |                                                                                                                                                                                                                                                                                                                                                                                          |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 60 | <pre>                         android:layout_width="wrap_content"                         android:layout_height="wrap_content"                         android:text="Detail"                         android:layout_marginStart="8dp"/&gt;                     &lt;/LinearLayout&gt;                 &lt;/LinearLayout&gt;             &lt;/androidx.cardview.widget.CardView&gt; </pre> |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### HomeFragment.kt :

*Tabel 15. Source Code Jawaban Soal 1 XML*

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | package com.example.listxml                             |
| 2  |                                                         |
| 3  | import android.content.Intent                           |
| 4  | import android.net.Uri                                  |
| 5  | import android.os.Bundle                                |
| 6  | import android.view.LayoutInflater                      |
| 7  | import android.view.View                                |
| 8  | import android.view.ViewGroup                           |
| 9  | import androidx.fragment.app.Fragment                   |
| 10 | import androidx.navigation.fragment.findNavController   |
| 11 | import androidx.recyclerview.widget.LinearLayoutManager |
| 12 | import                                                  |
| 13 | com.example.listxml.databinding.FragmentHomeBinding     |
| 14 |                                                         |
| 15 | class HomeFragment : Fragment() {                       |
| 16 | private var _binding: FragmentHomeBinding? = null       |
| 17 | private val binding get() = _binding!!                  |
| 18 |                                                         |
| 19 | override fun onCreateView(                              |
| 20 | inflater: LayoutInflater, container: ViewGroup?,        |
| 21 | savedInstanceState: Bundle?                             |
| 22 | ): View {                                               |
| 23 | _binding = FragmentHomeBinding.inflate(inflater,        |
| 24 | container, false)                                       |
| 25 | return binding.root                                     |
| 26 | }                                                       |
| 27 |                                                         |
| 28 | override fun onViewCreated(view: View,                  |
| 29 | savedInstanceState: Bundle?) {                          |
| 30 | val itemList = listOf(                                  |
| 31 | Item(R.drawable.lookism, "Lookism", "PTJ",              |
| 32 | "https://example.com"),                                 |
| 33 | Item(R.drawable.bones, "Bones", "This,                  |
| 34 | Picture : Mu Mu-min", "https://example.com"),           |
| 35 | Item(R.drawable.wee, "WEE!!!", "Amoeba UwU",            |
| 36 | "https://example.com"),                                 |

|    |                                                       |
|----|-------------------------------------------------------|
| 37 | Item(R.drawable.herohasreturned, "Hero Has            |
| 38 | Returned", "FUNGBACK", "https://example.com"),        |
| 39 | Item(R.drawable.studygroup, "Study Group",            |
| 40 | "BlueString", "https://example.com")                  |
| 41 | )                                                     |
| 42 |                                                       |
| 43 | val adapter = ItemAdapter(itemList,                   |
| 44 | onOpenClick = { url ->                                |
| 45 | val intent = Intent(Intent.ACTION_VIEW,               |
| 46 | Uri.parse(url))                                       |
| 47 | startActivity(intent)                                 |
| 48 | },                                                    |
| 49 | onDetailClick = { item ->                             |
| 50 | val action =                                          |
| 51 | HomeFragmentDirections.actionHomeToDetail(item.title, |
| 52 | item.author, item.imageResId)                         |
| 53 | findNavController().navigate(action)                  |
|    | })                                                    |
|    |                                                       |
|    | binding.recyclerView.layoutManager =                  |
|    | LinearLayoutManager(requireContext())                 |
|    | binding.recyclerView.adapter = adapter                |
|    | }                                                     |
|    |                                                       |
|    | override fun onDestroyView() {                        |
|    | super.onDestroyView()                                 |
|    | _binding = null                                       |
|    | }                                                     |
|    | }                                                     |

### fragment\_home.xml :

Tabel 16. Source Code Jawaban Soal 1 XML

|   |                                                           |
|---|-----------------------------------------------------------|
| 1 | <?xml version="1.0" encoding="utf-8"?>                    |
| 2 | <androidx.constraintlayout.widget.ConstraintLayout        |
| 3 |                                                           |
| 4 | xmlns:android="http://schemas.android.com/apk/res/android |
| 5 | "                                                         |
| 6 | xmlns:app="http://schemas.android.com/apk/res-auto"       |
| 7 | android:layout_width="match_parent"                       |
| 8 | android:layout_height="match_parent">                     |



|   |                                                      |
|---|------------------------------------------------------|
| 9 |                                                      |
| 1 | <androidx.recyclerview.widget.RecyclerView           |
| 0 | android:id="@+id/recyclerView"                       |
| 1 | android:layout_width="0dp"                           |
| 1 | android:layout_height="0dp"                          |
| 1 | android:clipToPadding="false"                        |
| 2 | android:padding="16dp"                               |
| 1 | app:layoutManager=                                   |
| 3 | "androidx.recyclerview.widget.LinearLayoutManager"   |
| 1 | app:layout_constraintTop_toTopOf="parent"            |
| 4 | app:layout_constraintBottom_toBottomOf="parent"      |
| 1 | app:layout_constraintStart_toStartOf="parent"        |
| 5 | app:layout_constraintEnd_toEndOf="parent"/>          |
| 1 | </androidx.constraintlayout.widget.ConstraintLayout> |
| 6 |                                                      |
| 1 |                                                      |
| 7 |                                                      |
| 1 |                                                      |
| 8 |                                                      |
| 1 |                                                      |
| 9 |                                                      |

## nav\_graph :

Tabel 17. Source Code Jawaban Soal 1 XML

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <navigation                                                |
| 3  | xmlns:android="http://schemas.android.com/apk/res/android" |
| 4  | xmlns:app="http://schemas.android.com/apk/res-auto"        |
| 5  | xmlns:tools="http://schemas.android.com/tools"             |
| 6  | android:id="@+id/nav_graph"                                |
| 7  | app:startDestination="@id/homeFragment">                   |
| 8  |                                                            |
| 9  | <fragment                                                  |
| 10 | android:id="@+id/homeFragment"                             |
| 11 | android:name="com.example.listxml.HomeFragment"            |
| 12 | android:label="Home"                                       |
| 13 | tools:layout="@layout/fragment_home">                      |
| 14 | <action                                                    |
| 15 | android:id="@+id/action_home_to_detail"                    |
| 16 | app:destination="@id/detailFragment" />                    |
| 17 | </fragment>                                                |
| 18 |                                                            |
| 19 | <fragment                                                  |
| 20 | android:id="@+id/detailFragment"                           |
| 21 | android:name="com.example.listxml.DetailFragment"          |
| 22 | android:label="Detail"                                     |
| 23 | tools:layout="@layout/fragment_detail">                    |

|    |                           |
|----|---------------------------|
| 24 | <argument                 |
| 25 | android:name="title"      |
| 26 | app:argType="string" />   |
| 27 | <argument                 |
| 28 | android:name="subtitle"   |
| 29 | app:argType="string" />   |
| 30 | <argument                 |
| 31 | android:name="imageResId" |
| 32 | app:argType="integer" />  |
| 33 | </fragment>               |
|    | </navigation>             |

### DetailFragment.kt :

*Tabel 18. Source Code Jawaban Soal 1 XML*

|    |                                                       |
|----|-------------------------------------------------------|
| 1  | package com.example.listxml                           |
| 2  |                                                       |
| 3  | import android.os.Bundle                              |
| 4  | import android.view.LayoutInflater                    |
| 5  | import android.view.View                              |
| 6  | import android.view.ViewGroup                         |
| 7  | import androidx.fragment.app.Fragment                 |
| 8  | import androidx.navigation.fragment.navArgs           |
| 9  | import                                                |
| 10 | com.example.listxml.databinding.FragmentDetailBinding |
| 11 |                                                       |
| 12 | class DetailFragment : Fragment() {                   |
| 13 | private var _binding: FragmentDetailBinding? = null   |
| 14 | private val binding get() = _binding!!                |
| 15 | private val args: DetailFragmentArgs by navArgs()     |
| 16 |                                                       |
| 17 | override fun onCreateView(                            |
| 18 | inflater: LayoutInflater, container: ViewGroup?,      |
| 19 | savedInstanceState: Bundle?                           |
| 20 | ): View {                                             |
| 21 | _binding = FragmentDetailBinding.inflate(inflater,    |
| 22 | container, false)                                     |
| 23 | return binding.root                                   |
| 24 | }                                                     |
| 25 |                                                       |
| 26 | override fun onViewCreated(view: View,                |
| 27 | savedInstanceState: Bundle?) {                        |
| 28 | binding.detailTitle.text = args.title                 |
| 29 | binding.detailSubtitle.text = args.subtitle           |
| 30 |                                                       |
| 31 | binding.detailImage.setImageResource(args.imageResId) |
| 32 | }                                                     |

|    |                                                                                                                                   |
|----|-----------------------------------------------------------------------------------------------------------------------------------|
| 33 |                                                                                                                                   |
| 34 | <pre>         override fun onDestroyView() {             super.onDestroyView()             _binding = null         }     } </pre> |

### Fragment\_detail.xml:

Tabel 19. Source Code Jawaban Soal 1 XML

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <ScrollView                                                |
| 3  | xmlns:android="http://schemas.android.com/apk/res/android" |
| 4  | xmlns:app="http://schemas.android.com/apk/res-auto"        |
| 5  | android:layout_width="match_parent"                        |
| 6  | android:layout_height="match_parent"                       |
| 7  | android:padding="16dp">                                    |
| 8  |                                                            |
| 9  | <LinearLayout                                              |
| 10 | android:layout_width="match_parent"                        |
| 11 | android:layout_height="wrap_content"                       |
| 12 | android:orientation="vertical">                            |
| 13 |                                                            |
| 14 |                                                            |
| 15 | <com.google.android.material.imageview.ShapeableImageView  |
| 16 | android:id="@+id/detailImage"                              |
| 17 | android:layout_width="match_parent"                        |
| 18 | android:layout_height="200dp"                              |
| 19 | android:scaleType="centerCrop"                             |
| 20 |                                                            |
| 21 | app:shapeAppearanceOverlay="@style/RoundedImageView" />    |
| 22 |                                                            |
| 23 | <TextView                                                  |
| 24 | android:id="@+id/detailTitle"                              |
| 25 | android:layout_width="wrap_content"                        |
| 26 | android:layout_height="wrap_content"                       |
| 27 | android:textStyle="bold"                                   |
| 28 | android:textSize="20sp"                                    |
| 29 | android:paddingTop="8dp" />                                |
| 30 |                                                            |
| 31 | <TextView                                                  |
| 32 | android:id="@+id/detailSubtitle"                           |
| 33 | android:layout_width="wrap_content"                        |
|    | android:layout_height="wrap_content" />                    |
|    | </LinearLayout>                                            |
|    | </ScrollView>                                              |

## MainActivity.kt:

Tabel 20. Source Code Jawaban Soal 1 XML

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | package com.example.listxml                                |
| 2  |                                                            |
| 3  | import android.os.Bundle                                   |
| 4  | import androidx.appcompat.app.AppCompatActivity            |
| 5  | import com.example.listxml.databinding.ActivityMainBinding |
| 6  |                                                            |
| 7  | class MainActivity : AppCompatActivity() {                 |
| 8  | private lateinit var binding: ActivityMainBinding          |
| 9  |                                                            |
| 10 | override fun onCreate(savedInstanceState: Bundle?) {       |
| 11 | super.onCreate(savedInstanceState)                         |
| 12 | binding =                                                  |
| 13 | ActivityMainBinding.inflate(layoutInflater)                |
| 14 | setContentView(binding.root)                               |
| 15 | }                                                          |
|    | }                                                          |

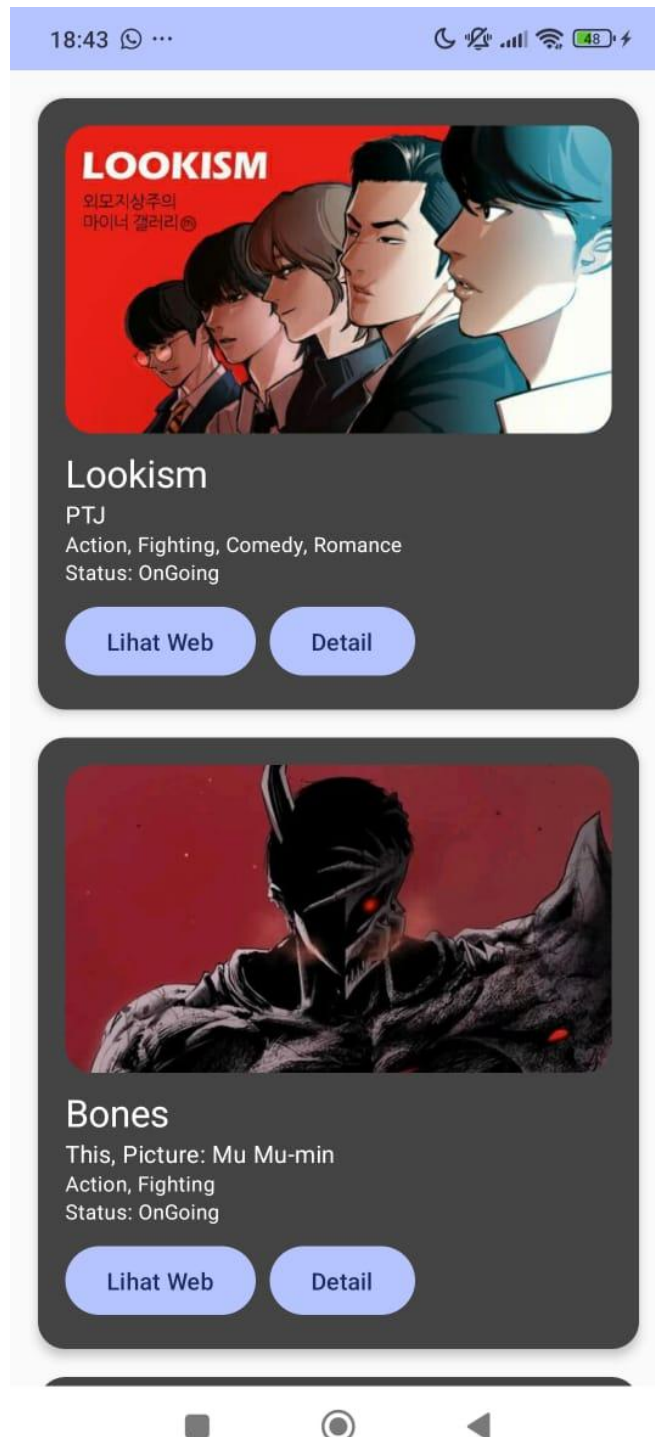
## activity\_main.xml:

Tabel 21. Source Code Jawaban Soal 1 XML

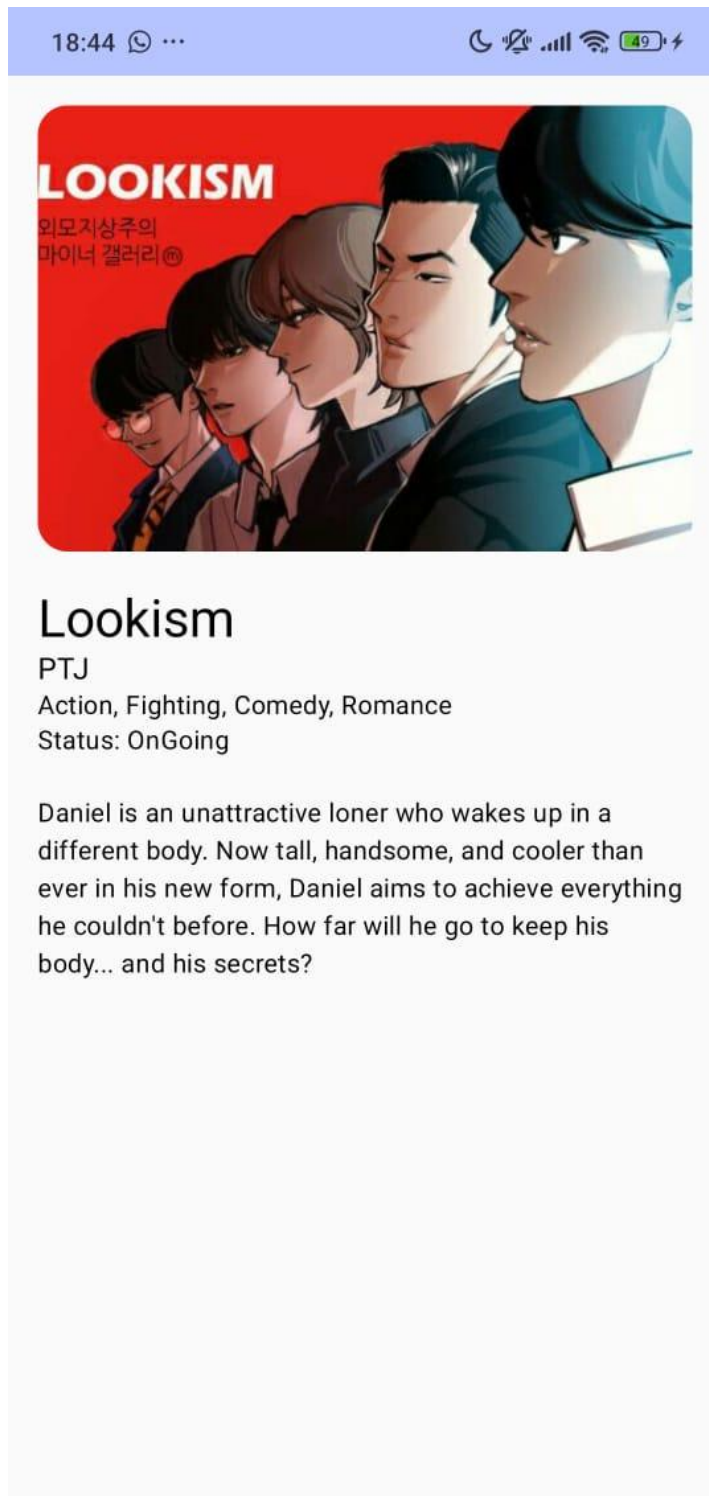
|    |                                                             |
|----|-------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                      |
| 2  | <androidx.fragment.app.FragmentContainerView                |
| 3  |                                                             |
| 4  | xmlns:android="http://schemas.android.com/apk/res/android"  |
| 5  | xmlns:app="http://schemas.android.com/apk/res-auto"         |
| 6  | android:id="@+id/nav_host_fragment"                         |
| 7  |                                                             |
| 8  | android:name="androidx.navigation.fragment.NavHostFragment" |
| 9  | android:layout_width="match_parent"                         |
| 10 | android:layout_height="match_parent"                        |
|    | app:navGraph="@navigation/nav_graph"                        |
|    | app:defaultNavHost="true" />                                |

## B. Output Program

Compose :

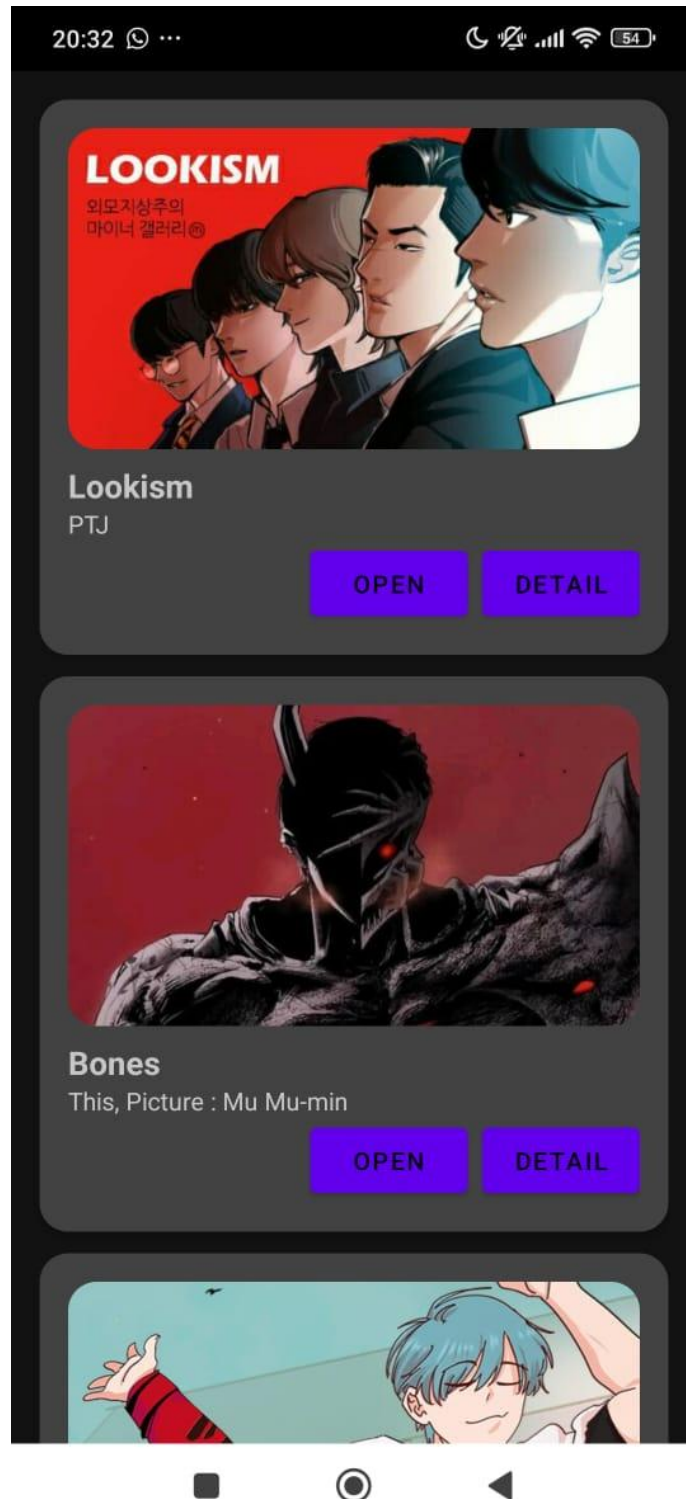


Gambar 20. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

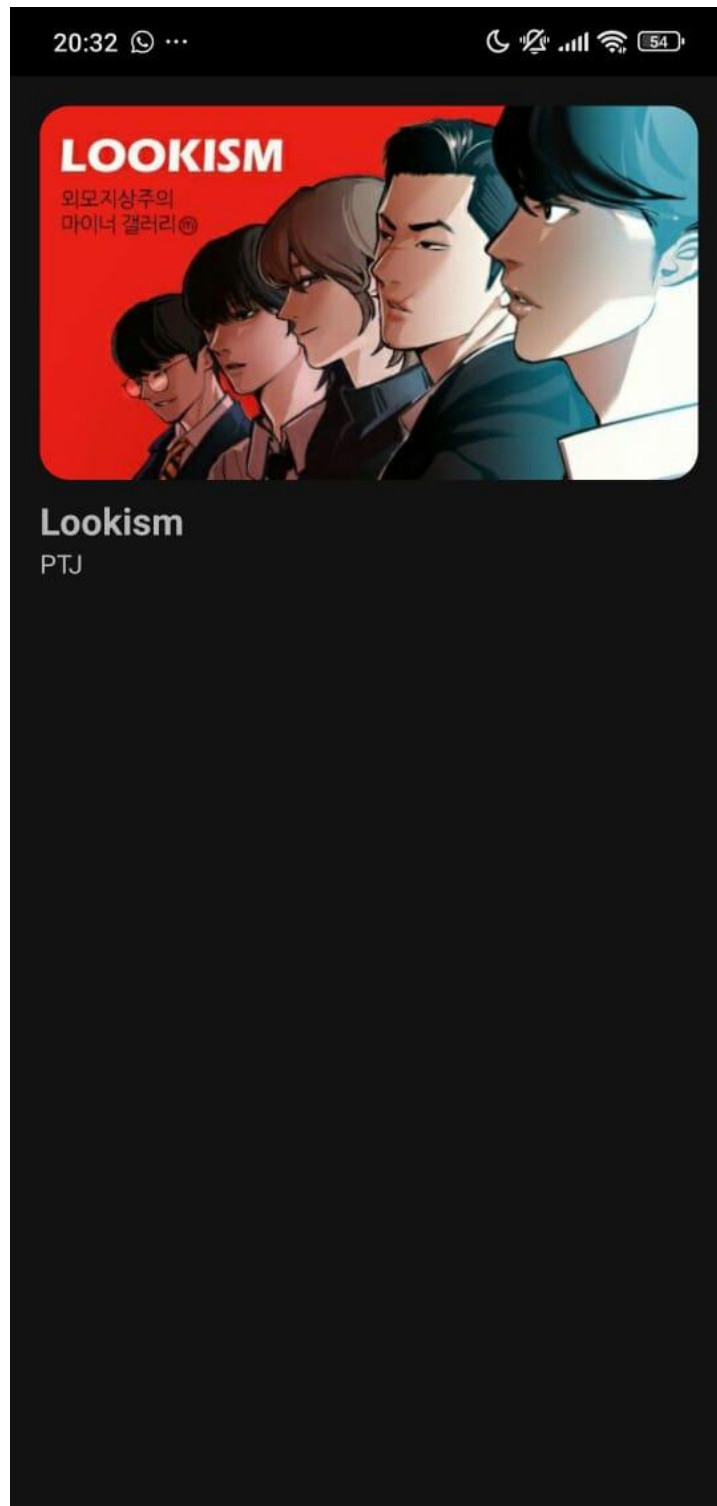


Gambar 21. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

XML :



Gambar 22. Screenshot Hasil Jawaban Soal 1 XML



Gambar 23. Screenshot Hasil Jawaban Soal 1 XML



## C. Pembahasan

### Jetpack Compose :

#### MainActivity.kt:

Line 18, Membuat dan menyimpan instance NavController (digunakan untuk navigasi antar layar). `rememberNavController()` memastikan controller tetap konsisten meskipun Compose melakukan recomposition (UI redraw).

Line 19, NavHost adalah **container navigasi Compose** yang menentukan layar mana yang akan ditampilkan berdasarkan route yang aktif. `startDestination = "list"` artinya layar pertama yang ditampilkan adalah dengan route "list".

Line 20 sampai 22, Mendefinisikan route "list", yaitu layar utama yang menampilkan daftar item. Saat route ini aktif, akan dipanggil fungsi:

`"ItemListScreen(navController)"` yang artinya akan menampilkan list (`ItemListScreen.kt`) dan membutuhkan NavController untuk navigasi saat tombol "Detail" ditekan.

Line 23 sampai 24, adalah rute untuk layar detail dengan parameter `itemId` sebagai bagian dari route. `ItemId` diambil dari Index di `ItemListScreen`

Line 25, Mendefinisikan bahwa `itemId` adalah parameter bertipe integer. Compose akan otomatis mengurai argumen tersebut dari string ke int saat route ini dipanggil.

Line 26 sampai 28, `BackStackEntry` berisi semua informasi tentang argumen dan status navigasi saat ini. `"val itemId = backStackEntry.arguments?.getInt("itemId") ?: 0"` Mengambil nilai `itemId` dari argumen. Jika tidak ada, default ke 0. Lalu memanggil: `DetailScreen(item = ItemData.items[itemId])` untuk menampilkan UI detail berdasarkan data item di `itemId`.

#### Item.kt :

Line 3 sampai 12, Ini mendeklarasikan **data class** bernama `Item`, yaitu blueprint untuk satu item yang akan ditampilkan di daftar (list) :

`val id: Int`, ID unik untuk tiap item. Bisa digunakan untuk indexing, navigasi, atau pembeda.

`val title: String`, judul dari item (misalnya judul webtoon atau komik).

`val author: String`, nama pembuat atau pengarang.

`val info1: String`, informasi tambahan 1, seperti genre.

`val info2: String`, informasi tambahan 2, seperti status (OnGoing, Completed).

val imageResId: Int, ID resource gambar (R.drawable.nama\_gambar) yang mereferensikan gambar di folder res/drawable.

val url: String, URL website atau sumber informasi item.

Line 13 sampai 21, Membuat object bernama ItemData yang berisi sebuah variabel items, yaitu list yang berisi beberapa objek dari data class Item. Setiap item memiliki data seperti ID, judul, penulis, genre, status, gambar, dan URL. Data ini nantinya digunakan untuk ditampilkan pada list dan detail screen.

### **DetailScreen.kt :**

Line 3 sampai 14, Import berbagai fungsi dan kelas dari Jetpack Compose yang digunakan untuk membangun UI. Image, Column, Spacer, Text merupakan elemen UI Compose. painterResource untuk memuat gambar dari res/drawable.

MaterialTheme mengambil gaya dari tema Material 3. Modifier dan dp untuk styling dan ukuran layout.

Line 16 sampai 17, @Composable memberitahu bahwa fungsi ini bisa digunakan untuk menyusun UI dalam Jetpack Compose. DetailScreen menerima parameter item bertipe Item, yang berisi data seperti gambar, judul, author, dll.

Line 18, mengelola dan menyimpan posisi scroll secara stateful pada layout yang bisa digulir.

Line 19 sampai 24, membuat layout vertical yang menempatkan elemen dari atas ke bawah, dan memberikan jarak di sekelilingnya sebesar 16dp

Line 25 sampai 32, digunakan untuk menampilkan gambar utama dari item, dengan pengaturan sebagai berikut: gambar dimuat menggunakan painterResource berdasarkan ID gambar (item.imageResId) yang berada di folder res/drawable, dan diberikan contentDescription berupa judul item untuk mendukung aksesibilitas (dibaca oleh screen reader). Gambar menggunakan Modifier agar memenuhi lebar layar (fillMaxWidth()), memiliki tinggi 240dp (height(240.dp)), dan sudutnya dibulatkan menggunakan clip(RoundedCornerShape(16.dp)). Selain itu, contentScale = ContentScale.Crop digunakan untuk memotong gambar agar pas dalam ukuran tersebut tanpa mengubah rasio aslinya.

Line 34, spacer untuk membuat jarak setinggi 16dp

Line 35 sampai 39, menampilkan informasi dari item secara berurutan, dimulai dari item.title sebagai judul dengan gaya teks besar, diikuti oleh item.author yang menunjukkan nama pengarang, lalu item.info1 sebagai genre atau kategori, dan terakhir item.info2 yang berisi status seperti "OnGoing"; setiap teks ini menggunakan

gaya dari `MaterialTheme.typography` yang mengikuti tampilan standar Material Design 3.

**ItemListScreen.kt :**

Line 3 sampai 29, Mengimpor berbagai komponen Compose, termasuk layout (`Column`, `Row`, `LazyColumn`, dll.), Material 3 (`Card`, `Button`, dll.), navigasi (`NavController`), dan Android Intent untuk membuka URL.

Line 31 dan 32, Fungsi composable utama yang menampilkan daftar item, `navController` digunakan untuk navigasi antar layar.

Line 33, Membuat membuat komponen tata letak vertikal yang bisa di scroll (mirip `RecyclerView`) dengan padding di sekeliling isi sebesar 8dp.

Line 34, melakukan loop sebanyak `ItemData.items` (saat ini ada 5) dan index adalah index item saat ini

Line 35, Mengambil item berdasarkan indeks.

Line 36 sampai 43, Membungkus setiap item dalam sebuah Card dengan sudut bulat (`RoundedCornerShape(16.dp)`), Padding 8dp, mengisi lebar sepenuh layer (di kode ini dikurangi oleh padding 8dp, warna latar gelap (`DarkGray`), dan elevasi bayangan 4dp.

Line 44, Menyusun isi card secara vertikal dengan padding dalam sebesar 16dp.

Line 45 sampai 53, Menampilkan gambar dari `res/drawable` berdasarkan `item.imageResId`. Lebar penuh (dikurangi padding 16dp), tinggi 180dp, sudut bulat. `contentScale = Crop`, agar gambar disesuaikan supaya penuh dan proporsional.

Line 54, Menambahkan jarak vertical setinggi 8dp.

Line 55 sampai 58, Menampilkan informasi `item.title`, `item.author`, `item.info1`, `item.info2` yang diambil dari `item.kt` dengan berbagai gaya teks dan warna putih.

Line 59, Menambahkan jarak vertical setinggi 8dp.

Line 60, `Row` menyusun tombol horizontal dengan jarak antar tombol 8dp.

Line 61, Mengakses Context Android lokal untuk membuka URL menggunakan Intent.

Line 62 sampai 65, Membuat button dengan teks "Lihat Web". Saat diklik, membuka browser dan menampilkan URL dari item menggunakan `Intent.ACTION_VIEW`.

Line 68 sampai 71, Saat diklik, navigasi ke halaman detail item berdasarkan `item.id` (0 sampai 4)

## **XML :**

### **Item.kt :**

Line 3, sebuah data class bernama Item

Line 4 sampai 7, merupakan objek-objek immutable (val) yang menyimpan id dari setiap gambar item (imageResId: Int), title atau judul item (title: String), nama pembuat, penulis, atau deskripsi tambahan. (author: String), dan URL yang akan dibuka jika tombol "Open" diklik. Biasanya berupa link web atau tautan aplikasi (url: String)

### **ItemAdapter.kt :**

Line 3 sampai 6, Berisikan import yang dibutuhkan :

LayoutInflater: untuk mengubah file XML layout menjadi objek View.

ViewGroup: induk dari layout.

RecyclerView: komponen utama untuk list.

ItemListAdapter: class binding otomatis yang dihasilkan dari file item\_list.xml.

Line 8 sampai 12, Adapter menerima sebuah list data (items) bertipe List<Item> untuk ditampilkan, serta dua lambda function yaitu onItemClick bertipe (String) -> Unit yang menangani klik tombol "Open" dan menerima URL sebagai parameter, dan onDetailClick bertipe (Item) -> Unit yang menangani klik tombol "Detail" dengan membawa seluruh data Item sebagai parameter. Unit dalam Kotlin setara dengan void di Java, artinya fungsi tersebut tidak mengembalikan nilai apa pun; adapter ini mewarisi dari RecyclerView.Adapter dan menggunakan ItemViewHolder sebagai kelas ViewHolder-nya.

Line 14, mendefinisikan kelas ItemViewHolder sebagai tempat menyimpan referensi ke layout per item (item\_list.xml) secara efisien menggunakan ViewBinding, dan mewarisi RecyclerView.ViewHolder agar dapat digunakan oleh adapter untuk menampilkan data dalam RecyclerView.

Line 16 sampai 19, onCreateViewHolder dipanggil oleh RecyclerView saat membutuhkan tampilan item baru (belum ada yang bisa digunakan ulang), fungsi ini akan menggunakan LayoutInflater untuk meng-inflate file layout XML item\_list.xml menjadi objek tampilan (View), membungkusnya ke dalam objek ItemListAdapter agar komponen UI-nya bisa diakses langsung tanpa findViewById, lalu membentuk dan mengembalikan ItemViewHolder yang menyimpan view tersebut sebagai wakil satu baris item di dalam list.

Line 21 dan 22, menghubungkan data ke tampilan item yang sudah dibuat oleh RecyclerView, dengan cara mengambil data Item dari list berdasarkan indeks position yang menunjukkan posisi item tersebut di dalam daftar, lalu memanfaatkan objek holder (yang berisi binding tampilan) untuk menampilkan data seperti gambar, judul, dan informasi lain ke tampilan.

Line 23 sampai 26, Men-set gambar, judul, dan penulis ke elemen UI di item\_list.xml.

Line 28 sampai 33, Tombol openButton akan menjalankan fungsi onOpenClick dengan parameter URL item. Tombol detailButton akan menjalankan fungsi onDetailClick dengan parameter seluruh data Item.

### **Item\_list.xml :**

Line 2, komponen dari AndroidX yang menampilkan isi dalam bentuk kartu dengan sudut melengkung dan bayangan (elevation). Ideal untuk membuat tampilan item yang rapi dan modern di dalam list.

Line 3 dan 4, Mendefinisikan namespace:

xmlns:android digunakan untuk atribut standar Android.

xmlns:card\_view digunakan untuk atribut khusus CardView, seperti cardCornerRadius dan cardElevation.

Line 5 dan 6, Ukuran CardView akan mengisi seluruh lebar parent (match\_parent) dan tingginya menyesuaikan dengan isi kontennya (wrap\_content).

Line 7, Memberikan jarak ke bawah antar item dalam list, sehingga item tidak terlihat terlalu rapat satu sama lain.

Line 8, Membuat sudut kartu melengkung dengan radius 16dp untuk tampilan lebih lembut dan modern.

Line 9, membuat sudut luar CardView melengkung dengan radius 16dp.

Line 10, memberi efek bayangan agar kartu tampak mengambang (elevated look).

Line 12 sampai 16, Kontainer vertikal yang menyusun isi kartu dari atas ke bawah: gambar, teks, dan tombol. Diberi padding 16dp agar isi tidak menempel ke tepi kartu.

Line 18 sampai 23 komponen gambar dari Material Components.  
shapeAppearanceOverlay="@style/RoundedImageView" memberi radius pada sudut gambar, membuat tampilannya membulat (rounded corner). Lebar penuh, tinggi tetap

180dp. `scaleType="centerCrop"` menjaga proporsi gambar agar mengisi view secara penuh tanpa distorsi.

Line 25 – 32, menampilkan judul item. `textStyle="bold"` dan `textSize="18sp"` membuatnya menonjol. `paddingTop="8dp"` memberi jarak dari gambar.

Line 34 sampai 38, menampilkan subjudul atau deskripsi singkat. Ditempatkan langsung di bawah judul.

Line 40 sampai 44, menyusun tombol secara horizontal. `gravity="end"` memastikan kedua tombol disejajarkan ke kanan.

Line 46 sampai 50 Tombol untuk membuka sesuatu, bisa diarahkan ke fitur atau aksi utama.

Line 52 sampai 57, tombol untuk menampilkan detail item. Diberi margin kiri 8dp agar tidak terlalu menempel dengan tombol "Open".

### **HomeFragment.kt :**

Line 3 sampai 12, import semua library yang dibutuhkan:

Intent dan Uri: untuk membuka browser menggunakan intent eksplisit.

Fragment: karena class ini merupakan turunan dari Fragment.

NavController: digunakan untuk navigasi antar fragment.

LinearLayoutManager: digunakan untuk menampilkan list secara vertikal.

FragmentHomeBinding: menghubungkan layout `fragment_home.xml` dengan kode menggunakan ViewBinding.

Line 14, Deklarasi class fragment utama bernama HomeFragment. Class ini digunakan untuk menampilkan daftar item.

Line 15 dan 16, `_binding` menyimpan binding layout, nilainya bisa null. `binding` mengakses `_binding` dengan jaminan tidak null (!!) untuk digunakan di seluruh fragment.

Line 18 sampai 24, Ketika `onCreateView` dipanggil, `LayoutInflater` digunakan untuk mengubah layout XML `fragment_home.xml` menjadi objek tampilan nyata di aplikasi. Proses ini dilakukan melalui `FragmentHomeBinding.inflate`, yang secara otomatis menghubungkan semua elemen UI dalam layout dengan kode menggunakan ViewBinding. Hasil dari proses ini kemudian dikembalikan melalui `binding.root`, yaitu tampilan utama (root view (View)) dari fragment tersebut yang akan ditampilkan di layar.

Line 26, Dipanggil setelah onCreateView selesai dan layout sudah siap ditampilkan.

Line 27 sampai 33, Membuat daftar item yang akan ditampilkan di RecyclerView. Setiap item berisi parameter yang diambil dari item.kt yaitu gambar (imageResId), judul (title), penulis (author), dan URL (url).

Line 35 sampai 39, Membuat objek adapter yang diberi itemList sebagai data. onClick digunakan ketika tombol “Open” diklik, akan membuka URL di browser pakai Intent.ACTION\_VIEW.

Line 40 sampai 43, onClick digunakan untuk membuka URL milik suatu item ke browser saat tombol “Open” diklik, dengan memanfaatkan Intent.ACTION\_VIEW untuk membuat intent eksplisit.

Line 45 sampai 46, binding.recyclerView.layoutManager menentukan pola tampilan item (vertikal), sedangkan binding.recyclerView.adapter menetapkan data dan tampilan item yang akan ditampilkan di RecyclerView menggunakan ItemAdapter

Line 49 sampai 52, Supaya tidak terjadi memory leak, binding di-nol-kan saat tampilan sudah tidak digunakan.

#### **fragment\_home.xml :**

Line 2 sampai 6, mendefinisikan ConstraintLayout sebagai root layout.

ConstraintLayout adalah jenis layout fleksibel yang memungkinkan pengaturan posisi elemen berdasarkan hubungan antar elemen (constraints). xmlns:android dan xmlns:app adalah deklarasi namespace agar atribut Android standar (android:) dan atribut tambahan dari library (app:) dikenali. layout\_width="match\_parent" dan layout\_height="match\_parent" membuat layout mengisi seluruh lebar dan tinggi dari layar atau container-nya.

Line 8 sampai 18, mendefinisikan komponen RecyclerView, yaitu elemen tampilan berbasis list yang digunakan untuk menampilkan data dalam jumlah banyak secara efisien.

android:id="@+id/recyclerView" memberikan ID unik agar RecyclerView dapat diakses dari kode Kotlin menggunakan data binding (binding.recyclerView).

android:layout\_width="0dp" dan android:layout\_height="0dp" digunakan karena dalam ConstraintLayout, ukuran 0dp berarti mengikuti aturan constraint yang diberikan (disebut juga *match constraints*).

android:clipToPadding="false" membuat konten yang ditampilkan oleh RecyclerView tidak dipotong oleh padding, memungkinkan efek visual seperti bayangan atau overscroll.

`android:padding="16dp"` memberikan jarak 16dp di sekeliling RecyclerView agar item tidak menempel ke tepi layar, meningkatkan kenyamanan visual.

`app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"` menentukan jenis layout manager yang digunakan oleh RecyclerView. Dalam hal ini, LinearLayoutManager digunakan untuk menampilkan item secara vertikal satu per satu.

`app:layout_constraintTop_toTopOf="parent"` dan constraint lainnya mengatur posisi RecyclerView agar menempel ke keempat sisi parent-nya (ConstraintLayout), membuatnya memenuhi seluruh ruang tampilan.

Line 19, menutup elemen ConstraintLayout, menandai akhir dari struktur layout.

### **nav\_graph.xml :**

Line 2 sampai 6, `<navigation>` adalah tag utama untuk mendeklarasikan Navigation Graph.

`xmlns:android`, `xmlns:app`, dan `xmlns:tools` adalah deklarasi namespace yang dibutuhkan untuk atribut-atribut dari Android SDK, Jetpack, dan tools.

`android:id="@+id/nav_graph"` menetapkan ID untuk graph ini agar bisa direferensikan di kode atau layout lain.

`app:startDestination="@id/homeFragment"` menyatakan bahwa homeFragment adalah fragment pertama yang ditampilkan ketika aplikasi dibuka.

Line 8 sampai 16, `android:id="@+id/homeFragment"` memberi ID pada fragment agar bisa direferensikan di graph dan kode.

`android:name="com.example.listxml.HomeFragment"` menunjukkan class fragment yang digunakan.

`android:label="Home"` memberikan label tampilan, biasanya digunakan di toolbar/judul fragment.

`tools:layout="@layout/fragment_home"` adalah preview hint untuk Android Studio saat menampilkan layout preview.

Di dalamnya terdapat tag `<action>`: `android:id="@+id/action_home_to_detail"` adalah ID untuk aksi navigasi dari Home ke Detail.

`app:destination="@id/detailFragment"` menentukan tujuan navigasi, yaitu ke detailFragment.



Line 18 sampai 32, Menyediakan DetailFragment sebagai halaman tujuan dengan parameter atau data yang akan ditampilkan secara dinamis berdasarkan item yang diklik di HomeFragment.

Fragment ini akan menerima data dari HomeFragment melalui tiga argumen, yaitu:

title (tipe string), subtitle (tipe string), imageResId (tipe integer, biasanya ID resource gambar)

Line 33, Menutup elemen <navigation>, yang menandakan akhir dari definisi Navigation Graph.

### **DetailFragment.kt :**

Line 11, Mendeklarasikan class DetailFragment yang merupakan subclass dari Fragment.

Line 12 dan 13, `_binding` adalah variabel nullable yang menyimpan instance binding dari layout `fragment_detail.xml`. `binding` adalah versi non-nullable dari `_binding`, digunakan untuk akses aman ke elemen layout. Teknik ini digunakan untuk menghindari memory leak dengan cara menghapus binding saat view di-destroy.

Line 14, Mendeklarasikan variabel `args` yang otomatis terisi oleh argument yang dikirim dari HomeFragment. `DetailFragmentArgs` adalah class yang secara otomatis dibuat oleh Navigation Component berdasarkan deklarasi di `nav_graph.xml`

Line 16 sampai 22, `onCreateView()` dipanggil saat fragment akan menampilkan layout-nya. `FragmentDetailBinding.inflate(...)` digunakan untuk menghubungkan layout XML ke fragment menggunakan View Binding. `binding.root` mengembalikan elemen root dari layout (yaitu `ConstraintLayout` dari `fragment_detail.xml`) sebagai tampilan utama fragment.

Line 24 sampai 28, Fungsi ini dipanggil setelah layout fragment selesai dibuat. Tiga baris di dalamnya mengatur isi tampilan berdasarkan argument yang diterima:

`binding.detailTitle.text = args.title`: Menampilkan judul yang dikirim.

`binding.detailSubtitle.text = args.subtitle`: Menampilkan subjudul/penulis.

`binding.detailImage.setImageResource(args.imageResId)`: Menampilkan gambar berdasarkan resource ID.

Line 30 sampai 33, Fungsi ini dipanggil saat tampilan fragment akan dihancurkan. `_binding` di-set ke null untuk menghindari kebocoran memori (memory leak) karena binding terikat dengan view yang sudah tidak ada.

### **Fragment\_detail.xml :**

Line 2 sampai 6, ScrollView adalah layout yang memungkinkan konten di dalamnya bisa di-scroll secara vertikal. xmlns:android dan xmlns:app adalah deklarasi namespace untuk atribut standar dan khusus., diperlukan agar atribut android: dan app: dikenali. layout\_width="match\_parent" dan layout\_height="match\_parent" membuat ScrollView mengisi seluruh lebar dan tinggi layar. padding="16dp" memberi jarak sebesar 16dp di sekeliling konten untuk tampilan yang lebih rapi dan tidak terlalu rapat ke tepi layar.

Line 8 sampai 11, LinearLayout digunakan untuk menata elemen UI secara **vertikal** (atas ke bawah). layout\_width="match\_parent", mengisi lebar kontainer (yaitu ScrollView). layout\_height="wrap\_content", tinggi menyesuaikan isi di dalamnya. orientation="vertical", menyusun elemen satu per satu dari atas ke bawah.

Line 13 sampai 18, ShapeableImageView digunakan untuk menampilkan gambar dari item yang dipilih. @+id/detailImage adalah ID yang digunakan untuk mengakses elemen ini di Kotlin. layout\_width="match\_parent", gambar mengisi seluruh lebar layar. layout\_height="200dp", tinggi gambar tetap, yaitu 200dp. scaleType="centerCrop", gambar dipotong dan diperbesar agar mengisi seluruh area, menjaga proporsi sambil menampilkan bagian tengah gambar. app:shapeAppearanceOverlay="@style/RoundedImageView": memberi efek sudut melengkung (rounded corner) pada gambar

Line 20 sampai 26, TextView ini digunakan untuk menampilkan judul item. wrap\_content, Lebar dan tinggi menyesuaikan panjang teks. textStyle="bold", Menjadikan teks tebal (bold). textSize="20sp", Ukuran huruf sebesar 20sp. paddingTop="8dp", jarak atas antara gambar dan teks sebesar 8dp agar tidak terlalu rapat.

Line 28 sampai 31, TextView ini digunakan untuk menampilkan subjudul atau nama penulis/penerbit item. wrap\_content: Ukuran elemen akan menyesuaikan isi teks. Tidak ada atribut gaya tambahan karena ini hanya teks penjelas.

### **MainActivity.kt :**

Line 2, mengimpor kelas Bundle, yang digunakan untuk menyimpan data sementara, terutama saat menyimpan atau memulihkan status aktivitas (Activity).

Line 3, mengimpor kelas AppCompatActivity, yaitu kelas dasar (superclass) untuk semua aktivitas yang mendukung fitur-fitur modern Android melalui AppCompatActivity library, seperti tema Material Design dan fragment.

Line 4, mengimpor kelas `ActivityMainBinding`, yaitu kelas otomatis yang dihasilkan oleh fitur `ViewBinding` berdasarkan file `activity_main.xml`. Ini memungkinkan kita mengakses langsung komponen UI dalam layout tanpa perlu `findViewById()`.

Line 7, mendefinisikan kelas `MainActivity` yang **mewarisi** (`extends`) `AppCompatActivity`. `MainActivity` berfungsi sebagai aktivitas utama (main screen) aplikasi.

Line 8, mendeklarasikan variabel binding dengan tipe `ActivityMainBinding`. `lateinit` artinya variabel akan diinisialisasi nanti, bukan saat deklarasi. Variabel ini akan digunakan untuk mengakses elemen UI yang didefinisikan dalam `activity_main.xml`.

Line 10, `onCreate()` adalah fungsi siklus hidup aktivitas yang pertama kali dipanggil saat aktivitas dibuat. Parameter `savedInstanceState` menyimpan status sebelumnya jika aktivitas pernah ditutup (misalnya karena rotasi layar).

Line 11, memanggil implementasi `onCreate()` dari superclass (`AppCompatActivity`) untuk memastikan bahwa semua proses internal Android dilakukan dengan benar sebelum melanjutkan.

Line 12, menginisialisasi binding menggunakan `inflate(layoutInflater)`, yang berarti sistem akan membuat view dari file `activity_main.xml`. Dengan ini, kita bisa mengakses elemen-elemen layout menggunakan nama ID-nya dalam kode Kotlin (contoh: `binding.textViewTitle`).

Line 13, Menetapkan tampilan utama (view) dari aktivitas menggunakan root view dari binding. `binding.root` adalah root layout dari `activity_main.xml`, yang akan ditampilkan di layar.

### **activity\_main.xml :**

Line 2, merupakan *view container* modern yang digunakan untuk menampilkan fragment. Ini adalah pengganti dari `<fragment>` di versi Android yang lebih baru. Berasal dari library `androidx.fragment.app`, dan memberikan kontrol yang lebih baik dalam menampilkan dan mengganti fragment secara dinamis.

Line 3 dan 4, `xmlns:android` dan `xmlns:app` adalah deklarasi namespace XML: `xmlns:android` digunakan untuk atribut standar Android (misalnya: `layout_width`, `id`). `xmlns:app` digunakan untuk atribut khusus AndroidX atau Jetpack (misalnya: `app:navGraph`).

Line 5, memberi ID `nav_host_fragment` pada `FragmentContainerView`. ID ini penting karena akan digunakan oleh kode Kotlin/Java atau XML lain untuk merujuk ke view ini, khususnya oleh `NavController`.

Line 6, menentukan bahwa container ini akan menampung **NavHostFragment**, yaitu fragment khusus dari Jetpack Navigation yang bertugas menjadi **penyedia navigasi**. NavHostFragment akan menjadi tempat semua fragment lain (seperti HomeFragment, DetailFragment) ditampilkan sesuai navigasi.

Line 7 dan 8, `match_parent` artinya lebar dan tinggi container akan menyesuaikan ukuran parent-nya, yaitu Activity.

Line 9, menentukan file navigasi graph (`nav_graph.xml`) yang digunakan oleh NavHostFragment. File ini berisi daftar fragment, arah navigasi antar fragment, dan data/argumen yang diperlukan saat berpindah.

Line 10, menandakan bahwa NavHostFragment ini adalah host navigasi utama dalam aplikasi. artinya, ketika pengguna menekan tombol back, sistem akan meminta navigasi kembali ke fragment sebelumnya di stack navigasi ini. hanya boleh ada satu `defaultNavHost="true"` di setiap activity.

#### **D. Tautan Git**

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul3>

## SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

### A. Pembahasan

RecyclerView masih banyak digunakan karena merupakan bagian dari View System yang telah lama ada dan stabil, serta kompatibel dengan berbagai library dan arsitektur Android lama. Meskipun penulisannya cenderung panjang dan boiler-plate, RecyclerView sangat fleksibel dan mendukung berbagai jenis layout serta interaksi kompleks seperti drag-and-drop dan item animasi khusus. Selain itu, banyak developer dan perusahaan besar sudah terbiasa serta memiliki infrastruktur berbasis RecyclerView, sehingga migrasi ke Compose memerlukan pertimbangan matang. LazyColumn dari Jetpack Compose memang menawarkan kode yang lebih ringkas, namun belum sepenuhnya menggantikan fleksibilitas dan kestabilan yang ditawarkan RecyclerView dalam kasus-kasus tertentu.

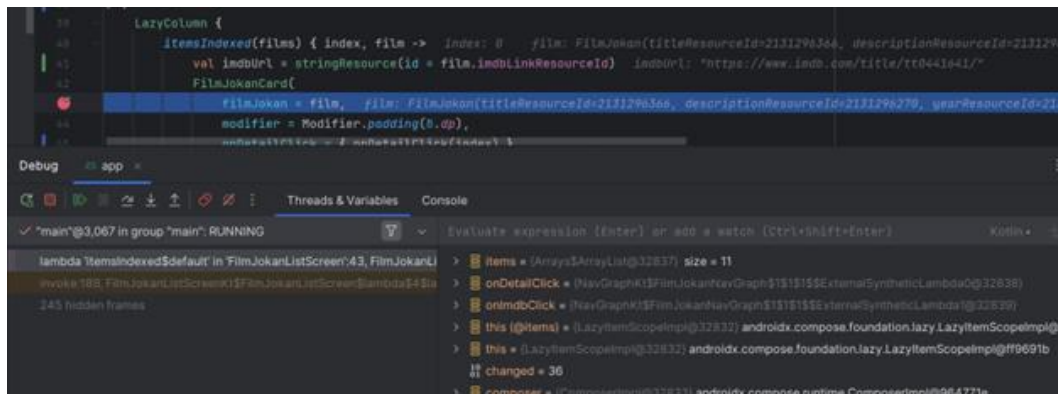
## MODUL 4 : Viewmodel and Debugging

### SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
  - a. Log saat data item masuk ke dalam list
  - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
  - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 24. Contoh Penggunaan Debugger

## A. Source Code

### Jetpack Compose :

#### MainActivity.kt :

Tabel 22. Source Code Jawaban Soal 1 Jetpack Compose

```
1 package com.example.listcompose
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.navigation.NavType
7 import androidx.navigation.compose.NavHost
8 import androidx.navigation.compose.composable
9 import androidx.navigation.compose.rememberNavController
10 import androidx.navigation.navArgument
11 import com.example.listcompose.ui.theme.ListComposeTheme
12 import timber.log.Timber
13
14 class MainActivity : ComponentActivity() {
15     override fun onCreate(savedInstanceState: Bundle?) {
16         super.onCreate(savedInstanceState)
17         Timber.plant(Timber.DebugTree())
18         setContent {
19             ListComposeTheme {
20                 val navController =
21 rememberNavController()
22                 NavHost(navController, startDestination
23 = "list") {
24                     composable("list") {
25                         ItemListScreen(navController)
26                     }
27                     composable(
28                         "detail/{itemId}",
29                         arguments =
30 listOf(navArgument("itemId") { type = NavType.IntType })
31                     ) { backStackEntry ->
32                         val itemId =
33 backStackEntry.arguments?.getInt("itemId") ?: 0
34                         val context = this@MainActivity
35                         val items =.getItems(context)
36                         DetailScreen(item =
37 items[itemId])
38                     }
39                 }
40             }
41         }
42     }
43 }
```

|  |                              |
|--|------------------------------|
|  | <pre>         }     } </pre> |
|--|------------------------------|

### Item.kt :

*Tabel 23. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.example.listcompose                          |
| 2  |                                                          |
| 3  | import android.content.Context                           |
| 4  |                                                          |
| 5  | data class Item(                                         |
| 6  | val id: Int,                                             |
| 7  | val title: String,                                       |
| 8  | val author: String,                                      |
| 9  | val info1: String,                                       |
| 10 | val info2: String,                                       |
| 11 | val imageResId: Int,                                     |
| 12 | val url: String,                                         |
| 13 | val description: String                                  |
| 14 | )                                                        |
| 15 |                                                          |
| 16 | fun.getItems(context: Context): List<Item> {             |
| 17 | val titles =                                             |
| 18 | context.resources.getStringArray(R.array.item_titles)    |
| 19 | val authors =                                            |
| 20 | context.resources.getStringArray(R.array.item_authors)   |
| 21 | val infos1 =                                             |
| 22 | context.resources.getStringArray(R.array.item_info1)     |
| 23 | val infos2 =                                             |
| 24 | context.resources.getStringArray(R.array.item_info2)     |
| 25 | val urls =                                               |
| 26 | context.resources.getStringArray(R.array.item_urls)      |
| 27 | val descriptions =                                       |
| 28 | context.resources.getStringArray(R.array.item_descriptio |
| 29 | ns)                                                      |
| 30 |                                                          |
| 31 | val imageIds = listOf(                                   |
| 32 | R.drawable.lookism,                                      |
| 33 | R.drawable.bones,                                        |
| 34 | R.drawable.wee,                                          |
| 35 | R.drawable.herohasreturned,                              |
| 36 | R.drawable.studygroup                                    |
| 37 | )                                                        |
| 38 |                                                          |
| 39 | return titles.indices.map { index ->                     |



|    |                                   |
|----|-----------------------------------|
| 40 | Item(                             |
| 41 | id = index,                       |
| 42 | title = titles[index],            |
| 43 | author = authors[index],          |
| 44 | info1 = infos1[index],            |
|    | info2 = infos2[index],            |
|    | imageResId = imageIds[index],     |
|    | url = urls[index],                |
|    | description = descriptions[index] |
|    | )                                 |
|    | }                                 |
|    | }                                 |

### ItemListScreen.kt :

*Tabel 24. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | package com.example.listcompose                         |
| 2  |                                                         |
| 3  | import android.content.Intent                           |
| 4  | import androidx.compose.foundation.Image                |
| 5  | import androidx.compose.foundation.layout.Arrangement   |
| 6  | import androidx.compose.foundation.layout.Column        |
| 7  | import androidx.compose.foundation.layout.PaddingValues |
| 8  | import androidx.compose.foundation.layout.Row           |
| 9  | import androidx.compose.foundation.layout.Spacer        |
| 10 | import androidx.compose.foundation.layout.fillMaxWidth  |
| 11 | import androidx.compose.foundation.layout.height        |
| 12 | import androidx.compose.foundation.layout.padding       |
| 13 | import androidx.compose.foundation.lazy.LazyColumn      |
| 14 | import                                                  |
| 15 | androidx.compose.foundation.shape.RoundedCornerShape    |
| 16 | import androidx.compose.material3.Button                |
| 17 | import androidx.compose.material3.Card                  |
| 18 | import androidx.compose.material3.CardDefaults          |
| 19 | import androidx.compose.material3.MaterialTheme         |
| 20 | import androidx.compose.material3.Text                  |
| 21 | import androidx.compose.runtime.Composable              |
| 22 | import androidx.compose.runtime.collectAsState          |
| 23 | import androidx.compose.runtime.getValue                |
| 24 | import androidx.compose.ui.Modifier                     |
| 25 | import androidx.compose.ui.draw.clip                    |
| 26 | import androidx.compose.ui.graphics.Color               |
| 27 | import androidx.compose.ui.layout.ContentScale          |
| 28 | import androidx.compose.ui.platform.LocalContext        |
| 29 | import androidx.compose.ui.res.painterResource          |
| 30 | import androidx.compose.ui.res.stringResource           |
| 31 | import androidx.compose.ui.unit.dp                      |

```

32 import androidx.core.net.toUri
33 import androidx.navigation.NavController
34 import timber.log.Timber
35
36 @Composable
37 fun ItemListScreen(navController: NavController,
38 viewModel: ItemViewModel =
39 androidx.lifecycle.viewmodel.compose.viewModel(factory =
40 ItemViewModelFactory(LocalContext.current, "List
41 Loaded"))) {
42     val context = LocalContext.current
43     val itemList by viewModel.itemList.collectAsState()
44
45     LazyColumn(contentPadding = PaddingValues(8.dp)) {
46         items(itemList.size) { index ->
47             val item = itemList[index]
48             Card(
49                 shape = RoundedCornerShape(16.dp),
50                 modifier = Modifier
51                     .padding(8.dp)
52                     .fillMaxWidth(),
53                 colors =
54 CardDefaults.cardColors(containerColor =
55 Color.DarkGray),
56                 elevation =
57 CardDefaults.cardElevation(defaultElevation = 4.dp)
58             ) {
59                 Column(modifier =
60 Modifier.padding(16.dp)) {
61                     Image(
62                         painter = painterResource(id =
63 item.imageResId),
64                         contentDescription = item.title,
65                         modifier = Modifier
66                             .fillMaxWidth()
67                             .height(180.dp)
68
69                     .clip(RoundedCornerShape(16.dp)),
70                         contentScale = ContentScale.Crop
71                     )
72                     Spacer(modifier =
73 Modifier.height(8.dp))
74                     Text(text = item.title, style =
75 MaterialTheme.typography.titleLarge, color =
76 Color.White)
77                     Text(text = item.author, style =
78 MaterialTheme.typography.bodyMedium, color =
79 Color.White)
80                     Text(text = item.info1, style =

```

```
81 MaterialTheme.typography.bodySmall, color = Color.White)
82         Text(text = item.info2, style =
83 MaterialTheme.typography.bodySmall, color = Color.White)
84         Spacer(modifier =
85 Modifier.height(8.dp))
86         Row(horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
            Button(onClick = {
                val intent =
Intent(Intent.ACTION_VIEW, item.url.toUri())
                Timber.d("Explicit intent to
URL: ${item.url}")

context.startActivity(intent)
            }) {
                Text(text =
stringResource(R.string.manhwa_website))
            }
            Button(onClick = {

navController.navigate("detail/${item.id}")
                Timber.d("Navigated to
Detail for item id: ${item.id}")
            }) {
                Text(text =
stringResource(R.string.manhwa_detail))
            }
        }
    }
}
}
```

### DetailScreen.kt :

Tabel 25. Source Code Jawaban Soal 1 Jetpack Compose

```
1 package com.example.listcompose
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.layout.*
5 import androidx.compose.foundation.rememberScrollState
6 import androidx.compose.foundation.verticalScroll
7 import
8 androidx.compose.foundation.shape.RoundedCornerShape
9 import androidx.compose.material3.*
10 import androidx.compose.runtime.Composable
```

|    |                                                |
|----|------------------------------------------------|
| 11 | import androidx.compose.ui.Modifier            |
| 12 | import androidx.compose.ui.draw.clip           |
| 13 | import androidx.compose.ui.layout.ContentScale |
| 14 | import androidx.compose.ui.res.painterResource |
| 15 | import androidx.compose.ui.unit.dp             |
| 16 |                                                |
| 17 | @Composable                                    |
| 18 | fun DetailScreen(item: Item) {                 |
| 19 | val scrollState = rememberScrollState()        |
| 20 | Column(                                        |
| 21 | modifier = Modifier                            |
| 22 | .padding(16.dp)                                |
| 23 | .fillMaxSize()                                 |
| 24 | .verticalScroll(scrollState)                   |
| 25 | ) {                                            |
| 26 | Image(                                         |
| 27 | painter = painterResource(id =                 |
| 28 | item.imageResId),                              |
| 29 | contentDescription = item.title,               |
| 30 | modifier = Modifier                            |
| 31 | .fillMaxWidth()                                |
| 32 | .height(240.dp)                                |
| 33 | .clip(RoundedCornerShape(16.dp)),              |
| 34 | contentScale = ContentScale.Crop               |
| 35 | )                                              |
| 36 | Spacer(modifier = Modifier.height(16.dp))      |
| 37 | Text(text = item.title, style =                |
| 38 | MaterialTheme.typography.headlineMedium)       |
| 39 | Text(text = item.author, style =               |
| 40 | MaterialTheme.typography.bodyLarge)            |
| 41 | Text(text = item.info1, style =                |
|    | MaterialTheme.typography.bodyMedium)           |
|    | Text(text = item.info2, style =                |
|    | MaterialTheme.typography.bodyMedium)           |
|    | Text(text = item.description, style =          |
|    | MaterialTheme.typography.bodyMedium)           |
|    | }                                              |
|    | }                                              |

### ItemViewModel.kt :

*Tabel 26. Source Code Jawaban Soal 1 Jetpack Compose*

|   |                                          |
|---|------------------------------------------|
| 1 | package com.example.listcompose          |
| 2 |                                          |
| 3 | import android.content.Context           |
| 4 | import androidx.lifecycle.ViewModel      |
| 5 | import androidx.lifecycle.viewModelScope |

```

6 import kotlinx.coroutines.flow.MutableStateFlow
7 import kotlinx.coroutines.flow.StateFlow
8 import kotlinx.coroutines.launch
9 import timber.log.Timber
10
11 class ItemViewModel(private val context: Context,
12 private val param: String) : ViewModel() {
13
14     private val _itemList =
15 MutableStateFlow<List<Item>>>(emptyList())
16     val itemList: StateFlow<List<Item>> = _itemList
17
18     init {
19         viewModelScope.launch {
20             val items =.getItems(context)
21             _itemList.value = items
22             Timber.d("ItemViewModel: $param - Loaded
23 ${items.size} items")
24         }
25     }
26 }

```

### ItemViewModelFactory.kt :

*Tabel 27. Source Code Jawaban Soal 1 Jetpack Compose*

```

1 package com.example.listcompose
2
3 import android.content.Context
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6
7 class ItemViewModelFactory(private val context: Context,
8 private val param: String) : ViewModelProvider.Factory {
9     override fun <T : ViewModel> create(modelClass:
10 Class<T>): T {
11         return ItemViewModel(context, param) as T
12     }
13 }

```

**XML :**

**Item.kt :**

*Tabel 28. Source Code Jawaban Soal 1 XML*

|   |                             |
|---|-----------------------------|
| 1 | package com.example.listxml |
| 2 |                             |
| 3 | data class Item(            |
| 4 | val imageResId: Int,        |
| 5 | val title: String,          |
| 6 | val author: String,         |
| 7 | val url: String)            |

**ItemAdapter.kt :**

*Tabel 29. Source Code Jawaban Soal 1 XML*

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.example.listxml                              |
| 2  |                                                          |
| 3  | import android.annotation.SuppressLint                   |
| 4  | import android.view.LayoutInflater                       |
| 5  | import android.view.ViewGroup                            |
| 6  | import androidx.recyclerview.widget.RecyclerView         |
| 7  | import com.example.listxml.databinding.ItemListBinding   |
| 8  |                                                          |
| 9  | class ItemAdapter(                                       |
| 10 | private var items: List<Item>,                           |
| 11 | private val onOpenClick: (String) -> Unit,               |
| 12 | private val onDetailClick: (Item) -> Unit                |
| 13 | ) : RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() { |
| 14 |                                                          |
| 15 | inner class ItemViewHolder(val binding:                  |
| 16 | ItemListBinding) : RecyclerView.ViewHolder(binding.root) |
| 17 |                                                          |
| 18 | override fun onCreateViewHolder(parent: ViewGroup,       |
| 19 | viewType: Int): ItemViewHolder {                         |
| 20 | val binding =                                            |
| 21 | ItemListBinding.inflate(LayoutInflater.from(parent.conte |
| 22 | xt), parent, false)                                      |
| 23 | return ItemViewHolder(binding)                           |
| 24 | }                                                        |
| 25 |                                                          |
| 26 | override fun onBindViewHolder(holder:                    |
| 27 | ItemViewHolder, position: Int) {                         |
| 28 | val item = items[position]                               |
| 29 | with(holder.binding) {                                   |
| 30 | itemImage.setImageResource(item.imageResId)              |
| 31 | titleText.text = item.title                              |
| 32 | subtitleText.text = item.author                          |

|    |                                               |
|----|-----------------------------------------------|
| 33 |                                               |
| 34 | openButton.setOnClickListener {               |
| 35 | onOpenClick(item.url)                         |
| 36 | }                                             |
| 37 | detailButton.setOnClickListener {             |
| 38 | onDetailClick(item)                           |
| 39 | }                                             |
| 40 | }                                             |
| 41 | }                                             |
| 42 |                                               |
| 43 | override fun getItemCount(): Int = items.size |
| 44 | @SuppressLint("NotifyDataSetChanged")         |
|    | fun updateItems(newItems: List<Item>) {       |
|    | items = newItems                              |
|    | notifyDataSetChanged()                        |
|    | }                                             |
|    | }                                             |

#### item\_list.xml :

Tabel 30. Source Code Jawaban Soal 1 XML

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <androidx.cardview.widget.CardView                         |
| 3  |                                                            |
| 4  | xmlns:android="http://schemas.android.com/apk/res/android" |
| 5  | xmlns:card_view="http://schemas.android.com/apk/res-       |
| 6  | auto"                                                      |
| 7  | xmlns:app="http://schemas.android.com/apk/res-auto"        |
| 8  | android:layout_width="match_parent"                        |
| 9  | android:layout_height="wrap_content"                       |
| 10 | android:layout_marginBottom="12dp"                         |
| 11 | card_view:cardCornerRadius="16dp"                          |
| 12 | card_view:cardElevation="4dp">                             |
| 13 |                                                            |
| 14 | <LinearLayout                                              |
| 15 | android:layout_width="match_parent"                        |
| 16 | android:layout_height="wrap_content"                       |
| 17 | android:orientation="vertical"                             |
| 18 | android:padding="16dp">                                    |
| 19 |                                                            |
| 20 |                                                            |
| 21 | <com.google.android.material.imageview.ShapeableImageView  |
| 22 | android:id="@+id/itemImage"                                |
| 23 | android:layout_width="match_parent"                        |
| 24 | android:layout_height="180dp"                              |
| 25 | android:scaleType="centerCrop"                             |
| 26 |                                                            |

```

27 app:shapeAppearanceOverlay="@style/RoundedImageView"/>
28
29     <TextView
30         android:id="@+id/titleText"
31         android:layout_width="wrap_content"
32         android:layout_height="wrap_content"
33         android:text="Judul"
34         android:textStyle="bold"
35         android:textSize="18sp"
36         android:paddingTop="8dp"/>
37
38     <TextView
39         android:id="@+id/subtitleText"
40         android:layout_width="wrap_content"
41         android:layout_height="wrap_content"
42         android:text="Subjudul"                                />
43
44     <LinearLayout
45         android:layout_width="match_parent"
46         android:layout_height="wrap_content"
47         android:orientation="horizontal"
48         android:gravity="end">
49
50         <Button
51             android:id="@+id/openButton"
52             android:layout_width="wrap_content"
53             android:layout_height="wrap_content"
54             android:text="Open"                                />
55
56         <Button
57             android:id="@+id/detailButton"
58             android:layout_width="wrap_content"
59             android:layout_height="wrap_content"
60             android:text="Detail"
61             android:layout_marginStart="8dp"/>
62     </LinearLayout>
63 </LinearLayout>
64 </androidx.cardview.widget.CardView>

```

## HomeFragment.kt :

*Tabel 31. Source Code Jawaban Soal 1 XML*

```
1 package com.example.listxml
2
3 import android.content.Intent
4 import android.os.Bundle
5 import android.view.LayoutInflater
```



```

6 import android.view.View
7 import android.view.ViewGroup
8 import androidx.core.net.toUri
9 import androidx.fragment.app.Fragment
10 import androidx.fragment.app.viewModels
11 import androidx.lifecycle.lifecycleScope
12 import androidx.navigation.fragment.findNavController
13 import androidx.recyclerview.widget.LinearLayoutManager
14 import
15 com.example.listxml.databinding.FragmentHomeBinding
16 import kotlinx.coroutines.launch
17 import timber.log.Timber
18
19 class HomeFragment : Fragment() {
20     private var _binding: FragmentHomeBinding? = null
21     private val binding get() = _binding!!
22
23     private lateinit var adapter: ItemAdapter
24
25     private val viewModel: ItemViewModel by viewModels {
26         ItemViewModelFactory("ParamDariHome")
27     }
28
29     override fun onCreateView(
30         inflater: LayoutInflater, container: ViewGroup?,
31         savedInstanceState: Bundle?
32     ): View {
33         _binding = FragmentHomeBinding.inflate(inflater,
34 container, false)
35         return binding.root
36     }
37
38     override fun onViewCreated(view: View,
39 savedInstanceState: Bundle?) {
40         adapter = ItemAdapter(
41             emptyList(),
42             onOpenClick = { url ->
43                 Timber.d("HomeFragment: Open URL $url")
44                 val intent = Intent(Intent.ACTION_VIEW,
45 url.toUri())
46                 startActivity(intent)
47             },
48             onDetailClick = { item ->
49                 viewModel.selectItem(item)
50                 Timber.d("HomeFragment: Navigate to
51 detail ${item.title}")
52                 val action =
53 HomeFragmentDirections.actionHomeToDetail(
54                     item.title,

```

|    |                                            |
|----|--------------------------------------------|
| 55 | item.author,                               |
| 56 | item.imageResId                            |
| 57 | )                                          |
| 58 | findNavController().navigate(action)       |
| 59 | }                                          |
| 60 | )                                          |
| 61 |                                            |
| 62 | binding.recyclerView.layoutManager =       |
| 63 | LinearLayoutManager(requireContext())      |
| 64 | binding.recyclerView.adapter = adapter     |
| 65 |                                            |
| 66 | viewLifecycleOwner.lifecycleScope.launch { |
| 67 | viewModel.itemList.collect { items ->      |
| 68 | adapter.updateItems(items)                 |
| 69 | Timber.d("HomeFragment: Received           |
| 70 | \${items.size} items")                     |
| 71 | }                                          |
|    | }                                          |
|    |                                            |
|    | override fun onDestroyView() {             |
|    | super.onDestroyView()                      |
|    | _binding = null                            |
|    | }                                          |
|    | }                                          |

### fragment\_home.xml :

Tabel 32. Source Code Jawaban Soal 1 XML

|   |                                                           |
|---|-----------------------------------------------------------|
| 1 | <?xml version="1.0" encoding="utf-8"?>                    |
| 2 | <androidx.constraintlayout.widget.ConstraintLayout        |
| 3 |                                                           |
| 4 | xmlns:android="http://schemas.android.com/apk/res/android |
| 5 | "                                                         |
| 6 | xmlns:app="http://schemas.android.com/apk/res-auto"       |
| 7 | android:layout_width="match_parent"                       |
| 8 | android:layout_height="match_parent">                     |
| 9 |                                                           |
| 1 | <androidx.recyclerview.widget.RecyclerView                |
| 0 | android:id="@+id/recyclerView"                            |
| 1 | android:layout_width="0dp"                                |
| 1 | android:layout_height="0dp"                               |
| 1 | android:clipToPadding="false"                             |
| 2 | android:padding="16dp"                                    |
| 1 | app:layoutManager=                                        |
| 3 | "androidx.recyclerview.widget.LinearLayoutManager"        |
|   | app:layout_constraintTop_toTopOf="parent"                 |

|   |                                                      |
|---|------------------------------------------------------|
| 1 | app:layout_constraintBottom_toBottomOf="parent"      |
| 4 | app:layout_constraintStart_toStartOf="parent"        |
| 1 | app:layout_constraintEnd_toEndOf="parent"/>          |
| 5 | </androidx.constraintlayout.widget.ConstraintLayout> |
| 1 |                                                      |
| 6 |                                                      |
| 1 |                                                      |
| 7 |                                                      |
| 1 |                                                      |
| 8 |                                                      |
| 1 |                                                      |
| 9 |                                                      |

### nav\_graph :

*Tabel 33. Source Code Jawaban Soal 1 XML*

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <navigation                                                |
| 3  | xmlns:android="http://schemas.android.com/apk/res/android" |
| 4  | xmlns:app="http://schemas.android.com/apk/res-auto"        |
| 5  | xmlns:tools="http://schemas.android.com/tools"             |
| 6  | android:id="@+id/nav_graph"                                |
| 7  | app:startDestination="@id/homeFragment">                   |
| 8  |                                                            |
| 9  | <fragment                                                  |
| 10 | android:id="@+id/homeFragment"                             |
| 11 | android:name="com.example.listxml.HomeFragment"            |
| 12 | android:label="Home"                                       |
| 13 | tools:layout="@layout/fragment_home">                      |
| 14 | <action                                                    |
| 15 | android:id="@+id/action_home_to_detail"                    |
| 16 | app:destination="@id/detailFragment" />                    |
| 17 | </fragment>                                                |
| 18 |                                                            |
| 19 | <fragment                                                  |
| 20 | android:id="@+id/detailFragment"                           |
| 21 | android:name="com.example.listxml.DetailFragment"          |
| 22 | android:label="Detail"                                     |
| 23 | tools:layout="@layout/fragment_detail">                    |
| 24 | <argument                                                  |
| 25 | android:name="title"                                       |
| 26 | app:argType="string" />                                    |
| 27 | <argument                                                  |
| 28 | android:name="subtitle"                                    |
| 29 | app:argType="string" />                                    |
| 30 | <argument                                                  |
| 31 | android:name="imageResId"                                  |
| 32 | app:argType="integer" />                                   |

|    |                              |
|----|------------------------------|
| 33 | </fragment><br></navigation> |
|----|------------------------------|

### DetailFragment.kt :

*Tabel 34. Source Code Jawaban Soal 1 XML*

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.example.listxml                              |
| 2  |                                                          |
| 3  | import android.os.Bundle                                 |
| 4  | import android.view.LayoutInflater                       |
| 5  | import android.view.View                                 |
| 6  | import android.view.ViewGroup                            |
| 7  | import androidx.fragment.app.Fragment                    |
| 8  | import androidx.navigation.fragment.navArgs              |
| 9  | import                                                   |
| 10 | com.example.listxml.databinding.FragmentDetailBinding    |
| 11 | import timber.log.Timber                                 |
| 12 |                                                          |
| 13 | class DetailFragment : Fragment() {                      |
| 14 | private var _binding: FragmentDetailBinding? = null      |
| 15 | private val binding get() = _binding!!                   |
| 16 | private val args: DetailFragmentArgs by navArgs()        |
| 17 |                                                          |
| 18 | override fun onCreateView(                               |
| 19 | inflater: LayoutInflater, container: ViewGroup?,         |
| 20 | savedInstanceState: Bundle?                              |
| 21 | ): View {                                                |
| 22 | _binding = FragmentDetailBinding.inflate(inflater,       |
| 23 | container, false)                                        |
| 24 | return binding.root                                      |
| 25 | }                                                        |
| 26 |                                                          |
| 27 | override fun onViewCreated(view: View,                   |
| 28 | savedInstanceState: Bundle?) {                           |
| 29 | Timber.d("DetailFragment: Received item - Title:         |
| 30 | \${args.title}, Subtitle: \${args.subtitle}, ImageResId: |
| 31 | \${args.imageResId}")                                    |
| 32 |                                                          |
| 33 | binding.detailTitle.text = args.title                    |
| 34 | binding.detailSubtitle.text = args.subtitle              |
| 35 |                                                          |
| 36 | binding.detailImage.setImageResource(args.imageResId)    |
| 37 | }                                                        |
|    |                                                          |
|    | override fun onDestroyView() {                           |
|    | super.onDestroyView()                                    |
|    | _binding = null                                          |

|  |                              |
|--|------------------------------|
|  | <pre>         }     } </pre> |
|--|------------------------------|

### Fragment\_detail.xml:

Tabel 35. Source Code Jawaban Soal 1 XML

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                     |
| 2  | <ScrollView                                                |
| 3  | xmlns:android="http://schemas.android.com/apk/res/android" |
| 4  | xmlns:app="http://schemas.android.com/apk/res-auto"        |
| 5  | android:layout_width="match_parent"                        |
| 6  | android:layout_height="match_parent"                       |
| 7  | android:padding="16dp">                                    |
| 8  |                                                            |
| 9  | <LinearLayout                                              |
| 10 | android:layout_width="match_parent"                        |
| 11 | android:layout_height="wrap_content"                       |
| 12 | android:orientation="vertical">                            |
| 13 |                                                            |
| 14 |                                                            |
| 15 | <com.google.android.material.imageview.ShapeableImageView  |
| 16 | android:id="@+id/detailImage"                              |
| 17 | android:layout_width="match_parent"                        |
| 18 | android:layout_height="200dp"                              |
| 19 | android:scaleType="centerCrop"                             |
| 20 |                                                            |
| 21 | app:shapeAppearanceOverlay="@style/RoundedImageView" />    |
| 22 |                                                            |
| 23 | <TextView                                                  |
| 24 | android:id="@+id/detailTitle"                              |
| 25 | android:layout_width="wrap_content"                        |
| 26 | android:layout_height="wrap_content"                       |
| 27 | android:textStyle="bold"                                   |
| 28 | android:textSize="20sp"                                    |
| 29 | android:paddingTop="8dp" />                                |
| 30 |                                                            |
| 31 | <TextView                                                  |
| 32 | android:id="@+id/detailSubtitle"                           |
| 33 | android:layout_width="wrap_content"                        |
|    | android:layout_height="wrap_content" />                    |
|    | </LinearLayout>                                            |
|    | </ScrollView>                                              |

### MainActivity.kt:

Tabel 36. Source Code Jawaban Soal 1 XML

|    |                                                            |
|----|------------------------------------------------------------|
| 1  | package com.example.listxml                                |
| 2  |                                                            |
| 3  | import android.os.Bundle                                   |
| 4  | import androidx.appcompat.app.AppCompatActivity            |
| 5  | import com.example.listxml.databinding.ActivityMainBinding |
| 6  | import timber.log.Timber                                   |
| 7  |                                                            |
| 8  | class MainActivity : AppCompatActivity() {                 |
| 9  | private lateinit var binding: ActivityMainBinding          |
| 10 |                                                            |
| 11 | override fun onCreate(savedInstanceState: Bundle?) {       |
| 12 | super.onCreate(savedInstanceState)                         |
| 13 | Timber.plant(Timber.DebugTree())                           |
| 14 | binding =                                                  |
| 15 | ActivityMainBinding.inflate(layoutInflater)                |
| 16 | setContentView(binding.root)                               |
| 17 | }                                                          |
|    | }                                                          |

### activity\_main.xml:

Tabel 37. Source Code Jawaban Soal 1 XML

|    |                                                             |
|----|-------------------------------------------------------------|
| 1  | <?xml version="1.0" encoding="utf-8"?>                      |
| 2  | <androidx.fragment.app.FragmentContainerView                |
| 3  |                                                             |
| 4  | xmlns:android="http://schemas.android.com/apk/res/android"  |
| 5  | xmlns:app="http://schemas.android.com/apk/res-auto"         |
| 6  | android:id="@+id/nav_host_fragment"                         |
| 7  |                                                             |
| 8  | android:name="androidx.navigation.fragment.NavHostFragment" |
| 9  | android:layout_width="match_parent"                         |
| 10 | android:layout_height="match_parent"                        |
|    | app:navGraph="@navigation/nav_graph"                        |
|    | app:defaultNavHost="true" />                                |

### ItemViewModel.kt :

Tabel 38. Source Code Jawaban Soal 1 XML

|   |                                     |
|---|-------------------------------------|
| 1 | package com.example.listxml         |
| 2 |                                     |
| 3 | import androidx.lifecycle.ViewModel |

```

4 import androidx.lifecycle.viewModelScope
5 import kotlinx.coroutines.flow.MutableStateFlow
6 import kotlinx.coroutines.flow.StateFlow
7 import kotlinx.coroutines.launch
8 import timber.log.Timber
9
1 class ItemViewModel(private val param: String) :
0 ViewModel() {
1
1     private val _itemList =
1 MutableStateFlow<List<Item>>(emptyList())
2     val itemList: StateFlow<List<Item>> = _itemList
1
3     private val _selectedItem =
1 MutableStateFlow<Item?>(null)
4
1     init {
5         viewModelScope.launch {
1             val items = getDummyItems()
6             _itemList.value = items
1             Timber.d("ItemViewModel: $param - Loaded
7 ${items.size} items")
1         }
8     }
1
9     fun selectItem(item: Item) {
2         _selectedItem.value = item
0         Timber.d("ItemViewModel: Selected item:
2 ${item.title}")
1     }
2
2     private fun getDummyItems(): List<Item> {
2         return listOf(
3             Item(R.drawable.lookism, "Lookism", "Author
2 1", "https://lookism.fandom.com/wiki/Lookism"),
4             Item(R.drawable.bones, "Bones", "Author 2",
2 "https://en.namu.wiki/w/%EB%B3%B8%EC%A6%88(%EC%9B%B9%ED%8
5 8%B0)"),
2             Item(R.drawable.wee, "Wee", "Author 3",
6 "https://webtoon.fandom.com/id/wiki/WEE!!!"),
2             Item(R.drawable.herohasreturned, "Hero Has
7 Returned", "Author 4", "https://hero-has-
2 returned.fandom.com/wiki/Hero_Has_Returned_Wiki"),
8             Item(R.drawable.studygroup, "Study Group",
2 "Author 5", "https://study-
9 group.fandom.com/wiki/Study_Group_Wiki")
3         )
0     }
1 }

```

|   |  |
|---|--|
| 3 |  |
| 1 |  |
| 3 |  |
| 2 |  |
| 3 |  |
| 3 |  |
| 3 |  |
| 4 |  |
| 3 |  |
| 5 |  |
| 3 |  |
| 6 |  |
| 3 |  |
| 7 |  |
| 3 |  |
| 8 |  |
| 3 |  |
| 9 |  |

### **ItemViewModelFactory.kt :**

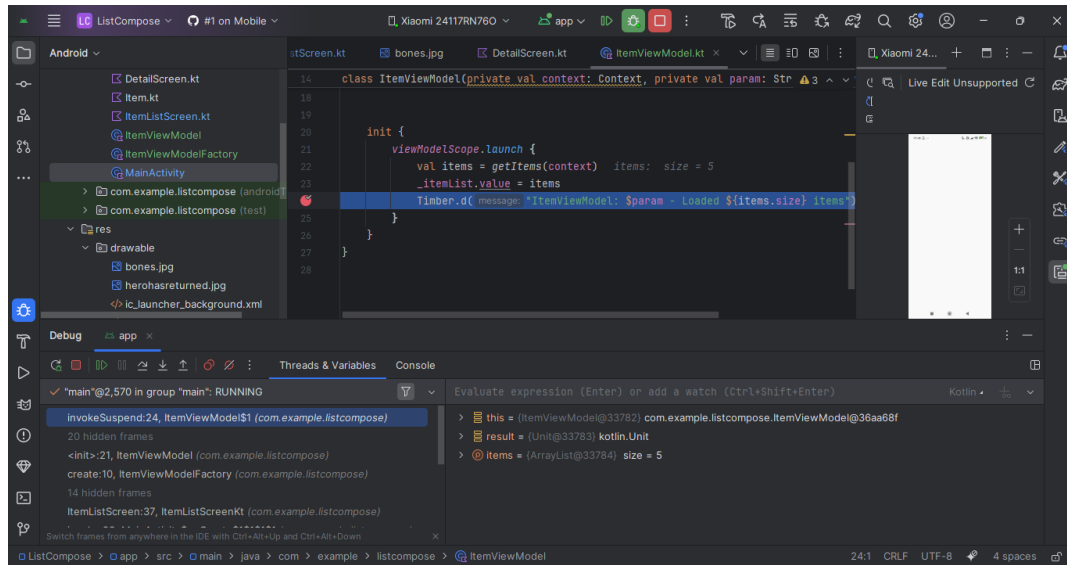
*Tabel 39. Source Code Jawaban Soal 1 XML*

|    |                                                 |
|----|-------------------------------------------------|
| 1  | package com.example.listxml                     |
| 2  |                                                 |
| 3  | import androidx.lifecycle.ViewModel             |
| 4  | import androidx.lifecycle.ViewModelProvider     |
| 5  |                                                 |
| 6  | class ItemViewModelFactory(                     |
| 7  | private val param: String                       |
| 8  | ) : ViewModelProvider.Factory {                 |
| 9  | override fun <T : ViewModel> create(modelClass: |
| 10 | Class<T>): T {                                  |
| 11 | return ItemViewModel(param) as T                |
| 12 | }                                               |
|    | }                                               |

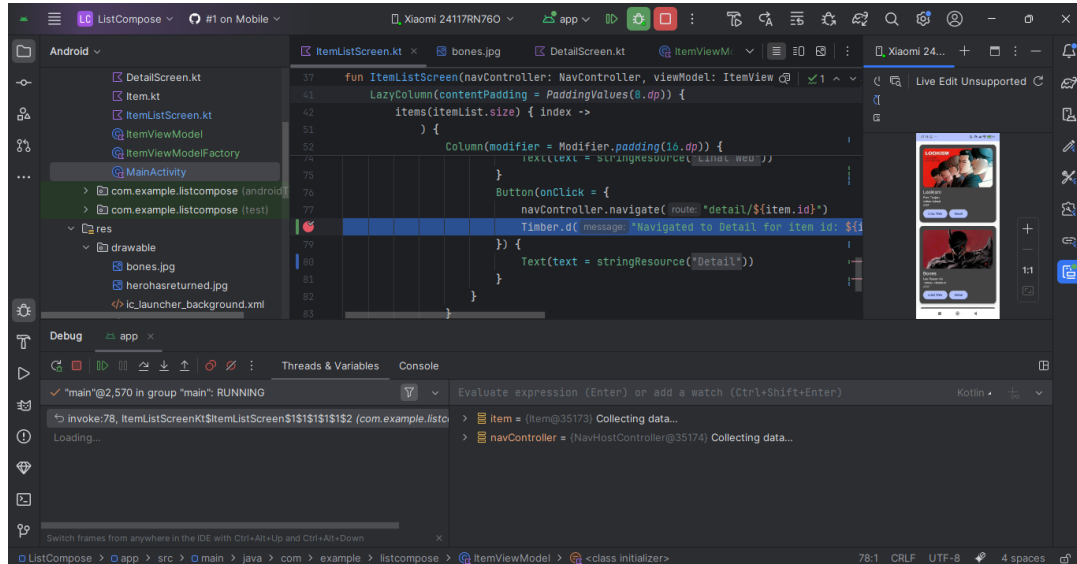


## B. Output Program

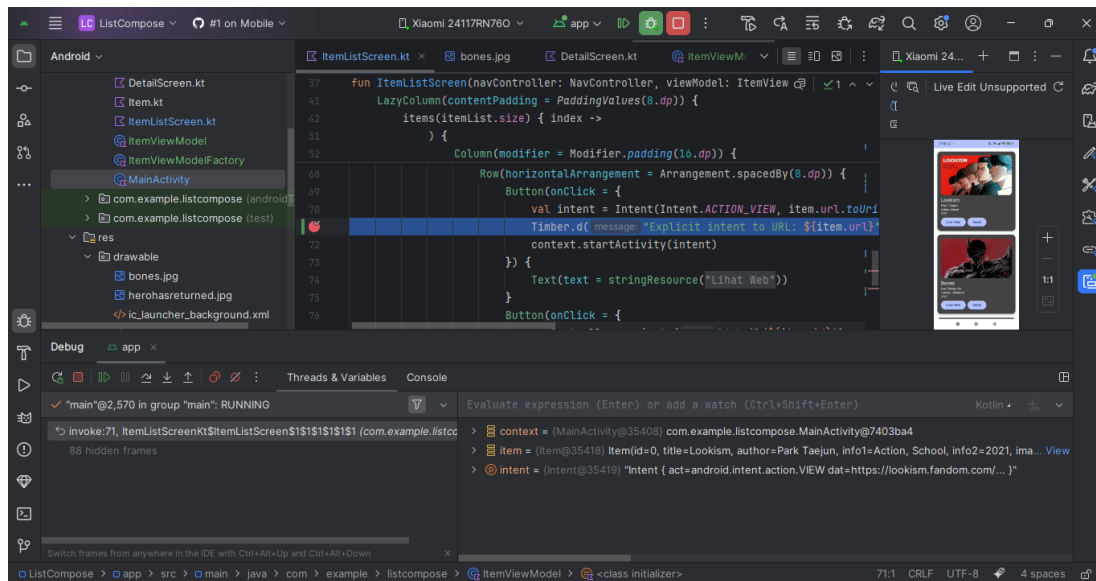
Compose :



Gambar 25. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

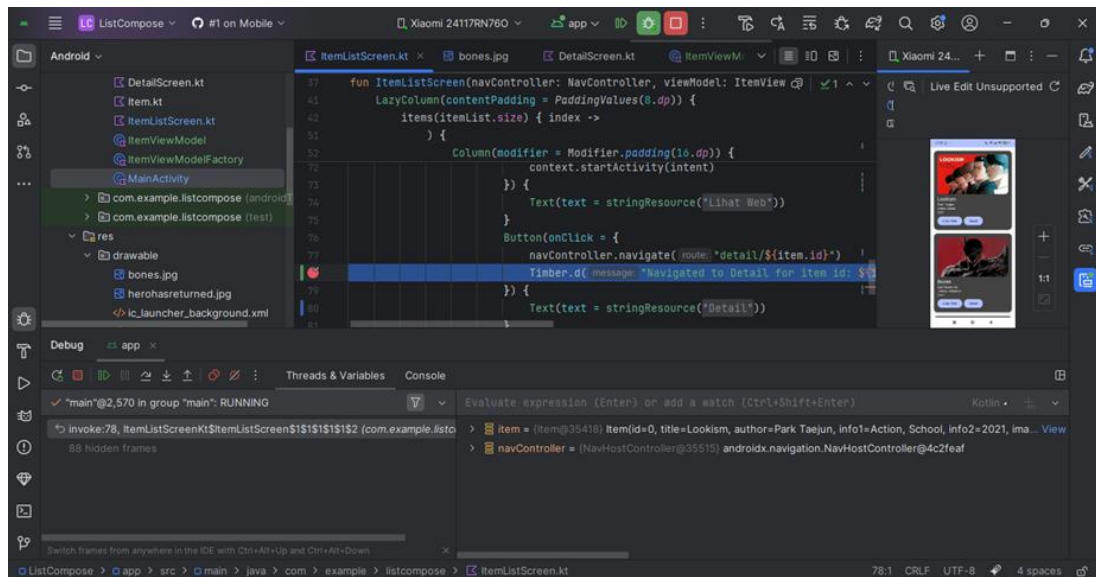


Gambar 26. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

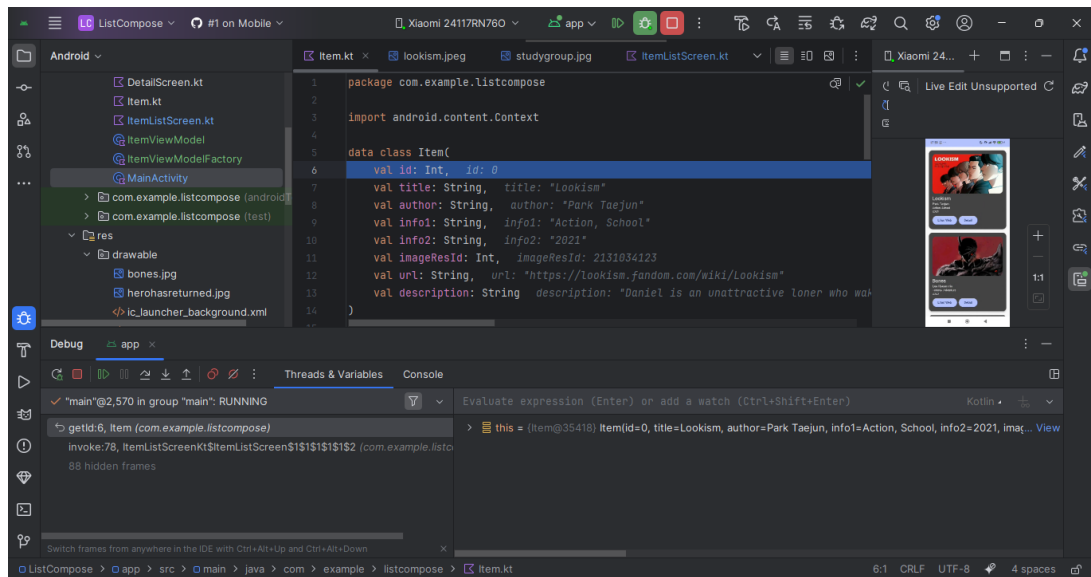


Gambar 27. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Step Into :

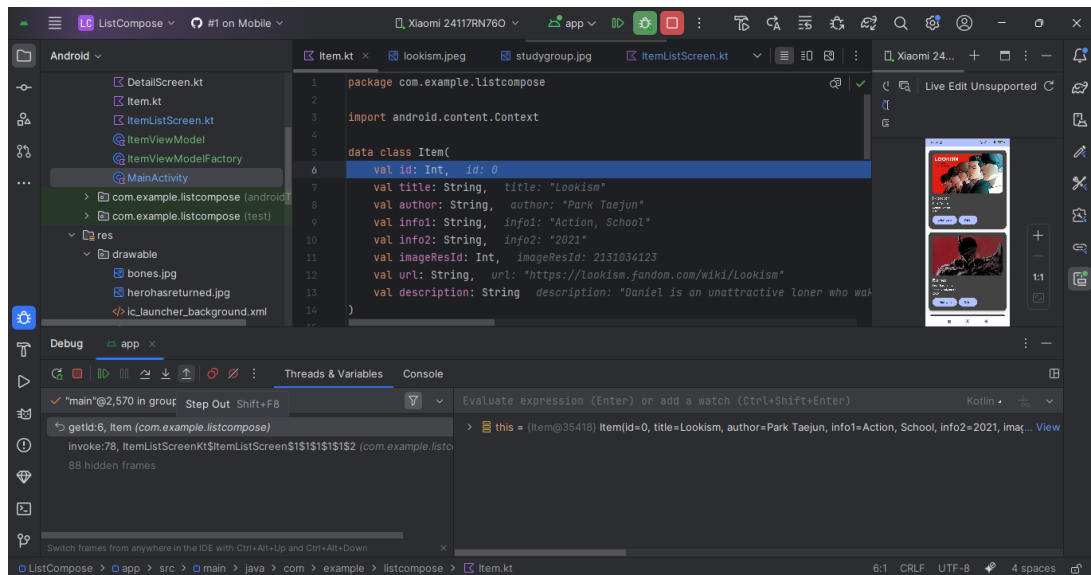


Gambar 28. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

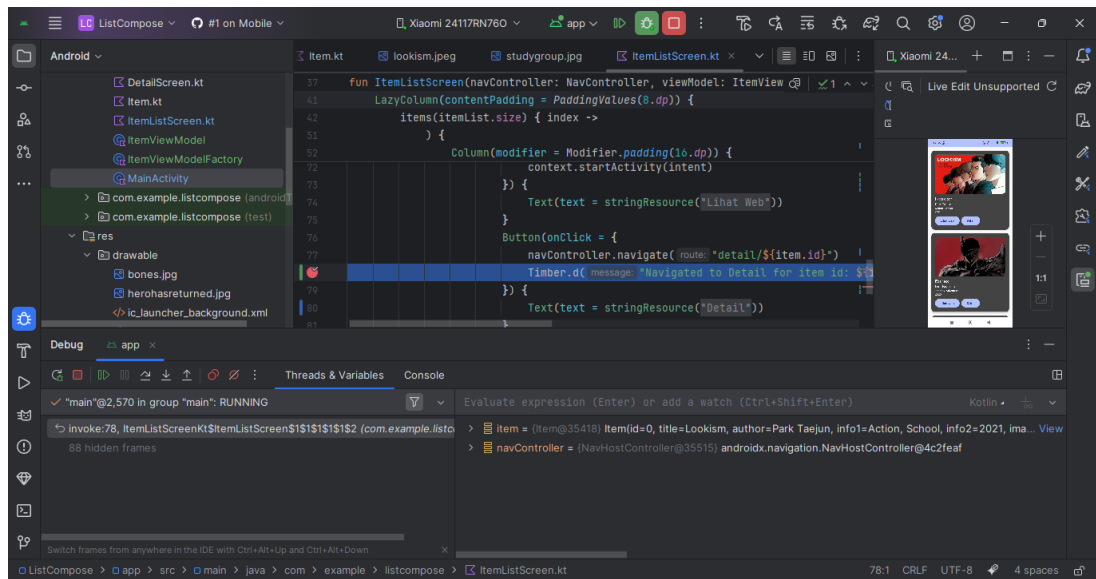


Gambar 29. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Step Out :

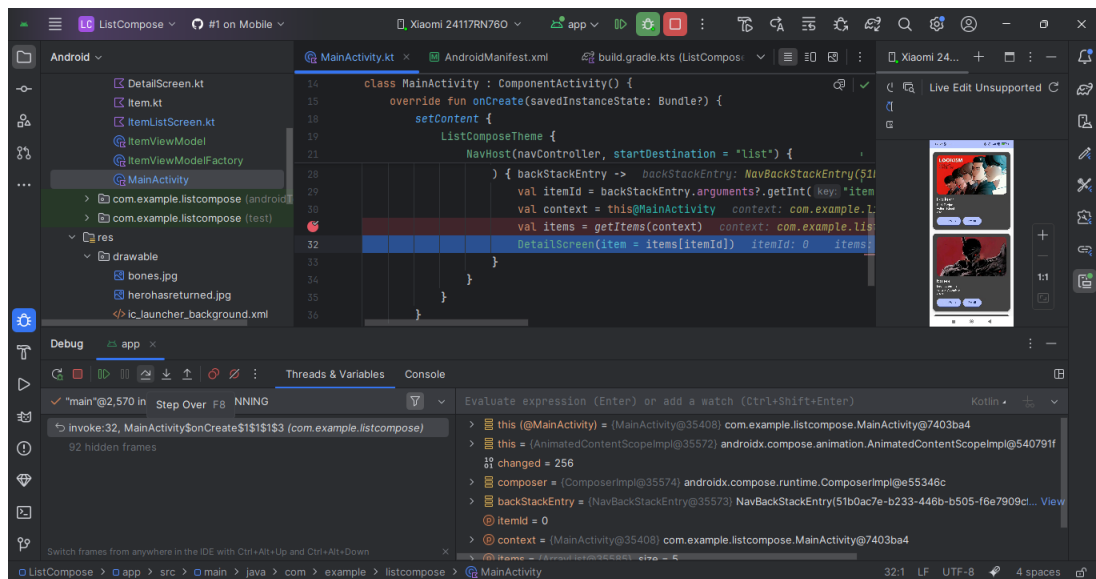


Gambar 30. Screenshot Hasil Jawaban Soal 1 Jetpack Compose



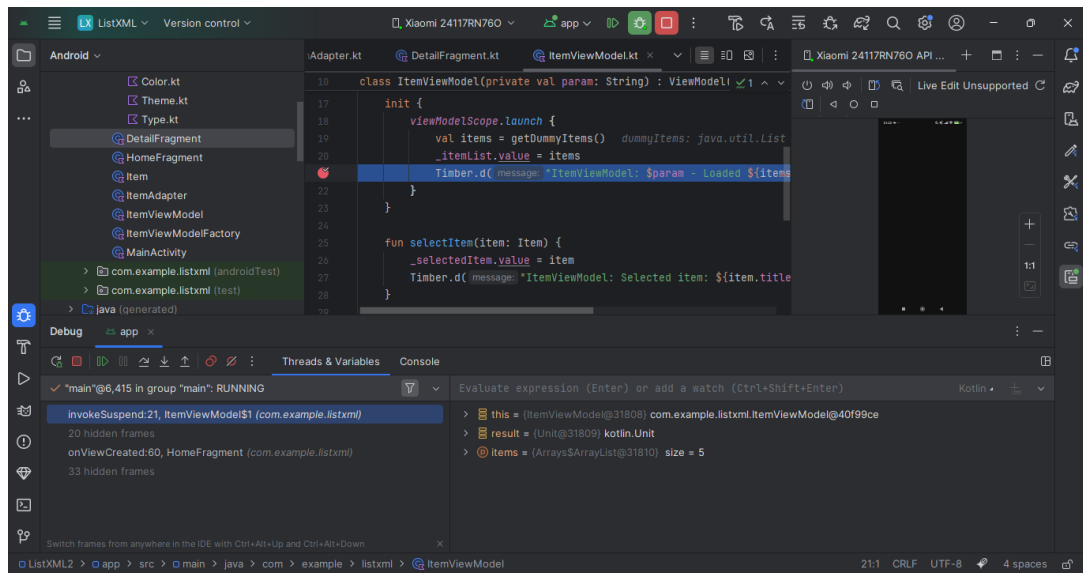
Gambar 31. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

Step Over :

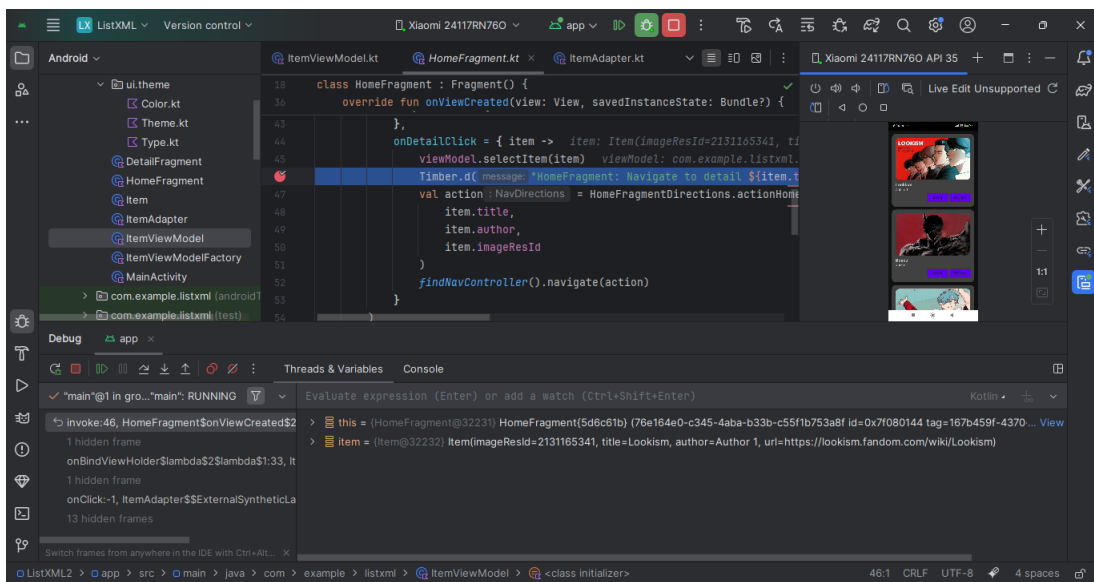


Gambar 32. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

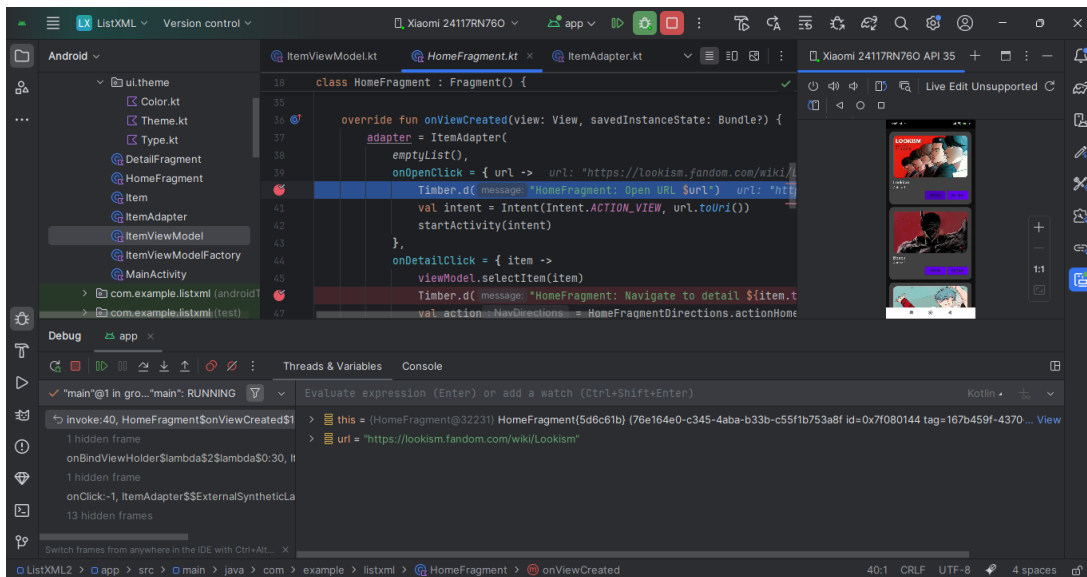
XML :



Gambar 33. Screenshot Hasil Jawaban Soal 1 XML

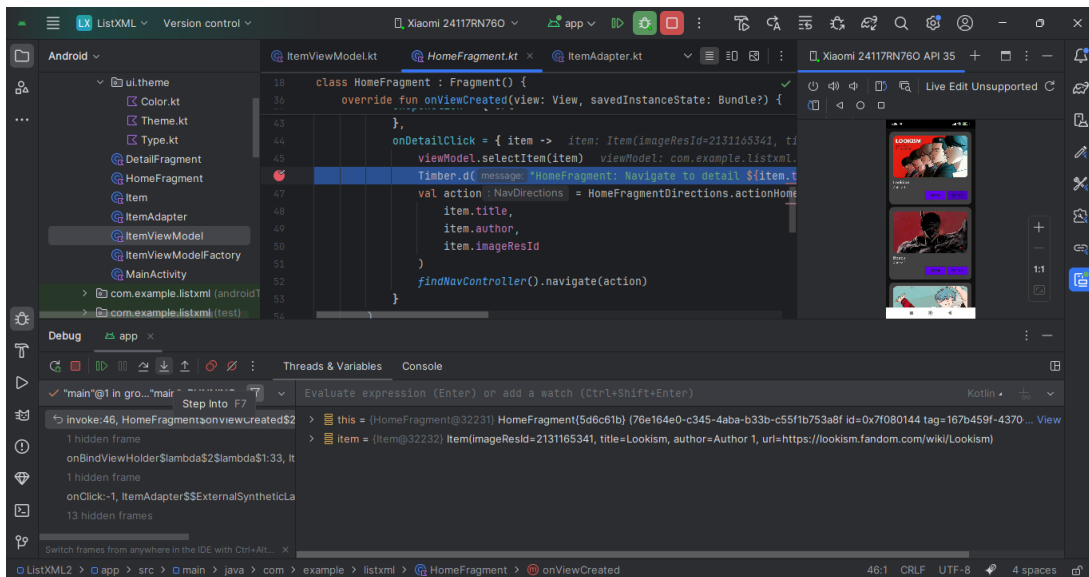


Gambar 34. Screenshot Hasil Jawaban Soal 1 XML

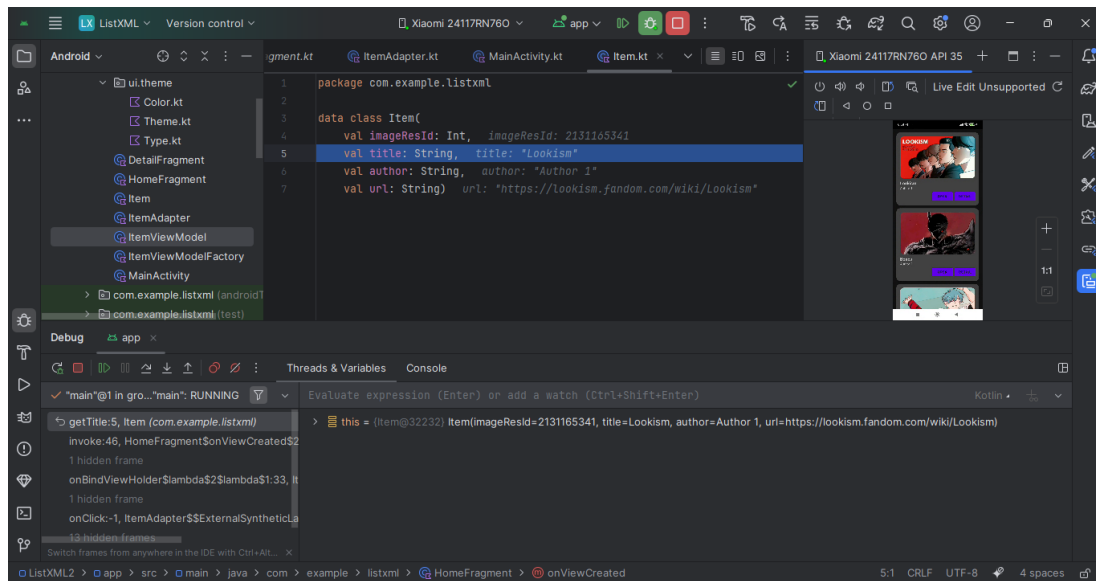


Gambar 35. Screenshot Hasil Jawaban Soal 1 XML

Step Into :

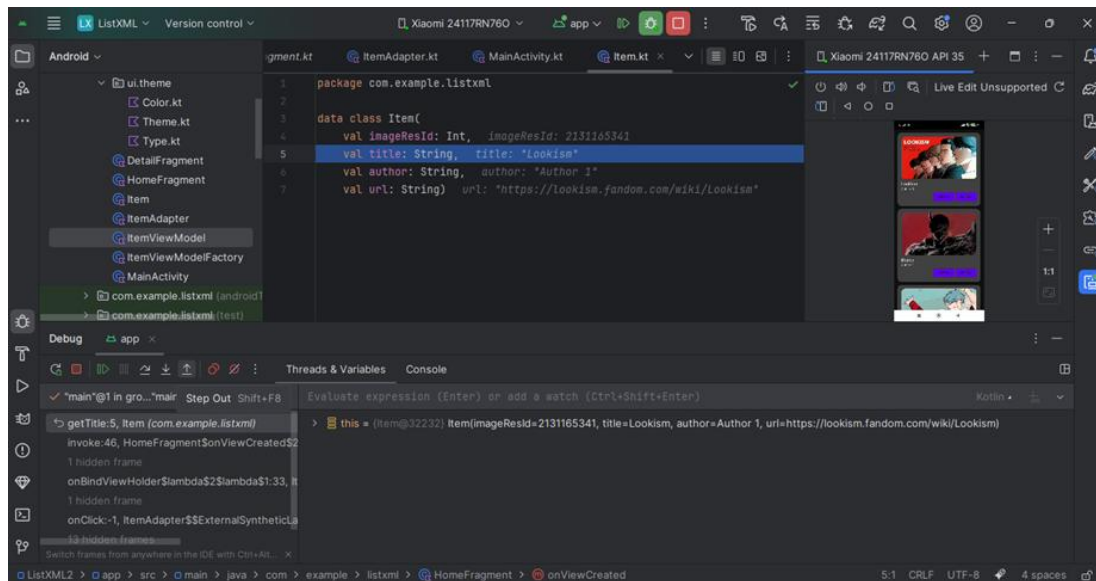


Gambar 36. Screenshot Hasil Jawaban Soal 1 XML



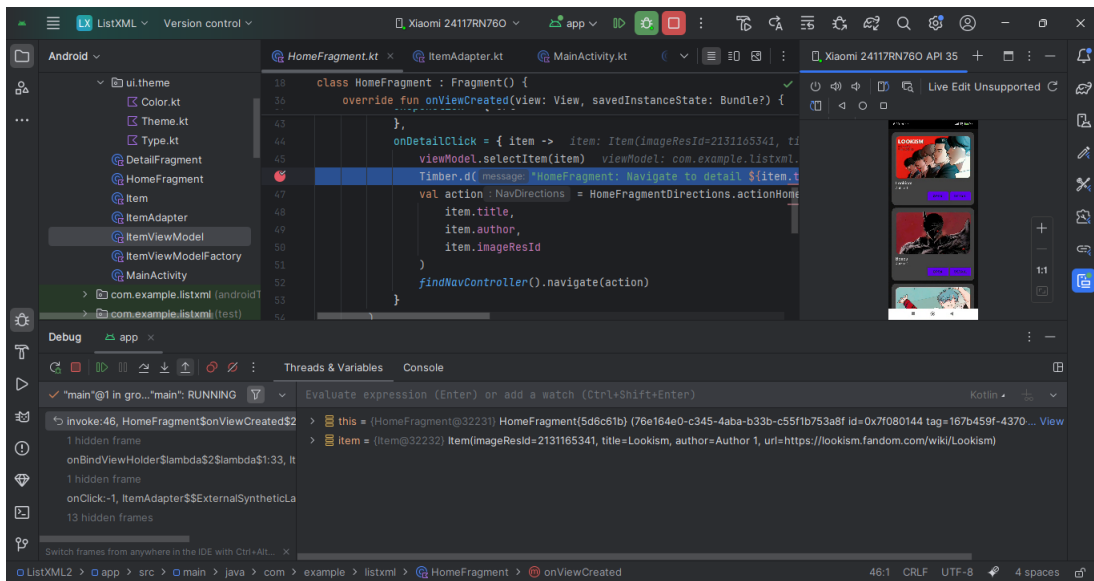
Gambar 37. Screenshot Hasil Jawaban Soal 1 XML

Step Out :



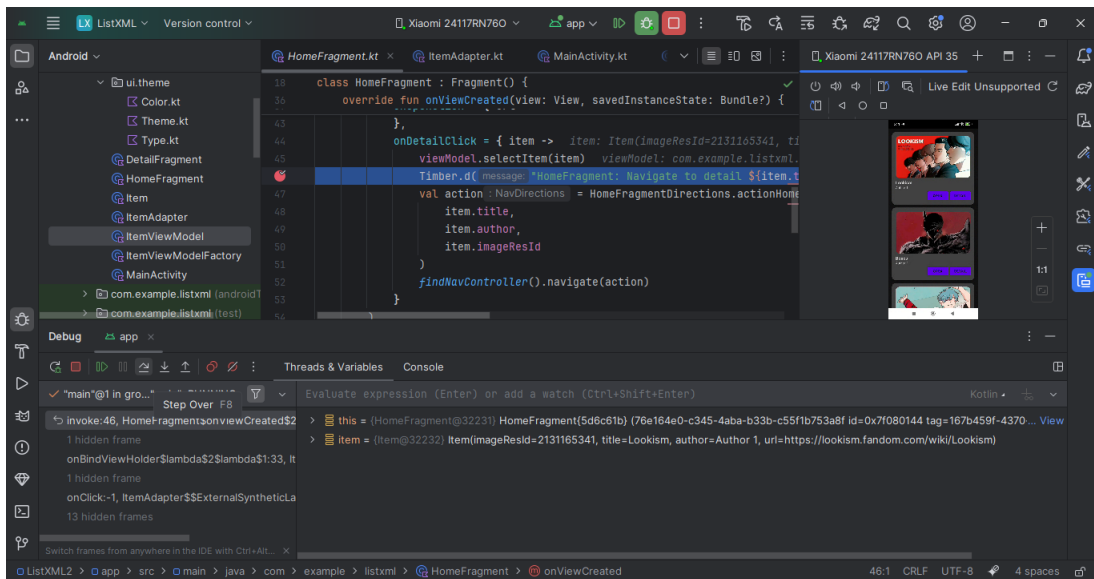
Gambar 38. Screenshot Hasil Jawaban Soal 1 XML





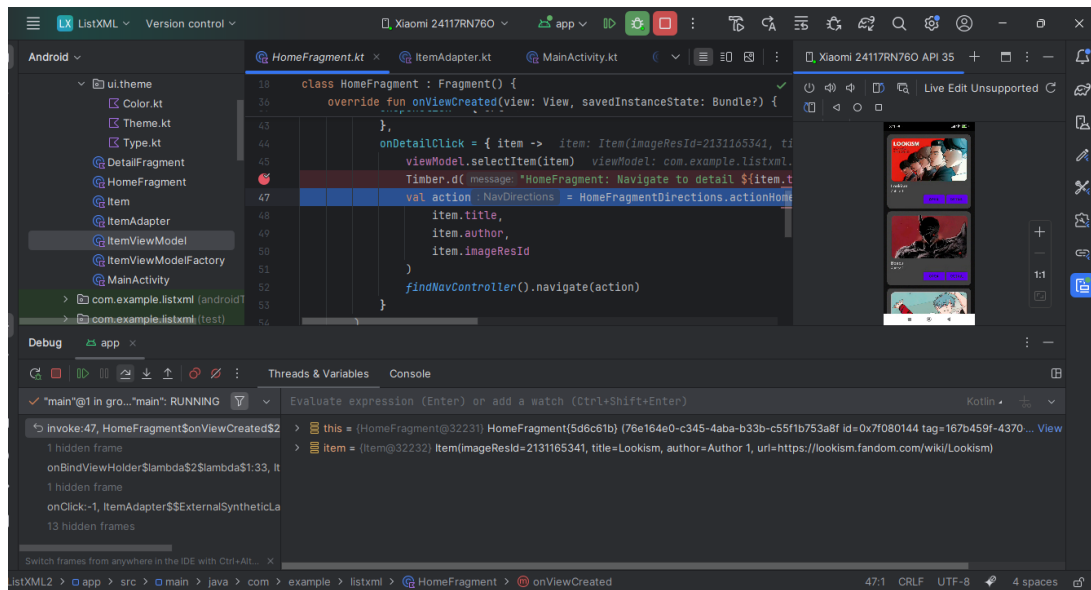
Gambar 39. Screenshot Hasil Jawaban Soal 1 XML

Step Over :



Gambar 40. Screenshot Hasil Jawaban Soal 1 XML





Gambar 41. Screenshot Hasil Jawaban Soal 1 XML

## C. Pembahasan

### Fungsi debugger :

- Menemukan penyebab **NullPointerException**, **crash**, atau hasil yang tidak sesuai.
- Melihat data yang masuk ke ViewModel, RecyclerView, atau Intent.
- Mengecek perubahan nilai StateFlow, LiveData, atau variabel lainnya.

### Cara menggunakan debugger :

#### 1. Pasang Breakpoint

- Klik di samping nomor baris kode (lingkaran merah akan muncul).
- Misalnya: Timber.d("Data: \$data") diberi breakpoint.

#### 2. Jalankan Aplikasi dalam Mode Debug

- Klik tombol "Debug App" di Android Studio.
- Aplikasi akan berjalan dan berhenti di baris yang diberi breakpoint.

#### 3. Gunakan Panel Debug

- Lihat nilai variabel di panel "Debugger".

- Bisa menambahkan Watch, melihat Stack Trace, dan mengontrol eksekusi.

### **Fitur Step Into, Step Over, dan Step Out :**

Step Into : Masuk ke dalam fungsi/metode yang sedang dipanggil.

Step Over : Loncat ke baris berikutnya tanpa masuk ke dalam fungsi/metode.

Step Out : Keluar dari fungsi saat ini dan kembali ke fungsi pemanggil.

### **Jetpack Compose :**

#### **MainActivity.kt:**

Dalam kode MainActivity, digunakan library Timber untuk keperluan logging selama proses pengembangan aplikasi. Logging sangat penting untuk memonitor alur program, memeriksa data yang sedang diproses, serta melacak bug atau error.

Line 17, digunakan untuk mengaktifkan Timber agar bisa menampilkan log saat aplikasi dijalankan dalam mode debug. DebugTree() merupakan implementasi Timber yang cocok digunakan selama proses pengembangan karena akan mencetak log ke Logcat.

Line 31, dapat dipasang **breakpoint** untuk menghentikan sementara eksekusi program saat sedang dijalankan dalam mode **debug**. Tujuannya adalah untuk:

- Melihat isi data yang dikembalikan oleh fungsi `getItems(context)`.
- Memastikan bahwa data items berisi list yang benar sebelum diteruskan ke DetailScreen.
- Memantau apakah proses pemanggilan data dari resource (judul, deskripsi, gambar) berhasil.

Setelah breakpoint dipasang dan aplikasi dijalankan dalam **mode debug**, kita dapat menggunakan fitur **Step Over (F8)** untuk melanjutkan ke baris berikutnya tanpa masuk ke dalam fungsi, atau **Step Into (F7)** untuk menelusuri bagaimana `getItems()` membentuk daftar Item.

#### **ItemListScreen.kt :**

Dalam ItemListScreen.kt, library Timber digunakan untuk melakukan logging aktivitas pengguna ketika tombol ditekan. Logging ini sangat membantu saat proses debugging karena memberikan informasi langsung ke Logcat Android Studio mengenai tindakan yang terjadi di aplikasi.

Line 70, mencatat pesan log ketika pengguna menekan **tombol "Website"** (explicit intent). Breakpoint bisa dipasang di baris ini untuk:

- Memastikan bahwa URL dari item yang dipilih benar dan sesuai harapan.
- Memverifikasi bahwa aksi Intent akan dilakukan ke alamat yang valid.
- Melihat kapan dan bagaimana fungsi startActivity dipicu untuk membuka halaman web.

Ini penting untuk memastikan bahwa **intent eksplisit** benar-benar membawa pengguna ke halaman manhua yang diinginkan.

Line 77, Logging ini dipicu ketika pengguna menekan tombol "**Detail**", yang akan mengarahkan ke halaman DetailScreen. Breakpoint di sini berguna untuk:

- Memeriksa nilai item.id yang dipilih dan akan dikirim melalui NavController.
- Menjamin bahwa navigasi dilakukan ke halaman detail dengan data yang benar.
- Memastikan bahwa event klik benar-benar ditangkap dan dijalankan oleh sistem navigasi.

#### **ItemViewModel.kt :**

Dalam file ItemViewModel.kt, penggunaan Timber dilakukan pada bagian inisialisasi ViewModel, tepatnya setelah data items berhasil dimuat melalui fungsi `getItems(context)`.

Line 20, Baris ini berfungsi untuk mencatat **jumlah data item** yang berhasil dimuat dari sumber data (`getItems(context)`) saat ViewModel pertama kali dijalankan. Logging ini membantu **developer mengetahui apakah data berhasil diambil** dan berapa banyak jumlahnya.

- Nilai \$param berguna untuk memberi label log yang berbeda jika ViewModel digunakan di beberapa tempat dengan parameter yang berbeda.
- `${items.size}` menampilkan jumlah item yang di-load ke dalam aplikasi.

#### **XML :**

##### **HomeFragment.kt :**

Di dalam HomeFragment, Timber digunakan untuk mencatat proses penting yang terjadi saat pengguna berinteraksi dengan daftar item. Logging ini membantu proses debugging dan pelacakan alur kerja aplikasi secara real-time melalui **Logcat** Android Studio.

Line 40, Baris ini digunakan untuk mencatat log saat pengguna menekan tombol "Open" pada salah satu item di dalam daftar. Log ini berfungsi untuk memastikan bahwa URL yang akan dibuka melalui intent eksplisit sudah benar dan proses intent telah dipicu.

Line 46, Logging ini mencatat saat pengguna menekan tombol "Detail" pada item. Ini menunjukkan bahwa data item telah berhasil dipilih oleh ViewModel, dan proses navigasi ke halaman Detail telah dimulai, beserta judul item yang sedang diproses.

Line 62, Baris ini mencatat jumlah data yang diterima dari ViewModel dan ditampilkan ke dalam RecyclerView. Logging ini penting untuk memastikan data benar-benar terkirim dan diperbarui ke adapter.

#### **DetailFragment.kt :**

Pada DetailFragment.kt, Timber digunakan sebagai alat pencatat log (*debug logging*) untuk menampilkan informasi data yang diterima dari HomeFragment melalui Safe Args. Penggunaan Timber ini sangat penting untuk membantu praktikan memverifikasi apakah data yang dikirim saat navigasi benar-benar sesuai.

Line 26, Baris log ini mencatat informasi yang diterima oleh DetailFragment setelah navigasi dilakukan dari HomeFragment. Log menampilkan data judul (title), penulis/subjudul (subtitle), dan ID gambar (imageResId) dari item yang dipilih. Tujuannya adalah memastikan bahwa semua data diteruskan dengan benar melalui argument navArgs.

#### **MainActivity.kt :**

Line 13, Baris ini berada di dalam fungsi onCreate dan digunakan untuk menginisialisasi Timber, yaitu sebuah library logging yang lebih fleksibel dan rapi dibandingkan Log.d bawaan Android.

#### **ItemViewModel.kt :**

Pada file ini, Timber digunakan untuk mencatat log debug yang berkaitan dengan dua hal penting dalam ViewModel:

1. Proses pemanggilan dan pemuatan data dummy item.
2. Proses pemilihan item oleh pengguna.

Line 21, Baris log ini dijalankan setelah daftar item dummy berhasil dimuat ke dalam \_itemList. Log mencatat jumlah item yang dimuat dan menampilkan nilai param yang dikirim ke ItemViewModel.

Line 27, Log ini dicetak ketika pengguna memilih salah satu item dari daftar. Log menampilkan judul item yang dipilih, sehingga bisa dipastikan bahwa proses pemilihan berjalan dengan benar.

#### **D. Tautan Git**

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul4>

## SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

### A. Pembahasan

Application class dalam arsitektur aplikasi Android adalah kelas dasar yang merepresentasikan seluruh aplikasi secara global dan dibuat saat aplikasi dijalankan. Fungsinya adalah untuk menginisialisasi komponen-komponen yang bersifat global dan hanya perlu dijalankan sekali selama aplikasi aktif, seperti setup library logging, dependency injection, atau database. Selain itu, Application class juga dapat menyimpan data atau state yang harus tersedia selama aplikasi berjalan. Untuk menggunakannya, kita membuat subclass dari Application dan mendaftarkannya di file AndroidManifest.xml sehingga sistem Android mengetahui kelas mana yang digunakan sebagai Application class. Application class membantu mengelola konfigurasi dan data global agar aplikasi berjalan dengan lebih terstruktur dan efisien.

## MODUL 5 : Array

### SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:  
<https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

#### A. Source Code

##### MainActivity.kt :

*Tabel 40. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                          |
|----|----------------------------------------------------------|
| 1  | package com.example.listcompose                          |
| 2  |                                                          |
| 3  | import android.os.Bundle                                 |
| 4  | import androidx.activity.ComponentActivity               |
| 5  | import androidx.activity.compose.setContent              |
| 6  | import androidx.lifecycle.viewmodel.compose.viewModel    |
| 7  | import androidx.navigation.NavType                       |
| 8  | import androidx.navigation.compose.*                     |
| 9  | import androidx.navigation.navArgument                   |
| 10 | import com.example.listcompose.data.local.AppDatabase    |
| 11 | import                                                   |
| 12 | com.example.listcompose.data.preferences.PreferencesMana |

```

13 ger
14 import
15 com.example.listcompose.data.remote.MovieRepository
16 import
17 com.example.listcompose.data.remote.RetrofitInstance
18 import com.example.listcompose.ui.screens.*
19 import com.example.listcompose.ui.theme.ListComposeTheme
20 import com.example.listcompose.viewmodel.MovieViewModel
21 import
22 com.example.listcompose.viewmodel.MovieViewModelFactory
23 import timber.log.Timber
24
25 class MainActivity : ComponentActivity() {
26     override fun onCreate(savedInstanceState: Bundle?) {
27         super.onCreate(savedInstanceState)
28         Timber.plant(Timber.DebugTree())
29
30         val context = this
31         val database = AppDatabase.getInstance(context)
32         val repository = MovieRepository(
33             apiService = RetrofitInstance.api,
34             movieDao = database.movieDao()
35         )
36         val preferencesManager =
37 PreferencesManager(context)
38         val viewModelFactory =
39 MovieViewModelFactory(application, repository,
40 preferencesManager)
41
42         setContent {
43             ListComposeTheme {
44                 val navController =
45 rememberNavController()
46                 val movieViewModel: MovieViewModel =
47 viewModel(factory = viewModelFactory)
48                 val username =
49 movieViewModel.getUsername()
50
51                 NavHost(navController, startDestination
52 = "home") {
53                     composable("home") {
54                         HomeScreen(
55                             username = username,
56                             onNavigateToMovies = {
57
58 movieViewModel.loadMovies()
59
60 navController.navigate("movies")
61                     },

```



|    |                                                      |
|----|------------------------------------------------------|
| 62 | onNavigateToLocalMovies = {                          |
| 63 |                                                      |
| 64 | navController.navigate("local_movies")               |
| 65 | },                                                   |
| 66 | onNavigateToSettings = {                             |
| 67 |                                                      |
| 68 | navController.navigate("settings")                   |
| 69 | },                                                   |
| 70 | navController =                                      |
| 71 | navController,                                       |
| 72 | viewModel = movieViewModel                           |
| 73 | )                                                    |
| 74 | }                                                    |
| 75 |                                                      |
| 76 | composable("movies") {                               |
| 77 | MovieListScreen(                                     |
| 78 | navController =                                      |
| 79 | navController,                                       |
| 80 | viewModel = movieViewModel,                          |
| 81 | isLocal = false                                      |
| 82 | )                                                    |
| 83 | }                                                    |
| 84 |                                                      |
| 85 | composable("local_movies") {                         |
| 86 | MovieListScreen(                                     |
| 87 | navController =                                      |
| 88 | navController,                                       |
| 89 | viewModel = movieViewModel,                          |
| 90 | isLocal = true                                       |
| 91 | )                                                    |
| 92 | }                                                    |
| 93 |                                                      |
| 94 | composable(                                          |
| 95 | "movie_detail/{id}",                                 |
| 96 | arguments =                                          |
|    | listOf(navArgument("id") { type = NavType.IntType }) |
|    | ) { backStackEntry ->                                |
|    | val id =                                             |
|    | backStackEntry.arguments?.getInt("id") ?: 0          |
|    | MovieDetailScreen(                                   |
|    | movieId = id,                                        |
|    | viewModel = movieViewModel                           |
|    | )                                                    |
|    | }                                                    |
|    |                                                      |
|    | composable("settings") {                             |
|    | SettingsScreen(                                      |
|    | viewModel = movieViewModel,                          |
|    | onBackClick = {                                      |

|  |                                                                                                   |
|--|---------------------------------------------------------------------------------------------------|
|  | <pre>navController.popBackStack() }<br/>    )<br/>    }<br/>    }<br/>    }<br/>    }<br/>}</pre> |
|--|---------------------------------------------------------------------------------------------------|

**local :**

Tabel 41. Source Code Jawaban Soal 1 Jetpack Compose

```

1 package com.example.listcompose.data.local
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7 import com.example.listcompose.data.model.Movie
8
9 @Database(
10     entities = [Movie::class],
11     version = 7,
12     exportSchema = true
13 )
14 abstract class AppDatabase : RoomDatabase() {
15     abstract fun movieDao(): MovieDao
16
17     companion object {
18         fun getInstance(context: Context): AppDatabase {
19             return Room.databaseBuilder(
20                 context,
21                 AppDatabase::class.java,
22                 "movie_db"
23             )
24                 .fallbackToDestructiveMigration()
25                 .build()
26         }
27     }
28 }

```

## MovieDao.kt :

Tabel 42. Source Code Jawaban Soal 1 Jetpack Compose

|    |                                                   |
|----|---------------------------------------------------|
| 1  | package com.example.listcompose.data.local        |
| 2  |                                                   |
| 3  | import androidx.room.Dao                          |
| 4  | import androidx.room.Insert                       |
| 5  | import androidx.room.OnConflictStrategy           |
| 6  | import androidx.room.Query                        |
| 7  | import com.example.listcompose.data.model.Movie   |
| 8  | import kotlinx.coroutines.flow.Flow               |
| 9  |                                                   |
| 10 | @Dao                                              |
| 11 | interface MovieDao {                              |
| 12 | @Query("SELECT * FROM movies")                    |
| 13 | suspend fun getAllMovies(): Flow<List<Movie>>     |
| 14 |                                                   |
| 15 | @Query("SELECT MAX(last_updated) FROM movies")    |
| 16 | suspend fun getLastUpdateTime(): Long?            |
| 17 |                                                   |
| 18 | @Insert(onConflict = OnConflictStrategy.REPLACE)  |
| 19 | suspend fun insertMovies(movies: List<Movie>)     |
| 20 |                                                   |
| 21 | @Query("DELETE FROM movies")                      |
| 22 | suspend fun clearAll()                            |
| 23 |                                                   |
| 24 | @Query("SELECT * FROM movies WHERE is_local = 1") |
| 25 | suspend fun getLocalMovies(): Flow<List<Movie>>   |
| 26 |                                                   |
| 27 | @Query("SELECT * FROM movies WHERE id = :id")     |
| 28 | suspend fun getMovieById(id: Int): Movie?         |
| 29 |                                                   |
| 30 | }                                                 |

## model :

## Movie.kt :

Tabel 43. Source Code Jawaban Soal 1 Jetpack Compose

|   |                                            |
|---|--------------------------------------------|
| 1 | package com.example.listcompose.data.model |
| 2 |                                            |
| 3 | import androidx.room.ColumnInfo            |
| 4 | import androidx.room.Entity                |
| 5 | import androidx.room.PrimaryKey            |

```

6 import kotlinx.serialization.SerialName
7 import kotlinx.serialization.Serializable
8
9 @Serializable
10 @Entity(tableName = "movies")
11 data class Movie(
12     @PrimaryKey
13     val id: Int,
14
15     val title: String,
16     val overview: String,
17
18     @SerialName("poster_path")
19     @ColumnInfo(name = "poster_path")
20     val posterPath: String?,
21
22     @SerialName("vote_average")
23     @ColumnInfo(name = "vote_average")
24     val voteAverage: Double,
25
26     @SerialName("release_date")
27     @ColumnInfo(name = "release_date")
28     val releaseDate: String,
29
30     val popularity: Double,
31
32     // Additional fields for local storage
33     @ColumnInfo(name = "is_local")
34     val isLocal: Boolean = false,
35
36     @ColumnInfo(name = "last_updated")
37     val lastUpdated: Long = System.currentTimeMillis(),
38
39     @ColumnInfo(name = "tmdb_url")
40     val tmdbUrl: String = "",
41
42     // Optional fields from API that we might not
43     always use
44     @SerialName("adult")
45     val adult: Boolean? = null,
46
47     @SerialName("backdrop_path")
48     val backdropPath: String? = null
49 )
50
51 @Serializable
52 data class MovieResponse(
53     val page: Int,
54     val results: List<Movie>,

```

|    |                                                        |
|----|--------------------------------------------------------|
| 55 | @SerializedName("total_pages") val totalPages: Int,    |
| 56 | @SerializedName("total_results") val totalResults: Int |
|    | )                                                      |

**preferences :**

**PreferencesManager.kt :**

*Tabel 44. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                  |
|----|--------------------------------------------------|
| 1  | package com.example.listcompose.data.preferences |
| 2  |                                                  |
| 3  | import android.content.Context                   |
| 4  | import android.content.SharedPreferences         |
| 5  | import androidx.core.content.edit                |
| 6  |                                                  |
| 7  | class PreferencesManager(context: Context) {     |
| 8  | private val prefs: SharedPreferences =           |
| 9  | context.getSharedPreferences("user_prefs",       |
| 10 | Context.MODE_PRIVATE)                            |
| 11 |                                                  |
| 12 | companion object {                               |
| 13 | private const val KEY_USERNAME = "username"      |
| 14 | private const val KEY_LAST_VIEWED_MOVIE_ID =     |
| 15 | "last_viewed_movie_id"                           |
| 16 | }                                                |
| 17 |                                                  |
| 18 | fun setUsername(username: String) {              |
| 19 | prefs.edit {                                     |
| 20 | putString(KEY_USERNAME, username)                |
| 21 | }                                                |
| 22 | }                                                |
| 23 |                                                  |
| 24 | fun getUsername(): String {                      |
| 25 | return prefs.getString(KEY_USERNAME, "") ?: ""   |
| 26 | }                                                |
| 27 |                                                  |
| 28 | fun saveLastViewedMovieId(movieId: Int) {        |
| 29 | prefs.edit { putInt(KEY_LAST_VIEWED_MOVIE_ID,    |
| 30 | movieId) }                                       |
| 31 | }                                                |
| 32 |                                                  |
|    | fun getLastViewedMovieId(): Int {                |
|    | return prefs.getInt(KEY_LAST_VIEWED_MOVIE_ID, -  |
|    | 1)                                               |
|    | }                                                |
|    | }                                                |

**remote :**

**MovieRepository.kt :**

*Tabel 45. Source Code Jawaban Soal 1 Jetpack Compose*

```
1 package com.example.listcompose.data.remote
2
3 import com.example.listcompose.BuildConfig
4 import com.example.listcompose.data.local.MovieDao
5 import com.example.listcompose.data.model.Movie
6 import kotlinx.coroutines.flow.Flow
7 import kotlinx.coroutines.flow.catch
8 import kotlinx.coroutines.flow.first
9 import timber.log.Timber
10
11 class MovieRepository(
12     private val apiService: TmdbApiService,
13     private val movieDao: MovieDao
14 ) {
15     companion object {
16         private const val CACHE_EXPIRATION_TIME_MS = 30
17         * 60 * 1000L
18     }
19
20     private var currentPage = 1
21     private var totalPages = 1
22
23     suspend fun refreshMovies(forceRefresh: Boolean =
24 false): Boolean {
25         return try {
26             val cachedMovies =
27 movieDao.getAllMovies().first()
28             val shouldRefresh = forceRefresh ||
29 isCacheExpired() || cachedMovies.isEmpty()
30
31             if (!shouldRefresh && currentPage >=
32 totalPages) {
33                 return false
34             }
35
36             if (shouldRefresh) {
37                 currentPage = 1
38                 if (forceRefresh) {
39                     movieDao.clearAll()
40                 }
41             }
42         }
```

```

43         val response = apiService.getPopularMovies(
44             apiKey = BuildConfig.TMDB_API_KEY,
45             language = "en-US",
46             page = currentPage
47         )
48
49         totalPages = response.totalPages
50
51         response.results.takeIf { it.isNotEmpty()
52     }?.let { movies ->
53             val entities = movies.map { movie ->
54                 Movie(
55                     id = movie.id,
56                     title = movie.title,
57                     overview = movie.overview,
58                     posterPath = movie.posterPath ?:
59                     "",
60                     voteAverage = movie.voteAverage,
61                     releaseDate = movie.releaseDate,
62                     popularity = movie.popularity,
63                     lastUpdated =
64                     System.currentTimeMillis(),
65                     tmdbUrl =
66                     "https://www.themoviedb.org/movie/${movie.id}"
67                 )
68             }
69             movieDao.insertMovies(entities)
70
71             if (!forceRefresh && currentPage <
72     totalPages) {
73                 currentPage++
74             }
75         }
76         true
77     } catch (e: Exception) {
78         Timber.e(e, "Failed to refresh movies")
79         throw e
80     }
81     }
82
83     private suspend fun isCacheExpired(): Boolean {
84         return movieDao.getLastUpdateTime()?.let {
85     lastUpdated ->
86         System.currentTimeMillis() - lastUpdated >
87     CACHE_EXPIRATION_TIME_MS
88     } ?: true
89     }
90
91     fun getMovies(): Flow<List<Movie>> =

```

```

92 movieDao.getAllMovies()
93     .catch { e ->
94         Timber.e(e, "Error loading movies from DB")
95         emit(emptyList())
96     }

    fun getLocalMovies(): Flow<List<Movie>> =
movieDao.getLocalMovies()

    suspend fun saveMovieToLocal(movie: Movie) {
        movieDao.insertMovies(listOf(movie.copy(
            lastUpdated = System.currentTimeMillis(),
            isLocal = true
        )))
    }

    suspend fun getMovieById(id: Int): Movie? {
        return movieDao.getMovieById(id)
    }
}

```

### **RetrofitInstance.kt :**

*Tabel 46. Source Code Jawaban Soal 1 Jetpack Compose*

```

1  package com.example.listcompose.data.remote
2
3  import com.jakewharton.retrofit2.converter.kotlinx
4  .serialization.asConverterFactory
5  import kotlinx.serialization.json.Json
6  import okhttp3.MediaType.Companion.toMediaType
7  import okhttp3.OkHttpClient
8  import okhttp3.logging.HttpLoggingInterceptor
9  import retrofit2.Retrofit
10
11 object RetrofitInstance {
12     private const val BASE_URL =
13     "https://api.themoviedb.org/3/"
14
15     private val json = Json {
16         ignoreUnknownKeys = true
17         isLenient = true
18         coerceInputValues = true
19     }
20
21     val api: TmdbApiService by lazy {
22         val loggingInterceptor =
23         HttpLoggingInterceptor().apply {

```



|    |                                              |
|----|----------------------------------------------|
| 24 | level = HttpLoggingInterceptor.Level.BODY    |
| 25 | }                                            |
| 26 |                                              |
| 27 | val client = OkHttpClient.Builder()          |
| 28 | .addInterceptor(loggingInterceptor)          |
| 29 | .build()                                     |
| 30 |                                              |
| 31 | Retrofit.Builder()                           |
| 32 | .baseUrl(BASE_URL)                           |
| 33 | .client(client)                              |
| 34 | .addConverterFactory(json.asConverterFactory |
| 35 | ("application/json".toMediaType()))          |
|    | .build()                                     |
|    | .create(TmdbApiService::class.java)          |
|    | }                                            |
|    | }                                            |

**TmdbApiService :**

*Tabel 47. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | package com.example.listcompose.data.remote             |
| 2  |                                                         |
| 3  | import com.example.listcompose.data.model.MovieResponse |
| 4  | import retrofit2.http.GET                               |
| 5  | import retrofit2.http.Query                             |
| 6  |                                                         |
| 7  | interface TmdbApiService {                              |
| 8  | @GET("movie/popular")                                   |
| 9  | suspend fun getPopularMovies(                           |
| 10 | @Query("api_key") apiKey: String,                       |
| 11 | @Query("language") language: String = "en-US",          |
| 12 | @Query("page") page: Int = 1                            |
| 13 | ): MovieResponse                                        |
| 14 | }                                                       |

**ui :**

**components :**

**Card.kt :**

*Tabel 48. Source Code Jawaban Soal 1 Jetpack Compose*

|   |                                               |
|---|-----------------------------------------------|
| 1 | package com.example.listcompose.ui.components |
| 2 |                                               |

```

3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.layout.*
5 import
6 androidx.compose.foundation.shape.RoundedCornerShape
7 import androidx.compose.material.icons.Icons
8 import
9 androidx.compose.material.icons.automirrored.filled.TrendingUp
10
11 import androidx.compose.material.icons.filled.Star
12 import androidx.compose.material3.*
13 import androidx.compose.runtime.Composable
14 import androidx.compose.ui.Alignment
15 import androidx.compose.ui.Modifier
16 import androidx.compose.ui.draw.clip
17 import androidx.compose.ui.graphics.Color
18 import androidx.compose.ui.layout.ContentScale
19 import androidx.compose.ui.res.stringResource
20 import androidx.compose.ui.unit.dp
21 import coil.compose.rememberAsyncImagePainter
22 import com.example.listcompose.R
23 import com.example.listcompose.data.model.Movie
24
25 @Composable
26 fun MovieCard(
27     movie: Movie,
28     onDetailClick: () -> Unit,
29     onSaveClick: (() -> Unit)? = null
30 ) {
31     Card(
32         modifier = Modifier
33             .padding(8.dp)
34             .fillMaxWidth(),
35         elevation =
36 CardDefaults.cardElevation(defaultElevation = 4.dp)
37     ) {
38         Column(modifier = Modifier.padding(16.dp)) {
39
40             Image(
41                 painter = rememberAsyncImagePainter(
42                     model =
43 "https://image.tmdb.org/t/p/w500${movie.posterPath}"
44                 ),
45                 contentDescription = movie.title,
46                 modifier = Modifier
47                     .fillMaxWidth()
48                     .height(200.dp)
49                     .clip(RoundedCornerShape(8.dp)),
50                 contentScale = ContentScale.Crop
51             )

```

```

52
53         Spacer(modifier = Modifier.height(8.dp))
54
55         Text(
56             text = movie.title,
57             style =
58 MaterialTheme.typography.titleLarge
59         )
60
61         Row(
62             modifier = Modifier.padding(top =
63 4.dp),
64             verticalAlignment =
65 Alignment.CenterVertically
66         ) {
67             Icon(
68                 imageVector = Icons.Default.Star,
69                 contentDescription =
70 stringResource(R.string.rating),
71                 tint = Color(0xFFFFD700)
72             )
73             Text(
74                 text = " ${movie.voteAverage}/10",
75                 modifier = Modifier.padding(start =
76 4.dp)
77             )
78
79             Spacer(modifier = Modifier.width(8.dp))
80
81             Icon(
82                 imageVector =
83 Icons.AutoMirrored.Filled.TrendingUp,
84                 contentDescription =
85 stringResource(R.string.popularity),
86                 tint = Color(0xFF4CAF50)
87             )
88             Text(
89                 text = "
90 ${movie.popularity.toInt()}",
91                 modifier = Modifier.padding(start =
92 4.dp)
93             )
94
95             Spacer(modifier = Modifier.width(8.dp))
96
97             Text(text = movie.releaseDate)
98         }
99
100         Text(

```

|     |                                                     |
|-----|-----------------------------------------------------|
| 101 | text = movie.overview.take(100) +                   |
| 102 | "...",                                              |
| 103 | modifier = Modifier.padding(top =                   |
| 104 | 8.dp),                                              |
| 105 | style =                                             |
| 106 | MaterialTheme.typography.bodyMedium                 |
| 107 | )                                                   |
| 108 |                                                     |
| 109 | Button(                                             |
| 110 | onClick = onDetailClick,                            |
| 111 | modifier = Modifier                                 |
| 112 | .fillMaxWidth()                                     |
| 113 | .padding(top = 16.dp)                               |
|     | ) {                                                 |
|     | Text(stringResource(R.string.view_details_button))  |
|     | }                                                   |
|     | if (onSaveClick != null) {                          |
|     | Button(                                             |
|     | onClick = onSaveClick,                              |
|     | modifier = Modifier                                 |
|     | .fillMaxWidth()                                     |
|     | .padding(top = 8.dp)                                |
|     | ) {                                                 |
|     | Text(stringResource(R.string.save_to_local_button)) |
|     | }                                                   |
|     | }                                                   |
|     | }                                                   |
|     | }                                                   |

**screens :**

**HomeScreen.kt :**

*Tabel 49. Source Code Jawaban Soal 1 Jetpack Compose*

|   |                                                        |
|---|--------------------------------------------------------|
| 1 | package com.example.listcompose.ui.screens             |
| 2 |                                                        |
| 3 | import androidx.compose.foundation.layout.Arrangement  |
| 4 | import androidx.compose.foundation.layout.Column       |
| 5 | import androidx.compose.foundation.layout.Spacer       |
| 6 | import androidx.compose.foundation.layout.fillMaxSize  |
| 7 | import androidx.compose.foundation.layout.fillMaxWidth |

```

8 import androidx.compose.foundation.layout.height
9 import androidx.compose.foundation.layout.padding
10 import androidx.compose.material3.Button
11 import androidx.compose.material3.ButtonDefaults
12 import androidx.compose.material3.MaterialTheme
13 import androidx.compose.material3.Text
14 import androidx.compose.runtime.Composable
15 import androidx.compose.runtime.collectAsState
16 import androidx.compose.runtime.derivedStateOf
17 import androidx.compose.runtime.getValue
18 import androidx.compose.runtime.remember
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.unit.dp
22 import androidx.lifecycle.viewmodel.compose.viewModel
23 import androidx.navigation.NavController
24 import com.example.listcompose.viewmodel.MovieViewModel
25
26 @Composable
27 fun HomeScreen(
28     username: String,
29     onNavigateToMovies: () -> Unit,
30     onNavigateToLocalMovies: () -> Unit,
31     onNavigateToSettings: () -> Unit,
32     navController: NavController,
33     viewModel: MovieViewModel = viewModel()
34 ) {
35     // Collect the local movies to check if last viewed
36     movie exists
37     val localMovies by
38     viewModel.localMovies.collectAsState()
39
40     // Get the last viewed movie and check if it exists
41     in local movies
42     val lastViewedMovie by remember {
43         derivedStateOf {
44             viewModel.getLastViewedMovie()?.takeIf {
45 movie ->
46                 localMovies.any { it.id == movie.id }
47             }
48         }
49     }
50
51     Column(
52         modifier = Modifier
53             .padding(16.dp)
54             .fillMaxSize(),
55         horizontalAlignment =
56 Alignment.CenterHorizontally

```

```

57     ) {
58         if (username.isNotEmpty()) {
59             Text(
60                 text = "Welcome, $username",
61                 style =
62                     MaterialTheme.typography.headlineMedium,
63                 modifier = Modifier.padding(bottom =
64                     16.dp))
65             }
66
67             Spacer(modifier = Modifier.height(32.dp))
68
69             // Main buttons
70             Column(
71                 modifier = Modifier
72                     .fillMaxWidth()
73                     .weight(1f),
74                 verticalArrangement = Arrangement.Center
75             ) {
76                 Button(
77                     onClick = onNavigateToMovies,
78                     modifier = Modifier
79                         .fillMaxWidth()
80                         .padding(vertical = 8.dp),
81                     colors = ButtonDefaults.buttonColors(
82                         containerColor =
83                         MaterialTheme.colorScheme.primaryContainer,
84                         contentColor =
85                         MaterialTheme.colorScheme.onPrimaryContainer
86                     )
87                 ) {
88                     Text("Browse TMDB Movies")
89                 }
90
91                 Button(
92                     onClick = onNavigateToLocalMovies,
93                     modifier = Modifier
94                         .fillMaxWidth()
95                         .padding(vertical = 8.dp),
96                     colors = ButtonDefaults.buttonColors(
97                         containerColor =
98                         MaterialTheme.colorScheme.primaryContainer,
99                         contentColor =
100                        MaterialTheme.colorScheme.onSecondaryContainer
101                     )
102                 ) {
103                     Text("View Saved Movies")
104                 }
105             }

```

```

106
107         // Bottom section with last viewed and settings
108         Column(
109             modifier = Modifier.fillMaxWidth(),
110             verticalArrangement = Arrangement.Bottom
111         ) {
112             if (lastViewedMovie != null) {
113                 Text(
114                     text = "Last viewed saved movies:
115 ${lastViewedMovie!!.title}",
116                     style =
117 MaterialTheme.typography.bodyMedium,
118                     modifier = Modifier.padding(bottom
119 = 8.dp)
120                 )
121                 Button(
122                     onClick = {
123
124 navController.navigate("movie_detail/${lastViewedMovie!
125 !.id}") {
126                                     // Clear back stack to
127 avoid multiple home screens
128
129 popUpTo(navController.graph.startDestinationId)
130                                     launchSingleTop = true
131                                 }
132                             },
133                     modifier = Modifier
134                         .fillMaxWidth()
135                         .padding(top = 8.dp),
136                     colors =
137 ButtonDefaults.buttonColors(
138                         containerColor =
139 MaterialTheme.colorScheme.tertiaryContainer,
140                         contentColor =
141 MaterialTheme.colorScheme.onTertiaryContainer
142                     )
143                 ) {
144                     Text("View Again")
145                 }
146
147                 Button(
148                     onClick = onNavigateToSettings,
149                     modifier = Modifier
150                         .fillMaxWidth()
151                         .padding(top = 8.dp),
152                     colors = ButtonDefaults.buttonColors(
153                         containerColor =

```

|  |                                                                                                                                                                                                                   |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre> MaterialTheme.colorScheme.secondaryContainer,                 contentColor = MaterialTheme.colorScheme.onSecondaryContainer             )         ) {             Text("Settings")         }     } } </pre> |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### MovieDetailScreen.kt :

*Tabel 50. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                               |
|----|---------------------------------------------------------------|
| 1  | package com.example.listcompose.ui.screens                    |
| 2  |                                                               |
| 3  | import android.content.Intent                                 |
| 4  | import android.widget.Toast                                   |
| 5  | import androidx.compose.foundation.Image                      |
| 6  | import androidx.compose.foundation.layout.*                   |
| 7  | import androidx.compose.foundation.rememberScrollState        |
| 8  | import                                                        |
| 9  | androidx.compose.foundation.shape.RoundedCornerShape          |
| 10 | import androidx.compose.foundation.verticalScroll             |
| 11 | import androidx.compose.material.icons.Icons                  |
| 12 | import                                                        |
| 13 | androidx.compose.material.icons.automirrored.filled.OpenInNew |
| 14 | import androidx.compose.material.icons.filled.Star            |
| 15 | import androidx.compose.material3.*                           |
| 16 | import androidx.compose.runtime.*                             |
| 17 | import androidx.compose.ui.Alignment                          |
| 18 | import androidx.compose.ui.Modifier                           |
| 19 | import androidx.compose.ui.draw.clip                          |
| 20 | import androidx.compose.ui.graphics.Color                     |
| 21 | import androidx.compose.ui.layout.ContentScale                |
| 22 | import androidx.compose.ui.platform.LocalContext              |
| 23 | import androidx.compose.ui.res.stringResource                 |
| 24 | import androidx.compose.ui.unit.dp                            |
| 25 | import androidx.core.net.toUri                                |
| 26 | import coil.compose.rememberAsyncImagePainter                 |
| 27 | import com.example.listcompose.R                              |
| 28 | import com.example.listcompose.data.model.Movie               |
| 29 | import com.example.listcompose.viewmodel.MovieViewModel       |
| 30 |                                                               |
| 31 |                                                               |
| 32 | @Composable                                                   |



```

33 fun MovieDetailScreen(
34     movieId: Int,
35     viewModel: MovieViewModel
36 ) {
37     val remoteMovies by
38     viewModel.movieList.collectAsState()
39     val localMovies by
40     viewModel.localMovies.collectAsState()
41     val context = LocalContext.current
42
43     var movie by remember {
44     mutableStateOf<Movie?>(null) }
45     var isLoading by remember { mutableStateOf(true) }
46     var error by remember {
47     mutableStateOf<String?>(null) }
48
49     LaunchedEffect(movieId, remoteMovies, localMovies)
50     {
51         isLoading = true
52         error = null
53
54         val localMovie = localMovies.find { it.id ==
55     movieId }
56         if (localMovie != null) {
57             movie = localMovie
58             viewModel.saveLastViewedMovieId(movieId)
59             isLoading = false
60             return@LaunchedEffect
61         }
62
63         val remoteMovie = remoteMovies.find { it.id ==
64     movieId }
65         if (remoteMovie != null) {
66             movie = remoteMovie
67             isLoading = false
68             return@LaunchedEffect
69         }
70
71         var errorResId: Int? = null
72
73         try {
74             val dbMovie =
75     viewModel.getMovieById(movieId)
76             if (dbMovie != null) {
77                 movie = dbMovie
78                 if (dbMovie.isLocal) {
79
80     viewModel.saveLastViewedMovieId(movieId)
81                 }

```

```

82         } else {
83             errorResId = R.string.movie_not_found
84         }
85     } catch (e: Exception) {
86         errorResId = R.string.error_loading_movie
87     }
88
89     if (errorResId != null) {
90         error = context.getString(errorResId)
91     }
92
93     isLoading = false
94 }
95
96 if (isLoading) {
97     Box(
98         modifier = Modifier.fillMaxSize(),
99         contentAlignment = Alignment.Center
100    ) {
101        CircularProgressIndicator()
102    }
103    return
104 }
105
106 if (error != null || movie == null) {
107     Box(
108         modifier = Modifier.fillMaxSize(),
109         contentAlignment = Alignment.Center
110    ) {
111        Text(error ?:
112 stringResource(R.string.movie_data_unavailable))
113    }
114    return
115 }
116
117 val scrollState = rememberScrollState()
118 val movieToShow = movie!!
119
120 Column(
121     modifier = Modifier
122         .fillMaxSize()
123         .verticalScroll(scrollState)
124         .padding(16.dp)
125 ) {
126     Image(
127         painter = rememberAsyncImagePainter(
128             model =
129 "https://image.tmdb.org/t/p/w500${movieToShow.posterPat
130 h}"

```

```

131         ),
132         contentDescription = movieToShow.title,
133         modifier = Modifier
134             .fillMaxWidth()
135             .height(300.dp)
136             .clip(RoundedCornerShape(12.dp)),
137         contentScale = ContentScale.Crop
138     )
139
140     Spacer(modifier = Modifier.height(16.dp))
141
142     Text(
143         text = movieToShow.title,
144         style =
145     MaterialTheme.typography.headlineLarge
146     )
147
148     Row(
149         modifier = Modifier.padding(top = 8.dp),
150         verticalAlignment =
151     Alignment.CenterVertically
152     ) {
153         Icon(
154             imageVector = Icons.Default.Star,
155             contentDescription =
156     stringResource(R.string.rating),
157             tint = Color(0xFFFFD700)
158         )
159         Text(
160             text = "
161     ${movieToShow.voteAverage}/10",
162             style =
163     MaterialTheme.typography.titleMedium,
164             modifier = Modifier.padding(start =
165     4.dp)
166         )
167         Spacer(modifier = Modifier.width(16.dp))
168         Text(
169             text = movieToShow.releaseDate,
170             style =
171     MaterialTheme.typography.titleMedium
172         )
173     }
174
175     Text(
176         text = movieToShow.overview,
177         style = MaterialTheme.typography.bodyLarge,
178         modifier = Modifier.padding(top = 16.dp)
179     )

```

```

180
181         Spacer(modifier = Modifier.height(24.dp))
182
183         Button(
184             onClick = {
185                 val tmdbUrl =
186 "https://www.themoviedb.org/movie/$movieId"
187                 try {
188                     val intent =
189 Intent(Intent.ACTION_VIEW, tmdbUrl.toUri())
190                     context.startActivity(intent)
191                 } catch (e: Exception) {
192                     Toast.makeText(
193                         context,
194
195 context.getString(R.string.unable_to_open_browser),
196                     Toast.LENGTH_SHORT
197                 ).show()
198             }
199         },
200         modifier = Modifier
201             .fillMaxWidth()
202             .padding(top = 8.dp),
203         colors = ButtonDefaults.buttonColors(
204             containerColor =
205 MaterialTheme.colorScheme.primary,
206             contentColor =
207 MaterialTheme.colorScheme.onPrimary
208         )
209     ) {
210         Icon(
211             imageVector =
212 Icons.AutoMirrored.Filled.OpenInNew,
213             contentDescription = null,
214             modifier =
215 Modifier.size(ButtonDefaults.IconSize)
216         )
217
218         Spacer(Modifier.width(ButtonDefaults.IconSpacing))
219         Text(stringResource(R.string.view_on_tmdb))
220     }
221 }
222

```

## MovieListScreen.kt :

*Tabel 51. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | package com.example.listcompose.ui.screens                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 2  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 3  | import androidx.compose.foundation.layout.*                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 4  | import androidx.compose.foundation.lazy.LazyColumn                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 5  | import androidx.compose.foundation.lazy.LazyListState                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 6  | import androidx.compose.foundation.lazy.items                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 7  | import                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 8  | androidx.compose.foundation.lazy.rememberLazyListState                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 9  | import androidx.compose.material3.*                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 10 | import androidx.compose.runtime.*                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 11 | import androidx.compose.ui.Alignment                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 12 | import androidx.compose.ui.Modifier                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 13 | import androidx.compose.ui.res.stringResource                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 14 | import androidx.compose.ui.unit.dp                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 15 | import androidx.navigation.NavController                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 16 | import com.example.listcompose.R                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 17 | import com.example.listcompose.data.model.Movie                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 18 | import com.example.listcompose.ui.components.MovieCard                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 19 | import com.example.listcompose.viewmodel.MovieViewModel                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 20 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 21 | @Composable                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 22 | fun MovieListScreen(<br>23     navController: NavController,<br>24     viewModel: MovieViewModel,<br>25     isLocal: Boolean<br>26 ) {<br>27     val movieState by<br>28     viewModel.movieState.collectAsState()<br>29     val movies by if (isLocal) {<br>30         viewModel.localMovies.collectAsState(initial =<br>31         emptyList())<br>32     } else {<br>33         viewModel.movieList.collectAsState()<br>34     }<br>35     val scrollState = rememberLazyListState()<br>36     val username by remember { derivedStateOf {<br>37     viewModel.getUsername() } }<br>38<br>39     LaunchedEffect(scrollState) {<br>40         snapshotFlow {<br>41         scrollState.layoutInfo.visibleItemsInfo }<br>42         .collect { visibleItems -> |

```

43         if (visibleItems.isNotEmpty() &&
44             visibleItems.last().index >=
45 movies.size - 5 &&
46             !isLocal &&
47             movieState !is
48 MovieViewModel.MovieState.Loading
49         ) {
50             viewModel.loadMovies()
51         }
52     }
53 }
54
55     Column(modifier = Modifier.fillMaxSize()) {
56         if (!isLocal) {
57             Button(
58                 onClick = {
59 viewModel.loadMovies(forceRefresh = true) },
60                 modifier = Modifier
61                     .fillMaxWidth()
62                     .padding(16.dp),
63                 enabled = movieState !is
64 MovieViewModel.MovieState.Loading
65             ) {
66
67 Text(stringResource(R.string.refresh_movies))
68             }
69         }
70
71         if (username.isNotEmpty()) {
72             Text(
73                 text =
74 stringResource(R.string.greeting_user, username),
75                 style =
76 MaterialTheme.typography.titleMedium,
77                 modifier = Modifier.padding(16.dp)
78             )
79         }
80
81         when (val state = movieState) {
82             is MovieViewModel.MovieState.Loading -> {
83                 if (movies.isEmpty()) {
84                     LoadingView()
85                 } else {
86                     MovieListView(movies,
87 navController, isLocal, scrollState, viewModel)
88                     LoadingView(modifier =
89 Modifier.fillMaxWidth())
90                 }
91             }

```

```

92
93         is MovieViewModel.MovieState.Error -> {
94             ErrorView(state.message)
95             if (state.hasData) {
96                 MovieListView(movies,
97 navController, isLocal, scrollState, viewModel)
98             } else {
99                 EmptyView(isLocal)
100             }
101         }
102
103         is MovieViewModel.MovieState.Success -> {
104             if (state.isRefreshed) {
105                 LaunchedEffect(Unit) {
106                     scrollState.scrollToItem(0)
107                 }
108             }
109
110             if (movies.isEmpty()) {
111                 EmptyView(isLocal)
112             } else {
113                 MovieListView(movies,
114 navController, isLocal, scrollState, viewModel)
115             }
116         }
117     }
118 }
119 }
120
121 @Composable
122 private fun MovieListView(
123     movies: List<Movie>,
124     navController: NavController,
125     isLocal: Boolean,
126     scrollState: LazyListState,
127     viewModel: MovieViewModel
128 ) {
129     LazyColumn(state = scrollState) {
130         items(movies) { movie ->
131             MovieCard(
132                 movie = movie,
133                 onDetailClick = {
134                     if (isLocal) {
135
136 viewModel.saveLastViewedMovieId(movie.id)
137                     }
138
139 navController.navigate("movie_detail/${movie.id}")
140                 },

```

```

141         onSaveClick = if (!isLocal) {
142             { viewModel.saveMovieToLocal(movie)
143         }
144         } else null
145     )
146 }
147 }
148 }
149
150 @Composable
151 private fun LoadingView(modifier: Modifier = Modifier)
152 {
153     Box(
154         modifier = modifier
155             .fillMaxSize()
156             .padding(16.dp),
157         contentAlignment = Alignment.Center
158     ) {
159         Column(horizontalAlignment =
160 Alignment.CenterHorizontally) {
161             CircularProgressIndicator()
162             Spacer(modifier = Modifier.height(8.dp))
163             Text(stringResource(R.string.loading_movies))
164         }
165     }
166 }
167 }
168
169 @Composable
170 private fun ErrorView(message: String) {
171     Box(
172         modifier = Modifier
173             .fillMaxSize()
174             .padding(16.dp),
175         contentAlignment = Alignment.Center
176     ) {
177         Text(
178             text =
179 stringResource(R.string.error_prefix, message),
180             color = MaterialTheme.colorScheme.error
181         )
182     }
183 }
184
185 @Composable
186 private fun EmptyView(isLocal: Boolean) {
187     Box(
188         modifier = Modifier.fillMaxSize(),
189         contentAlignment = Alignment.Center
190     )
191 }

```



|  |                                                                                                                                                                                                                                                                                                |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre>         ) {             Text(                 text = stringResource(                     if (isLocal) R.string.no_local_movies                 else R.string.no_remote_movies                 ),                 modifier = Modifier.padding(16.dp)             )         }     } </pre> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## SettingsScreen.kt :

*Tabel 52. Source Code Jawaban Soal 1 Jetpack Compose*

|    |                                                         |
|----|---------------------------------------------------------|
| 1  | package com.example.listcompose.ui.screens              |
| 2  |                                                         |
| 3  | import androidx.compose.foundation.layout.*             |
| 4  | import androidx.compose.material3.*                     |
| 5  | import androidx.compose.runtime.*                       |
| 6  | import androidx.compose.ui.Alignment                    |
| 7  | import androidx.compose.ui.Modifier                     |
| 8  | import androidx.compose.ui.res.stringResource           |
| 9  | import androidx.compose.ui.unit.dp                      |
| 10 | import com.example.listcompose.R                        |
| 11 | import com.example.listcompose.viewmodel.MovieViewModel |
| 12 |                                                         |
| 13 | @Composable                                             |
| 14 | fun SettingsScreen(                                     |
| 15 | viewModel: MovieViewModel,                              |
| 16 | onBackClick: () -> Unit                                 |
| 17 | ) {                                                     |
| 18 | val currentUsername by remember { derivedStateOf {      |
| 19 | viewModel.getUsername() } }                             |
| 20 | var usernameInput by remember {                         |
| 21 | mutableStateOf(currentUsername) }                       |
| 22 |                                                         |
| 23 | Column(                                                 |
| 24 | modifier = Modifier                                     |
| 25 | .fillMaxSize()                                          |
| 26 | .padding(16.dp),                                        |
| 27 | horizontalAlignment =                                   |
| 28 | Alignment.CenterHorizontally                            |
| 29 | ) {                                                     |
| 30 | Text(                                                   |
| 31 | text =                                                  |
| 32 | stringResource(R.string.settings_title),                |
| 33 | style =                                                 |

|    |                                                     |
|----|-----------------------------------------------------|
| 34 | MaterialTheme.typography.headlineMedium,            |
| 35 | modifier = Modifier.padding(bottom = 24.dp)         |
| 36 | )                                                   |
| 37 |                                                     |
| 38 | OutlinedTextField(                                  |
| 39 | value = usernameInput,                              |
| 40 | onValueChange = { usernameInput = it },             |
| 41 | label = {                                           |
| 42 | Text(stringResource(R.string.username_label)) },    |
| 43 | modifier = Modifier.fillMaxWidth()                  |
| 44 | )                                                   |
| 45 |                                                     |
| 46 | Spacer(modifier = Modifier.height(16.dp))           |
| 47 |                                                     |
| 48 | Button(                                             |
| 49 | onClick = {                                         |
| 50 | viewModel.updateUsername(usernameInput)             |
| 51 | onBackClick()                                       |
| 52 | },                                                  |
| 53 | modifier = Modifier.fillMaxWidth()                  |
| 54 | ) {                                                 |
| 55 |                                                     |
| 56 | Text(stringResource(R.string.save_username_button)) |
| 57 | }                                                   |
| 58 |                                                     |
| 59 | Spacer(modifier = Modifier.height(8.dp))            |
| 60 |                                                     |
| 61 | OutlinedButton(                                     |
|    | onClick = onBackClick,                              |
|    | modifier = Modifier.fillMaxWidth()                  |
|    | ) {                                                 |
|    | Text(stringResource(R.string.back_button))          |
|    | }                                                   |
|    | }                                                   |
|    | }                                                   |

**viewmodel :**

**MovieViewModel.kt :**

*Tabel 53. Source Code Jawaban Soal 1 Jetpack Compose*

|   |                                            |
|---|--------------------------------------------|
| 1 | package com.example.listcompose.viewmodel  |
| 2 |                                            |
| 3 | import android.app.Application             |
| 4 | import androidx.lifecycle.AndroidViewModel |
| 5 | import androidx.lifecycle.viewModelScope   |

```

6  import com.example.listcompose.data.model.Movie
7  import
8  com.example.listcompose.data.preferences.PreferencesMan
9  ager
10 import
11 com.example.listcompose.data.remote.MovieRepository
12 import kotlinx.coroutines.flow.MutableStateFlow
13 import kotlinx.coroutines.flow.SharingStarted
14 import kotlinx.coroutines.flow.StateFlow
15 import kotlinx.coroutines.flow.first
16 import kotlinx.coroutines.flow.stateIn
17 import kotlinx.coroutines.launch
18 import timber.log.Timber
19
20 class MovieViewModel(
21     application: Application,
22     private val repository: MovieRepository,
23     private val preferencesManager: PreferencesManager
24 ) : AndroidViewModel(application) {
25
26     sealed class MovieState {
27         data object Loading : MovieState()
28         data class Success(val movies: List<Movie>, val
29 isRefreshed: Boolean) : MovieState()
30         data class Error(val message: String, val
31 hasData: Boolean) : MovieState()
32     }
33
34     private val _movieState =
35 MutableStateFlow<MovieState>(MovieState.Loading)
36     val movieState: StateFlow<MovieState> = _movieState
37
38     val movieList: StateFlow<List<Movie>> =
39 repository.getMovies()
40         .stateIn(
41             scope = viewModelScope,
42             started =
43 SharingStarted.WhileSubscribed(5000),
44             initialValue = emptyList()
45         )
46
47     val localMovies: StateFlow<List<Movie>> =
48 repository.getLocalMovies()
49         .stateIn(
50             scope = viewModelScope,
51             started =
52 SharingStarted.WhileSubscribed(5000),
53             initialValue = emptyList()
54         )

```

```

55
56     fun loadMovies(forceRefresh: Boolean = false) {
57         viewModelScope.launch {
58             _movieState.value = MovieState.Loading
59             try {
60                 val isRefreshed =
61 repository.refreshMovies(forceRefresh)
62                 val currentMovies =
63 repository.getMovies().first()
64                 _movieState.value =
65 MovieState.Success(currentMovies, isRefreshed)
66             } catch (e: Exception) {
67                 val hasData =
68 repository.getMovies().first().isNotEmpty()
69                 _movieState.value = MovieState.Error(
70                     "Failed to load movies: ${e.message}
71     ?: "Unknown error"})",
72                     hasData
73                 )
74                 Timber.e(e, "Error loading movies")
75             }
76         }
77     }
78
79     fun saveLastViewedMovieId(id: Int) {
80         viewModelScope.launch {
81
82 preferencesManager.saveLastViewedMovieId(id)
83         }
84     }
85
86     private fun getLastViewedMovieId(): Int {
87         return
88 preferencesManager.getLastViewedMovieId()
89     }
90
91     fun getLastViewedMovie(): Movie? {
92         val lastId = getLastViewedMovieId()
93         if (lastId == -1) return null
94
95         return findMovieById(lastId)?.takeIf {
96             movieList.value.any { it.id == lastId } ||
97 localMovies.value.any { it.id == lastId }
98         }
99     }
100
101     fun getUsername(): String {
102         return preferencesManager.getUsername()
103     }

```

```

104
105     fun updateUsername(newUsername: String) {
106         viewModelScope.launch {
107
preferencesManager.saveUsername(newUsername)
            }
        }

        fun saveMovieToLocal(movie: Movie) {
            viewModelScope.launch {
                repository.saveMovieToLocal(movie)
            }
        }

        suspend fun getMovieById(id: Int): Movie? {
            return repository.getMovieById(id) ?:
findMovieById(id)
        }

        private fun findMovieById(id: Int): Movie? {
            return localMovies.value.find { it.id == id }
                ?: movieList.value.find { it.id == id }
        }
    }
}

```

### MovieViewModelFactory :

*Tabel 54. Source Code Jawaban Soal 1 Jetpack Compose*

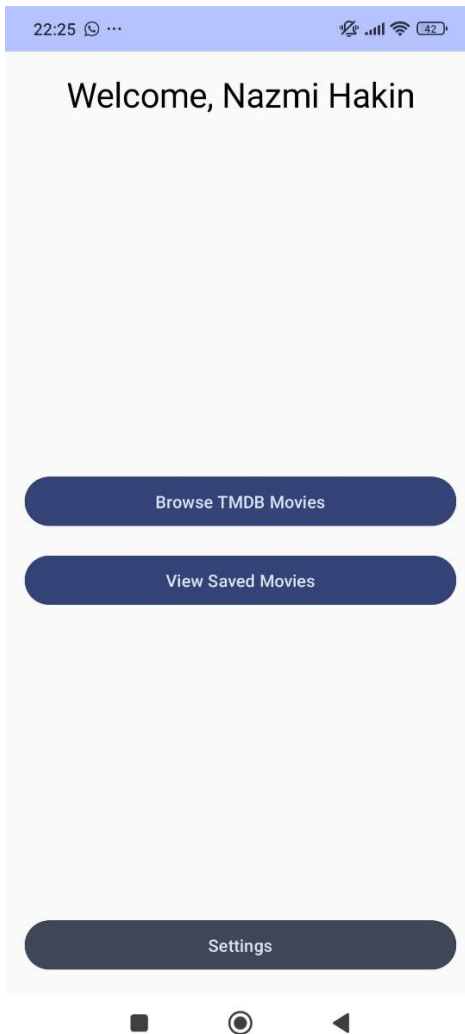
```

1 package com.example.listcompose.viewmodel
2
3 import android.app.Application
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6 import
7 com.example.listcompose.data.preferences.PreferencesManag
8 er
9 import
10 com.example.listcompose.data.remote.MovieRepository
11
12 class MovieViewModelFactory(
13     private val application: Application,
14     private val repository: MovieRepository,
15     private val preferencesManager: PreferencesManager
16 ) : ViewModelProvider.Factory {
17     override fun <T : ViewModel> create(modelClass:
18 Class<T>): T {
19         if
20

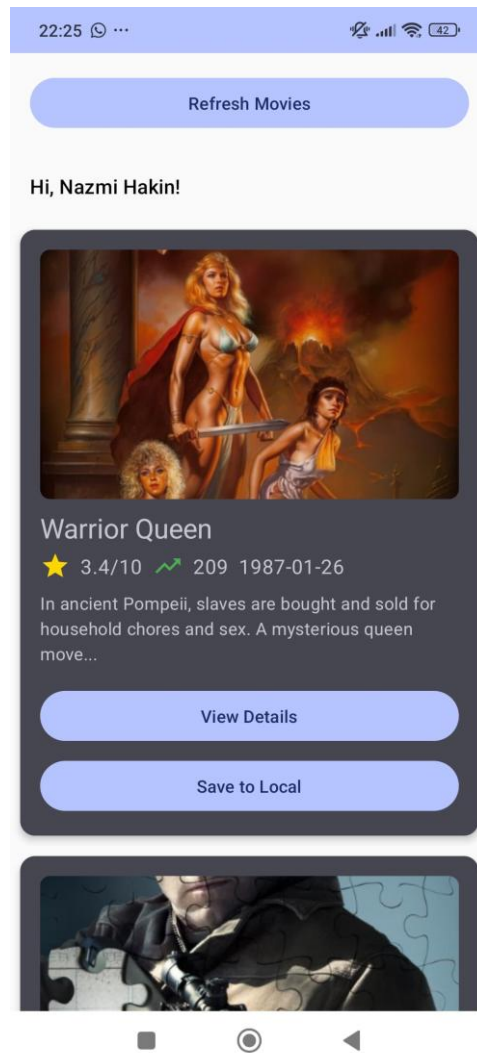
```

|   |                                                           |
|---|-----------------------------------------------------------|
| 1 | (modelClass.isAssignableFrom(MovieViewModel::class.java)) |
| 5 | {                                                         |
| 1 | @Suppress("UNCHECKED_CAST")                               |
| 6 | return MovieViewModel(application,                        |
| 1 | repository, preferencesManager) as T                      |
| 7 | }                                                         |
| 1 | throw IllegalArgumentException("Unknown ViewModel         |
| 8 | class")                                                   |
| 1 | }                                                         |
| 9 | }                                                         |
| 2 |                                                           |
| 0 |                                                           |
| 2 |                                                           |
| 1 |                                                           |

## B. Output Program



*Gambar 42. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*

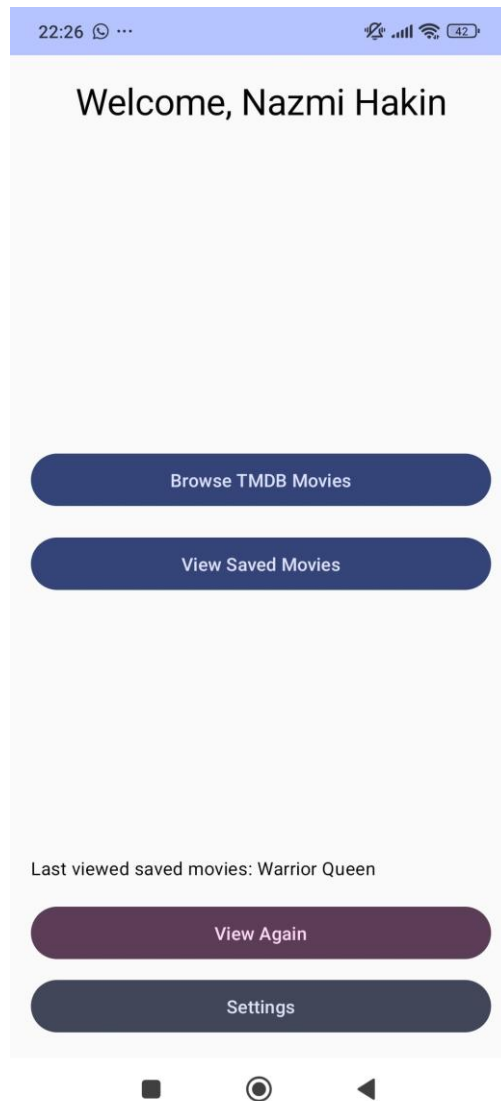


*Gambar 43. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*

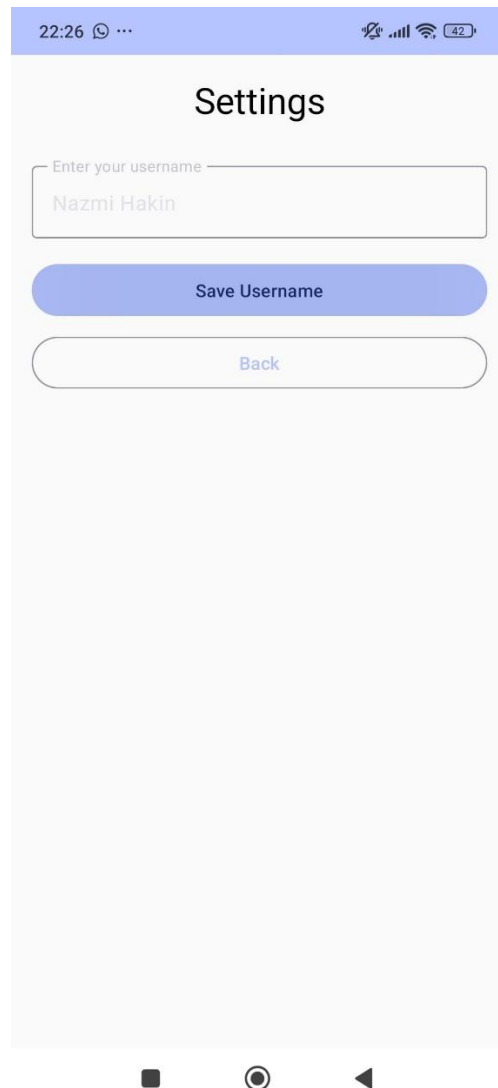




*Gambar 44. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*



*Gambar 45. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*



*Gambar 46. Screenshot Hasil Jawaban Soal 1 Jetpack Compose*

## **C. Pembahasan**

### **A. Networking dengan Retrofit + Status/Error Handling + Flow :**

#### **1. Networking with Retrofit (Pemanggilan API)**

##### **TmdbApiService**

Ini adalah interface Retrofit yang mendefinisikan endpoint:

`@GET("movie/popular")`

```
suspend fun getPopularMovies(
```

```
    @Query("api_key") apiKey: String,
```

```
    @Query("language") language: String = "en-US",
```

```
    @Query("page") page: Int = 1
```

```
): MovieResponse
```

- Menggunakan anotasi **@GET** untuk mengambil daftar **movie populer** dari TMDB.
- `suspend` artinya fungsi ini dipanggil dalam **coroutine**.

### **RetrofitInstance**

Singleton object untuk membangun Retrofit instance.

```
Retrofit.Builder()
```

```
    .baseUrl(BASE_URL)
```

```
    .client(client) // dengan logging
```

```
    .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
```

```
    .build()
```

```
    .create(TmdbApiService::class.java)
```

- Menggunakan **kotlinx.serialization** sebagai converter JSON.
- Dilengkapi dengan **HttpLoggingInterceptor** untuk debugging.

## **2. Flow + Repository Layer (Caching dan Stream Data)**

### **MovieRepository**

Bridge antara remote API dan local database (Room).

#### **Fungsi utama:**

```
suspend fun refreshMovies(forceRefresh: Boolean = false): Boolean
```

- Mengecek apakah perlu **refresh** data dari API (berdasarkan waktu dan flag `forceRefresh`).
- Jika perlu, akan:
  - Memanggil API → `apiService.getPopularMovies(...)`
  - Mapping data → ke dalam model `Movie`
  - Simpan ke database lewat `movieDao.insertMovies(...)`

### Fungsi pendukung:

```
fun getMovies(): Flow<List<Movie>>
```

```
fun getLocalMovies(): Flow<List<Movie>>
```

- Mengembalikan **Flow** dari Room DAO
- Jika ada error saat mengambil dari Room, akan log dan emit `emptyList()` (graceful fallback)

### Cek expired:

```
private suspend fun isCacheExpired(): Boolean
```

- Cek waktu `lastUpdated` movie dan dibandingkan dengan konstanta `CACHE_EXPIRATION_TIME_MS`.

## 3. ViewModel Layer + State/Error Handling

### MovieViewModel

#### State Management

```
sealed class MovieState {
```

```
    object Loading
```

```
    data class Success(val movies: List<Movie>, val isRefreshed: Boolean)
```

```
    data class Error(val message: String, val hasData: Boolean)
```

```
}
```

- Enum-like class untuk menunjukkan status:
  - Loading: sedang ambil data
  - Success: data berhasil didapat

- Error: ada kesalahan + info apakah masih ada data lokal

### **Alur utama: loadMovies()**

```
fun loadMovies(forceRefresh: Boolean = false) {
    viewModelScope.launch {
        _movieState.value = MovieState.Loading
        try {
            val isRefreshed = repository.refreshMovies(forceRefresh)
            val currentMovies = repository.getMovies().first()
            _movieState.value = MovieState.Success(currentMovies, isRefreshed)
        } catch (e: Exception) {
            val hasData = repository.getMovies().first().isEmpty()
            _movieState.value = MovieState.Error(
                "Failed to load movies: ${e.message ?: "Unknown error"}",
                hasData
            )
            Timber.e(e, "Error loading movies")
        }
    }
}
```

- Saat dipanggil:
  - Set state ke Loading
  - Coba refresh data dari API
  - Jika sukses → ambil dari database, lalu set Success
  - Jika gagal → ambil data lokal dan set Error dengan hasData = true/false

### **Live Movie List**

```
val movieList: StateFlow<List<Movie>> = repository.getMovies().stateIn(...)
```

- Data dari Room dibungkus jadi StateFlow agar bisa di-observe real-time di Compose UI.

### **Alur Lengkap: Dari ViewModel → API/DB → UI**

1. **UI (Compose)** memanggil viewModel.loadMovies()
2. ViewModel set MovieState.Loading
3. Repository memutuskan:
  - Jika cache valid: ambil dari DB
  - Jika perlu refresh: ambil dari API, simpan ke DB
4. Setelah sukses, ViewModel set MovieState.Success
5. Jika error:
  - Tetap load dari DB jika ada
  - Set MovieState.Error
6. **UI** mereaksi pada perubahan movieState dan movieList via collectAsState() di Compose

## **B. KotlinX Serialization**

### **1. Penyiapan Konverter KotlinX Serialization**

Pada RetrofitInstance.kt:

```
val json = Json {
    ignoreUnknownKeys = true
    isLenient = true
    coerceInputValues = true
}
```

#### **Penjelasan:**

- ignoreUnknownKeys = true: jika respons JSON mengandung field yang **tidak didefinisikan** di model Kotlin, akan **diabaikan** (tidak error).

- `isLenient = true`: memperbolehkan format JSON yang tidak sepenuhnya strict.
- `coerceInputValues = true`: jika ada field yang nilainya tidak valid, maka akan diganti dengan nilai default.

`.addConverterFactory(json.asConverterFactory("application/json".toMediaType()))`

Ini menghubungkan **KotlinX Serialization** sebagai **converter JSON** ke Retrofit.

## 2. Model Data: Movie dan MovieResponse

**@Serializable**

**@Serializable**

`data class Movie(...)`

Menandakan bahwa data class `Movie` dan `MovieResponse` **bisa diserialisasi dan deserialisasi** oleh KotlinX Serialization.

**Contoh dari API TMDb (disingkat):**

`json`

`SalinEdit`

```
{
  "page": 1,
  "results": [
    {
      "id": 123,
      "title": "Example",
      "overview": "Some description",
      "poster_path": "/abc.jpg",
      ...
    }
  ],
  "total_pages": 100,
  "total_results": 2000
}
```



```
}
```

Akan diubah menjadi:

```
MovieResponse(  
    page = 1,  
    results = listOf(  
        Movie(  
            id = 123,  
            title = "Example",  
            overview = "Some description",  
            posterPath = "/abc.jpg",  
            ...  
        )  
    ),  
    totalPages = 100,  
    totalResults = 2000  
)
```

### 3. Mapping Nama JSON ke Properti Kotlin

```
@SerializedName("poster_path")
```

```
@ColumnInfo(name = "poster_path")
```

```
val posterPath: String?,
```

- `@SerializedName("poster_path")` → memberitahu KotlinX Serialization bahwa JSON `poster_path` harus dipetakan ke properti Kotlin `posterPath`.
- Ini **penting karena JSON menggunakan snake\_case**, sedangkan Kotlin menggunakan camelCase.

### 4. Integrasi dengan Room

```
@Entity(tableName = "movies")
```

data class Movie(...)

- Karena Movie juga akan disimpan di **Room Database**, maka digunakan juga:
  - @Entity → supaya bisa jadi tabel
  - @ColumnInfo → jika nama kolom Room ingin diubah

Artinya **1 data class Movie digunakan untuk 3 hal sekaligus**:

1. Parsing dari JSON (via KotlinX Serialization)
2. Penyimpanan di Room
3. Penggunaan logika di ViewModel/UI

## 5. Alur Deserialisasi Sebenarnya

Saat **MovieRepository.refreshMovies()** dipanggil:

```
val response = apiService.getPopularMovies(...)
```

- Retrofit menerima response berupa JSON dari API.
- Converter dari RetrofitInstance akan mem-parsing JSON ke objek MovieResponse, yang di dalamnya ada List<Movie>.
- Semua itu dilakukan secara otomatis oleh KotlinX Serialization **berdasarkan struktur dan anotasi @Serializable**.

Dengan KotlinX Serialization:

- JSON dari TMDB langsung diubah menjadi MovieResponse dan List<Movie> otomatis
- Nama-nama field JSON seperti poster\_path, vote\_average, dll. bisa langsung dipetakan ke field Kotlin posterPath, voteAverage menggunakan @SerializedName
- Tidak perlu membuat DTO (data transfer object) tambahan
- Parsing lebih cepat dan lebih ringan daripada Gson

## C. Image Loading dengan Coil

### 1. Memanggil Coil dengan rememberAsyncImagePainter

```
painter = rememberAsyncImagePainter(  
    model = "https://image.tmdb.org/t/p/w500${movie.posterPath}"  
)
```

- Fungsi `rememberAsyncImagePainter()` adalah **pintu masuk utama untuk memuat gambar secara asinkron di Jetpack Compose menggunakan Coil**.
- Parameter `model` diisi dengan URL gambar dari TMDB API.
- Fungsi ini bersifat **@Composable** dan menggunakan **remember** agar tidak terus-menerus reload saat recomposition.

## 2. Mengambil Gambar dari Internet

- Coil melakukan **permintaan HTTP ke URL** tersebut menggunakan `OkHttp` (library HTTP yang digunakan Coil).
- Saat memuat gambar:
  - Jika gambar belum ada di cache, Coil akan mengunduhnya dari internet.
  - Jika sudah ada di memori/disk cache, Coil akan langsung menampilkannya tanpa unduh ulang.

## 3. Caching Otomatis

- Coil **secara otomatis menyimpan gambar ke cache memori dan disk**, jadi kamu tidak perlu mengelola caching sendiri.
- Ini membantu efisiensi aplikasi — tidak perlu mengunduh ulang saat gambar ditampilkan kembali.

## 4. Menampilkan Gambar di UI

```
Image(  
    painter = rememberAsyncImagePainter(...),  
    contentDescription = movie.title,  
    modifier = Modifier
```

```

        .fillMaxWidth()
        .height(200.dp)
        .clip(RoundedCornerShape(8.dp)),
        contentScale = ContentScale.Crop
    )

```

- Gambar yang dimuat dari `rememberAsyncImagePainter` ditampilkan dengan `Image()`.
- `contentScale = ContentScale.Crop` memastikan gambar memenuhi lebar container dan dipotong sesuai ukuran.
- `clip(RoundedCornerShape(8.dp))` membuat sudut gambar membulat.

## D. Penggunaan API TMDB

### 1. API Request dengan Retrofit (TmdbApiService)

```

@GET("movie/popular")
suspend fun getPopularMovies(
    @Query("api_key") apiKey: String,
    @Query("language") language: String = "en-US",
    @Query("page") page: Int = 1
): MovieResponse

```

- **Endpoint:** `/movie/popular`
- Menggunakan **@GET Retrofit** untuk memanggil data film populer dari TMDB.
- Parameter:
  - `api_key`: API Key dari TMDB
  - `language`: default en-US
  - `page`: untuk pagination

### 2. Deserialisasi JSON dengan Kotlinx Serialization

Respons TMDB berupa JSON seperti:

```
{
  "page": 1,
  "results": [
    {
      "id": 123,
      "title": "Movie Title",
      "overview": "...",
      "poster_path": "/abc.jpg",
      "vote_average": 8.5,
      "release_date": "2023-01-01",
      "popularity": 123.45,
      ...
    }
  ],
  "total_pages": 500,
  "total_results": 10000
}
```

- Kotlinx Serialization otomatis mengubah JSON ini menjadi objek Kotlin:
  - MovieResponse untuk respons utama
  - Movie untuk data individual
- Ini dimungkinkan karena kamu menandai `@Serializable` di data class, dan gunakan `@SerializedName` untuk mencocokkan nama field JSON yang berbeda dari nama properti Kotlin.

### 3. Pengelolaan Data di Repository (MovieRepository)

```
val response = apiService.getPopularMovies(...)
```

- Repository mengelola:

- **Pengambilan data dari API**
- **Caching ke Room (local database)**
- **Pagination & validasi cache**
- Data dari response.results diubah menjadi Movie yang cocok untuk Room, lalu:

movieDao.insertMovies(entities)

- Kemudian disimpan ke lokal (Room DB).

#### **4. Room DAO Menyimpan dan Mengambil Data**

Room DAO seperti:

@Query("SELECT \* FROM movies")

fun getAllMovies(): Flow<List<Movie>>

- Digunakan untuk mengakses cache lokal sebagai Flow, agar bisa dipantau secara reaktif (otomatis update di UI saat data berubah).

#### **5. Flow Data ke ViewModel (MovieViewModel)**

ViewModel menyimpan movieList dan localMovies sebagai:

val movieList: StateFlow<List<Movie>>

val localMovies: StateFlow<List<Movie>>

- ViewModel menerima data dari Repository, dan meneruskannya ke UI dengan cara yang bisa di-*observe*.

#### **6. Tampilkan ke UI (Compose)**

**Contoh di MovieCard dan MovieDetailScreen:**

Image(

painter = rememberAsyncImagePainter(

model = "https://image.tmdb.org/t/p/w500\${movie.posterPath}"

),

...  
)

Text(text = movie.title)

Text(text = movie.overview)

- Data ditampilkan sebagai UI card atau detail screen.
- Gambar di-load dari TMDB dengan Coil.
- Data lain seperti title, voteAverage, releaseDate langsung di-bind ke komponen UI.

## **E. Data Persistence**

### **1. SharedPreferences (PreferencesManager)**

#### **Tujuan:**

Untuk menyimpan **data ringan dan sederhana**, seperti:

- Username terakhir yang login
- ID film terakhir yang dilihat

#### **Alur Penggunaan:**

##### **Simpan Data ke SharedPreferences**

Contoh: saat pengguna melihat detail film

```
preferencesManager.saveLastViewedMovieId(movieId)
```

- Disimpan dalam file XML lokal di device
- Hanya key-value pair (string, int, boolean, dll)

##### **Ambil Data dari SharedPreferences**

Contoh: menampilkan username atau film terakhir

```
preferencesManager.getUsername()
```

```
preferencesManager.getLastViewedMovieId()
```

- Dipanggil oleh MovieViewModel saat ingin menampilkan data personal pengguna.

## 2. Room Database (MovieDao + AppDatabase)

### Tujuan:

Menyimpan dan mengambil **data film** secara permanen ke dalam SQLite Database.

### 1. Menyimpan film dari TMDB ke Room

Di dalam MovieRepository:

```
movieDao.insertMovies(entities)
```

- Setelah memanggil API TMDB, respons di-*map* jadi Movie dan disimpan ke DB.
- Pakai OnConflictStrategy.REPLACE untuk memperbarui data jika sudah ada.

### 2. Mengambil semua film dari database

```
movieDao.getAllMovies(): Flow<List<Movie>>
```

- Di-*observe* sebagai Flow dalam MovieViewModel, lalu dikirim ke UI.
- Otomatis update saat data berubah (misalnya saat refresh TMDB API).

### 3. Cek apakah cache kedaluwarsa

```
movieDao.getLastUpdateTime()
```

- Digunakan oleh MovieRepository untuk mengecek kapan terakhir data diperbarui.

### 4. Menyimpan film favorit ke lokal (mode offline)

```
movieDao.insertMovies(listOf(movie.copy(isLocal = true)))
```

- User bisa menyimpan film tertentu secara lokal.

### 5. Mengambil film lokal (favorit)

```
movieDao.getLocalMovies(): Flow<List<Movie>>
```

## Integrasi ViewModel

ViewModel menyatukan semuanya:



### **Ambil data dari Room:**

```
val movieList = repository.getMovies().stateIn(...)
```

### **Simpan ke Room:**

```
repository.saveMovieToLocal(movie)
```

### **Ambil data kecil dari SharedPreferences:**

```
preferencesManager.getUsername()
```

```
preferencesManager.getLastViewedMovieId()
```

## **F. Caching Strategy pada Room**

### **1. Ambil Data dari Room (cache) terlebih dahulu**

```
movieDao.getAllMovies()
```

- MovieViewModel menggunakan getMovies() yang me-*return* Flow<List<Movie>>
- Data ini berasal dari Room dan digunakan untuk menampilkan film di UI secara reaktif

### **2. Cek apakah cache perlu diperbarui**

#### **Metode utama:**

```
suspend fun refreshMovies(forceRefresh: Boolean = false): Boolean
```

#### **Langkah-langkah pengecekan:**

```
val shouldRefresh = forceRefresh || isCacheExpired() || cachedMovies.isEmpty()
```

- forceRefresh: Refresh paksa (misal: user swipe to refresh)
- isCacheExpired(): Cek apakah waktu simpan data melebihi batas 30 menit
- cachedMovies.isEmpty(): Tidak ada data di DB sama sekali

### **3. Cek Kedaluwarsa Cache**

```
private suspend fun isCacheExpired(): Boolean {
```

```

return movieDao.getLastUpdateTime()?.let { lastUpdated ->
    System.currentTimeMillis() - lastUpdated > CACHE_EXPIRATION_TIME_MS
} ?: true
}

```

- Jika `last_updated` setiap film sudah lebih dari 30 menit, maka dianggap kadaluarsa

#### 4. Jika perlu, ambil data baru dari API

```
val response = apiService.getPopularMovies(...)
```

- Data dari API langsung di-*map* ke entitas `Movie` lokal, lalu disimpan:

```
movieDao.insertMovies(entities)
```

#### 5. Bersihkan data lama jika `forceRefresh = true`

```

if (forceRefresh) {
    movieDao.clearAll()
}

```

#### 6. Update `lastUpdated` saat simpan ke DB

```
lastUpdated = System.currentTimeMillis()
```

- Disimpan dalam properti `last_updated` di entitas `Movie`
- Ini yang digunakan sebagai indikator umur data

#### 7. Gunakan data cache dari Room sebagai default

Jika API gagal, `MovieViewModel` masih bisa mengambil dari:

```
repository.getMovies().first()
```

#### **D. Tautan Git**

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul5>